

Chapter 3

Transport Layer

A note on the use of these ppt slides:

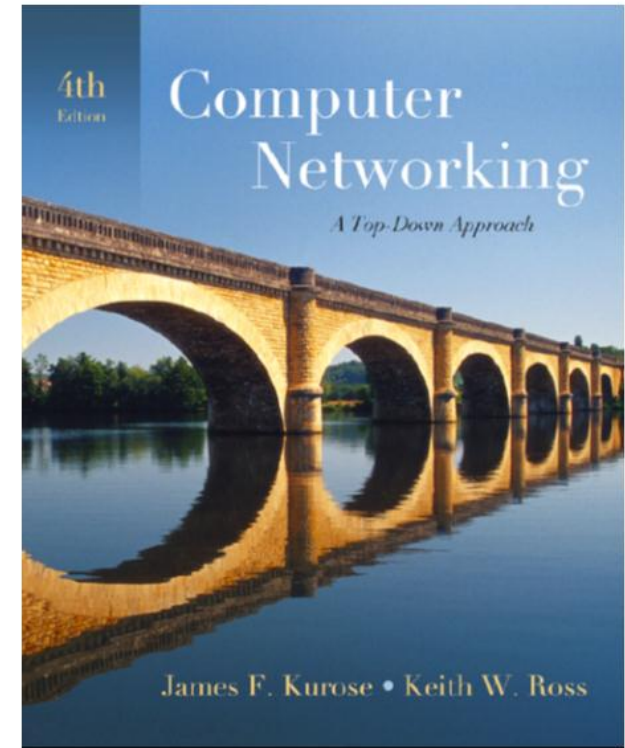
We're making these slides freely available to all (faculty, students, readers). They're in PowerPoint form so you can add, modify, and delete slides (including this one) and slide content to suit your needs. They obviously represent a *lot* of work on our part. In return for use, we only ask the following:

- ❑ If you use these slides (e.g., in a class) in substantially unaltered form, that you mention their source (after all, we'd like people to use our book!)
- ❑ If you post any slides in substantially unaltered form on a www site, that you note that they are adapted from (or perhaps identical to) our slides, and note our copyright of this material.

Thanks and enjoy! JFK/KWR

All material copyright 1996-2007

J.F Kurose and K.W. Ross, All Rights Reserved



*Computer Networking:
A Top Down Approach
4th edition.*

*Jim Kurose, Keith Ross
Addison-Wesley, July
2007.*

Chapter 3: Transport Layer

Our goals:

- ❑ understand principles behind transport layer services:
 - multiplexing/demultiplexing
 - reliable data transfer
 - flow control
 - congestion control
- ❑ learn about transport layer protocols in the Internet:
 - UDP: connectionless transport
 - TCP: connection-oriented transport
 - TCP congestion control

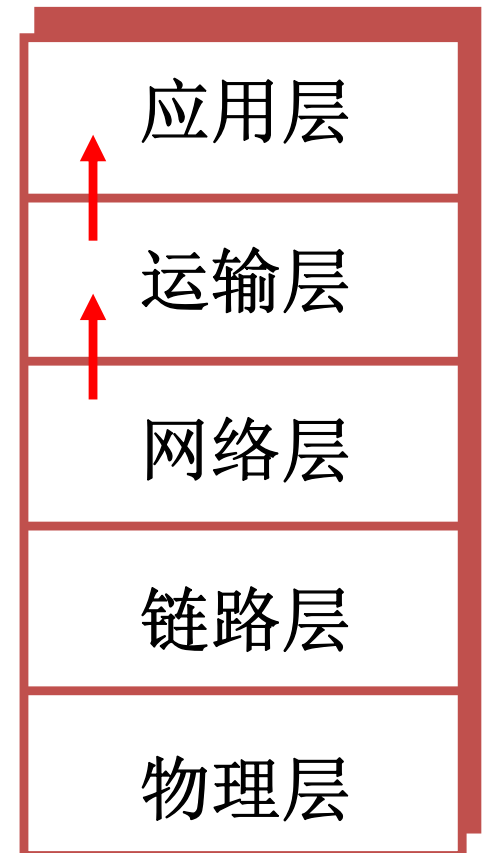
第 3 章 运输层

□ 位于应用层和网络层之间

- 为主机上的应用进程提供直接的通信服务
- 接受网络层提供的服务

□ 主要内容

- 运输层服务
- 运输层和网络层的关系
- 两个实体如何可靠地通信
- 拥塞控制技术

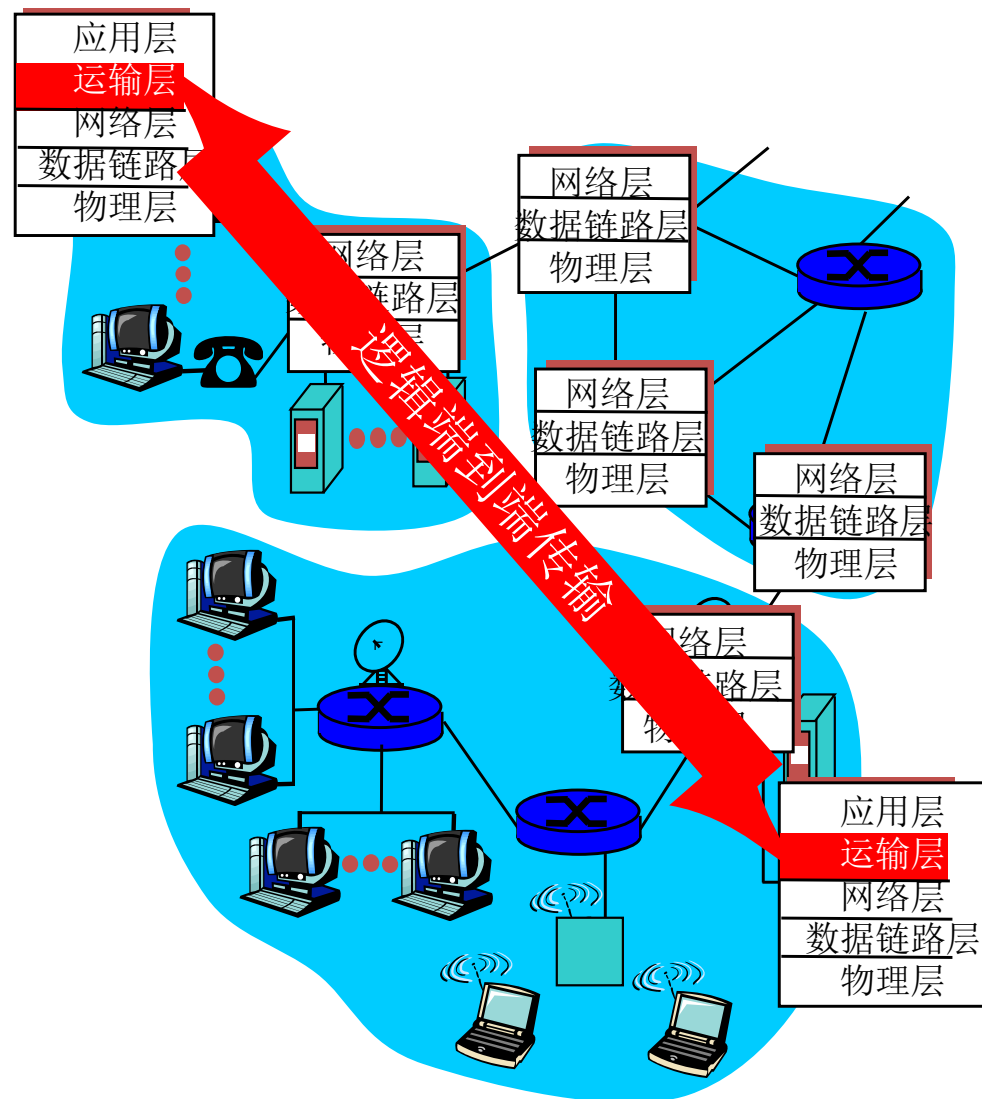


Chapter 3 outline

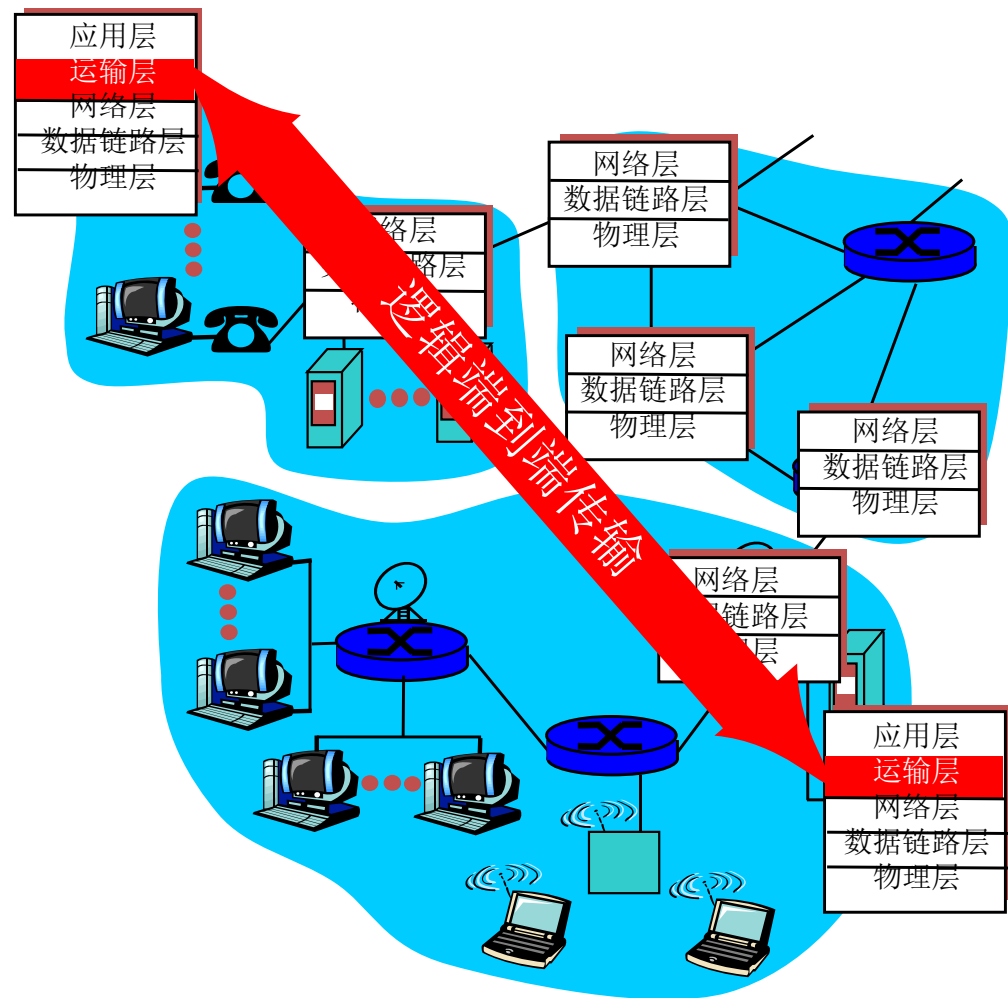
- ❑ 3.1 Transport-layer services
- ❑ 3.2 Multiplexing and demultiplexing
- ❑ 3.3 Connectionless transport: UDP
- ❑ 3.4 Principles of reliable data transfer
- ❑ 3.5 Connection-oriented transport: TCP
 - segment structure
 - reliable data transfer
 - flow control
 - connection management
- ❑ 3.6 Principles of congestion control
- ❑ 3.7 TCP congestion control

3.1 运输层服务

- 运输层协议
 - ✓ 为运行在不同主机上的应用进程提供**逻辑通信**功能（主机好像是直接相连的）
 - ✓ 进程之间**使用逻辑通信功能彼此发送报文**，无需考虑具体物理链路



- ✓ 运输层协议运行在端系统，不在路由器中
- ◆ 发送方：将应用进程的报文划分成若干段，封装后传给网络层
- ◆ 接收方：将网络层上传的报文段，重新装配为报文，传向应用层
- ✓ 路由器只根据网络层字段而动作



3.1.1 运输层和网络层的关系

3.1.2 运输层概述

3.1.1 运输层和网络层的关系

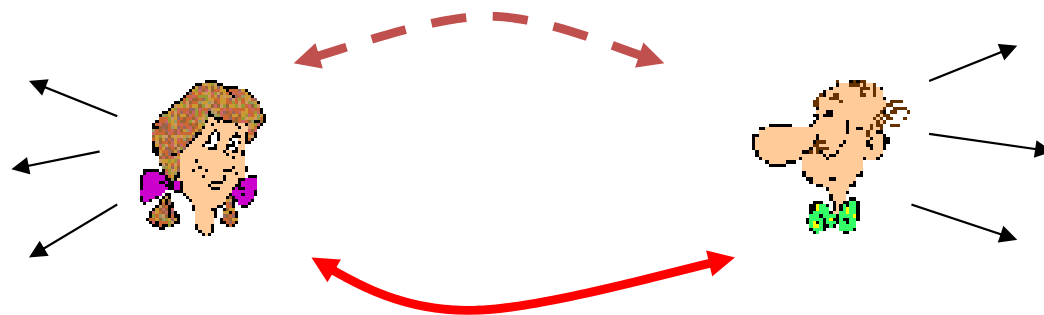
运输层位于网络层上

- ✓ 网络层提供**主机之间的逻辑通信**
- ✓ 运输层为运行在不同主机上的**进程之间提供逻辑通信**
 - 依赖、强化网络层服务

家庭通信类比

两个家庭的堂兄弟姐妹之间互相通信

- ✓ 每家有12个孩子，每家有一个孩子负责收发邮件，分别为Ann和Bill
- ✓ 每封信通过**传统的邮政**服务发送
- ✓ **邮政服务**为两个家庭间提供**逻辑通信**：将信件从一家送往另一家
- ✓ **Ann和Bill**为堂兄弟姐妹之间提供了**逻辑通信**：向兄弟姐妹收取或交付信件
- ✓ Ann和Bill是端到端交付过程的一部分(即端系统部分)，是邮件服务



3.1.1 Transport vs. Network layer

- ❑ *network layer*: logical communication between hosts
- ❑ *transport layer*: logical communication between processes
 - relies on, enhances, network layer services

Household analogy:

12 kids sending letters to 12 kids

- ❑ processes = kids
- ❑ app messages = letters in envelopes
- ❑ hosts = houses
- ❑ transport protocol = Ann and Bill
- ❑ network-layer protocol = postal service

与网络术语对比

应用层	{	应用报文	=	信封中的信
运输层		进程	=	孩子
网络层		运输协议	=	Ann和Bill
链路层		网络层协议	=	邮政服务
物理层				

主机 = 家庭

说明

- ① Ann和Bill 在各自家里工作，不参与邮件分拣及传递
 - ✓ 运输层协议工作在端系统中，将来自应用进程的报文移动到网络边界(网络层)，不考虑报文在网络核心如何传递
 - ✓ 中间路由器既不处理也不识别运输层加在报文上的任何信息
- ② Ann和Bill 外出，其他人接替工作，服务方式、效果可能不同
 - 计算机网络中有多种运输层协议，每种协议为应用程序提供不同的服务模型

③ Ann和Bill能够提供的服务要受到邮政服务提供服务的限制

运输层协议所能提供的服务受底层网络协议服务模型的限制，如，时延和带宽保证

④ 某些特定服务即使底层网络协议不提供，运输层协议也能提供

如，当底层网络协议是不可靠的（分组丢失、混乱和重复），运输层同样能为应用程序提供可靠的传输服务。

3.1.2 因特网运输层概述

□ 因特网运输层协议

- ✓ **UDP(用户数据报协议):** 为应用程序提供不可靠、无连接的服务
- ✓ **TCP(传输控制协议):** 为应用程序提供可靠的、面向连接的服务
- 术语
 - ✓ **报文段 (segment):** 运输层分组
 - ✓ **数据报 (datagram):** 网络层分组

- 因特网网络层

- ✓ IP（网际协议）：为主机之间提供逻辑通信

- ✓ IP服务模型：尽力交付服务

尽最大的努力在通信的主机之间交付报文段，但不保证按序交付或数据的完整性，属于不可靠服务

- ✓ IP地址：主机的网络层地址，网络上的每台主机至少有一个

运输层的多路
复用与分解

- *UDP和TCP的服务模型*

- ✓ 将两个端系统间IP的交付服务扩展为运行在两个端系统上的进程之间的交付服务

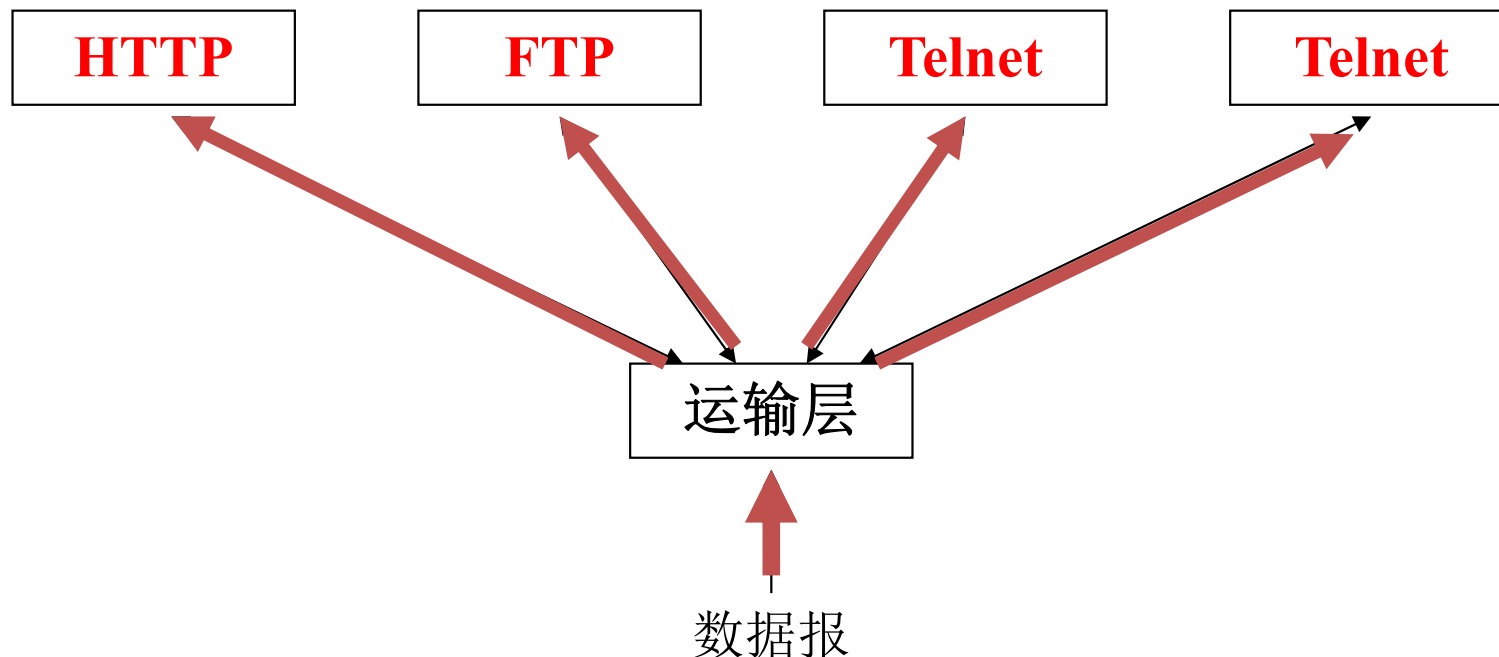
Chapter 3 outline

- ❑ 3.1 Transport-layer services
- ❑ 3.2 Multiplexing and demultiplexing
- ❑ 3.3 Connectionless transport: UDP
- ❑ 3.4 Principles of reliable data transfer
- ❑ 3.5 Connection-oriented transport: TCP
 - segment structure
 - reliable data transfer
 - flow control
 - connection management
- ❑ 3.6 Principles of congestion control
- ❑ 3.7 TCP congestion control

3.2 多路复用与分解

将网络层所提供的**主机到主机**交付服务扩展到在主机上运行的**应用程序到应用程序**的交付服务。

通常，主机上可以有多个应用程序进程运行（如一个HTTP、一个FTP、两个Telnet）。当运输层从底层网络接收数据时，应能正确地定向到相应的一个进程（套接字）



- **多路复用 (发送主机)**

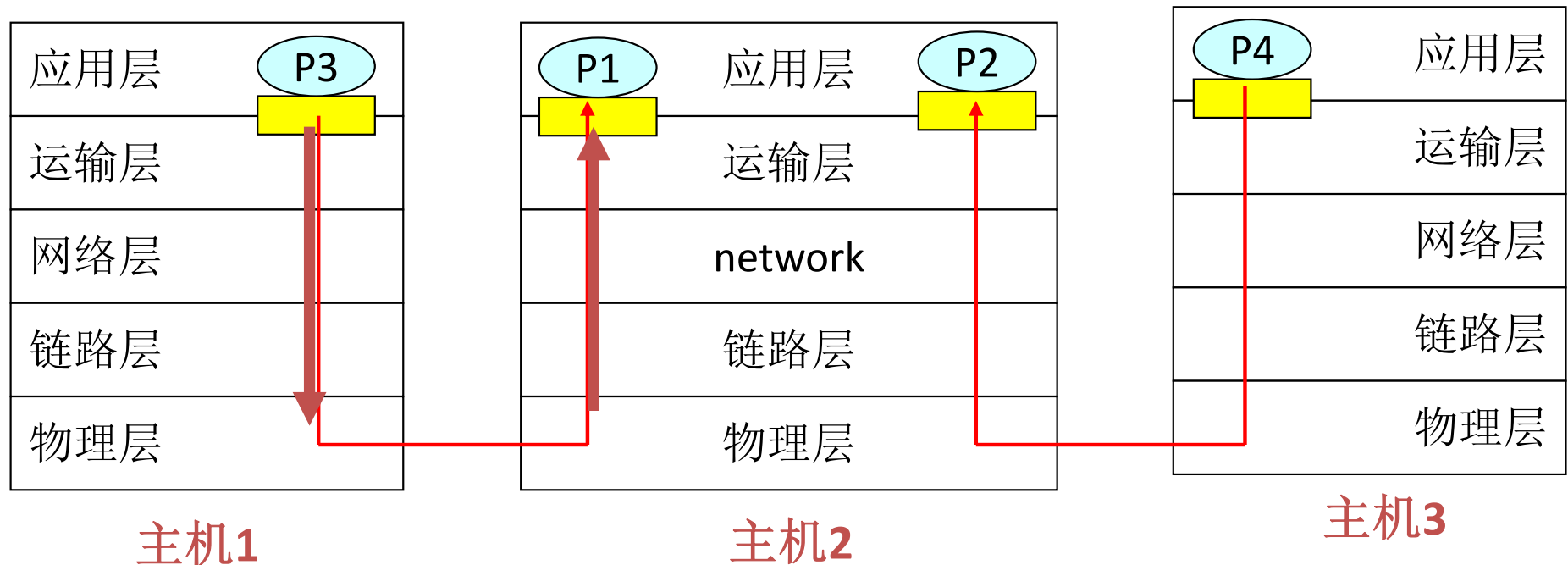
从不同套接字收集数据块，并为每个数据块封装上首部信息，生成报文段，传递到网络层。

- **多路分解 (接收主机)**

将报文段中的数据交付到正确的套接字。即接收方运输层从报文段的多个字段中，识别出套接字，并将报文段定向到该套接字。

Ann和Bill

- ❑ **主机1**上的运输层收集套接字输出的数据，形成运输层报文段，将其传递给下面的网络层（**多路复用**）
- ❑ **主机2**的运输层将从其网络层收到的报文段**多路分解**后通过相应的套接字交给其上的**P1**进程



目的主机的分解过程

- ✓ 数据报到网络层，每个数据报有源IP地址和目的IP地址，每个数据报有一个报文段
- ✓ 当报文段到达运输层时，运输层检查报文段中的目的端口号，将其定向到相应的套接字
- ✓ 报文段中的数据通过套接字进入其所连接的进程

报文段格式

- **端口号**：主机上的每个套接字分配一个端口号
- 16位 (0~65535)
- ✓ 0~1023为周知端口号，保留给固定的应用程序
- ✓ 开发一个新应用时，需选择一个端口号



TCP/UDP 段格式

无连接的多路复用与分解

- **创建UDP套接字**

- ✓ **自动为该套接字分配一个端口号。如**

`clientSocket = socket(socket.AF_INET, socket.SOCK_DGRAM)`

从1024~65535的号码中分配一个主机尚未使用的
UDP端口号

- ✓ **直接为套接字指定一个特定的端口号。如**

`clientSocket.bind(("", 19157))`

- **客户端：自动分配端口号**

- **服务器端：分配一个特定的端口号**

- **若使用“周知协议”，必须分配一个相应的周知端口号**

套接字编程问题汇总

- ❑ 查看已占用端口号： 在命令行下运行netstat -ano
- ❑ 先开服务器端再运行客户端；服务器端的代码要一直运行

```
C:\WINDOWS\system32\cmd.exe
C:\Users\Min>netstat -no

活动连接

 协议 本地地址          外部地址          状态          PID
TCP    127.0.0.1:49839    127.0.0.1:5357    TIME_WAIT     0
TCP    127.0.0.1:49844    127.0.0.1:5357    TIME_WAIT     0
TCP    127.0.0.1:49847    127.0.0.1:5357    TIME_WAIT     0
TCP    127.0.0.1:49859    127.0.0.1:5357    TIME_WAIT     0
TCP    192.168.1.107:49830 23.43.75.27:80    TIME_WAIT     0
TCP    192.168.1.107:49831 117.18.237.29:80  TIME_WAIT     0
TCP    192.168.1.107:49836 104.85.195.76:443 ESTABLISHED    17544
TCP    192.168.1.107:49838 58.144.136.61:443 TIME_WAIT     0
TCP    192.168.1.107:49840 220.194.91.159:80 TIME_WAIT     0
TCP    192.168.1.107:49843 157.255.174.105:80 TIME_WAIT     0
TCP    192.168.1.107:49845 157.255.174.101:80 TIME_WAIT     0
TCP    192.168.1.107:49861 58.251.111.101:80 TIME_WAIT     0
TCP    192.168.1.107:49862 58.251.100.119:8080 ESTABLISHED    24348
TCP    192.168.1.107:49863 58.247.204.145:8080 TIME_WAIT     0
TCP    192.168.1.107:49865 140.205.218.11:443 TIME_WAIT     0
TCP    192.168.1.107:49872 52.114.128.9:443  ESTABLISHED    11040
TCP    192.168.1.107:49875 52.184.92.48:80   ESTABLISHED    8640
TCP    192.168.1.107:49877 104.85.195.76:80  TIME_WAIT     0
TCP    192.168.1.107:49886 47.56.121.40:80   CLOSE_WAIT    15720
TCP    192.168.1.107:49888 113.207.34.228:80 ESTABLISHED    15720
TCP    192.168.1.107:49889 58.247.205.183:80 ESTABLISHED    24348
TCP    192.168.1.107:49890 64.233.187.188:443 SYN_SENT      992
TCP    192.168.1.107:49891 52.114.132.74:443 TIME_WAIT     0
TCP    192.168.1.107:49892 220.194.122.199:443 ESTABLISHED    15424
TCP    192.168.1.107:49893 13.33.142.168:443 ESTABLISHED    992
TCP    192.168.1.107:49894 172.217.160.106:443 SYN_SENT      992
TCP    192.168.1.107:49895 172.217.160.106:443 SYN_SENT      992
TCP    192.168.1.107:49896 58.251.121.55:8000 SYN_SENT      15424
TCP    192.168.1.107:64540 202.108.23.113:80 ESTABLISHED    13988
TCP    192.168.1.107:64559 112.80.248.85:5287 ESTABLISHED    13988
TCP    192.168.1.107:64613 52.139.250.253:443 ESTABLISHED    6024
TCP    192.168.1.107:64618 58.251.80.102:443 TIME_WAIT     0
TCP    192.168.1.107:64672 58.251.81.32:443  ESTABLISHED    992
TCP    192.168.1.107:64699 58.251.81.32:443  ESTABLISHED    992
```

说明

- ✓ UDP套接字组成：二元组（IP地址，端口号）
- ✓ 两个具有不同源套接字，但具有相同目的套接字的UDP报文段，可通过相同套接字定向到相同的目的进程（多对一）
- ✓ 源端口号的用途：目的方回发报文段中的“返回地址”

面向连接的多路复用与分解

- **TCP连接套接字**

四元组 (源IP地址, 源端口号, 目的IP地址, 目的端口号)

- ✓ 目的主机使用全部四个值来将收到的TCP报文段定向(分解)到相应的套接字
- ✓ 两个具有**不同源套接字**的TCP报文段, 将被**定向到两个不同的套接字**

TCP连接及分解

- ✓ **服务器应用程序**在某个端口上等待TCP客户机连接建立请求，当服务器收到客户机的连接请求后，通知服务器进程，并**创建一个连接套接字**，如

```
connectionSocket, addr = serverSocket.accept()
```

- ✓ **TCP客户机**在相应端口上产生一个连接建立请求报文段

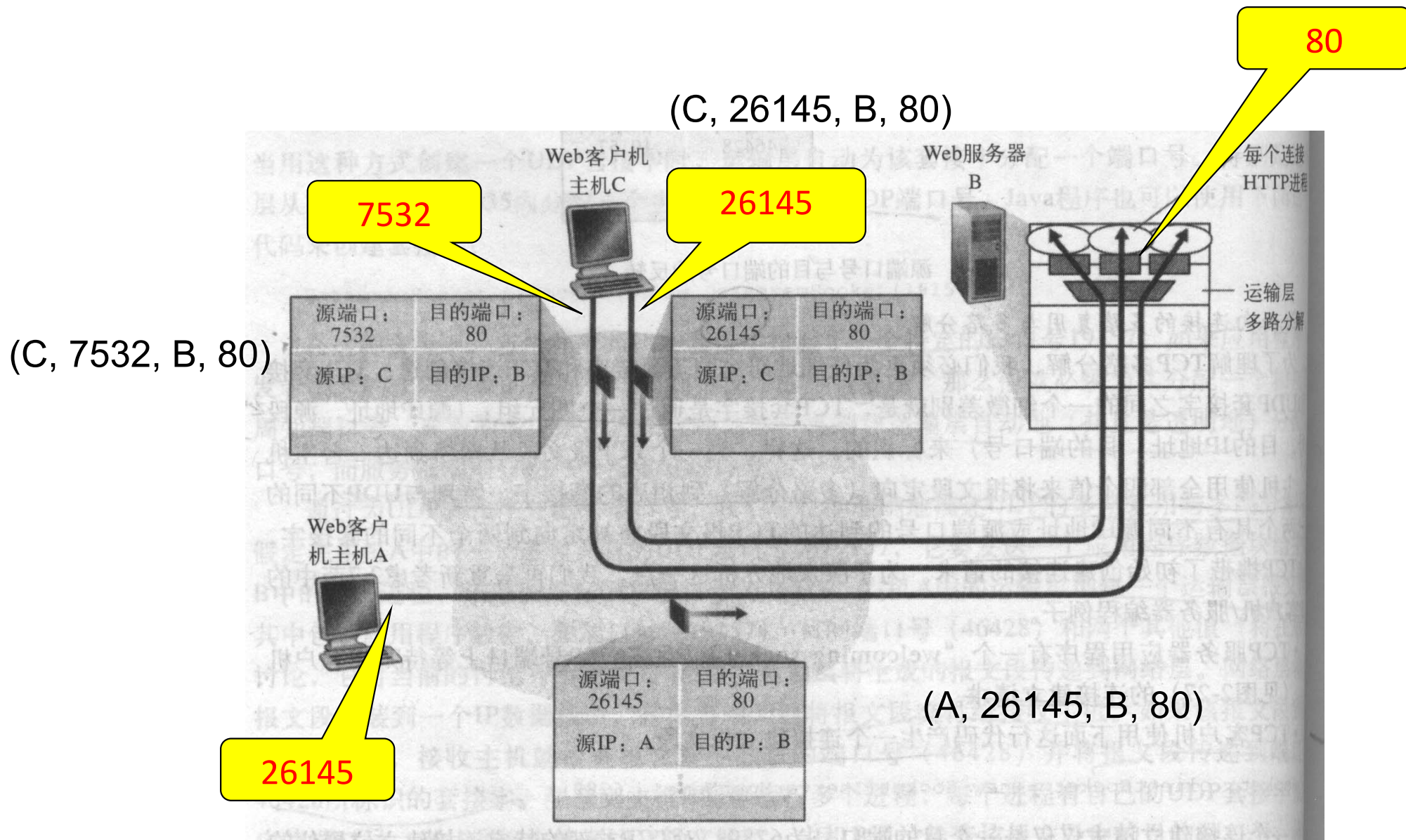
```
clientSocket = socket(AF_INET, SOCK_STREAM)
```

```
clientSocket.connect((serverName,serverPort))
```

- ✓ 连接套接字通过4个值来标识
- ✓ TCP连接完成后，客户机和服务器可相互发送数据
- ✓ 当一个TCP报文段到达主机时，根据4个字段值的来定向(**多路分解**)到相应的套接字

例

主机C向服务器B发起两个HTTP会话
主机A向服务器B发起一个HTTP会话



谢谢大家！

Chapter 3

Transport Layer

A note on the use of these ppt slides:

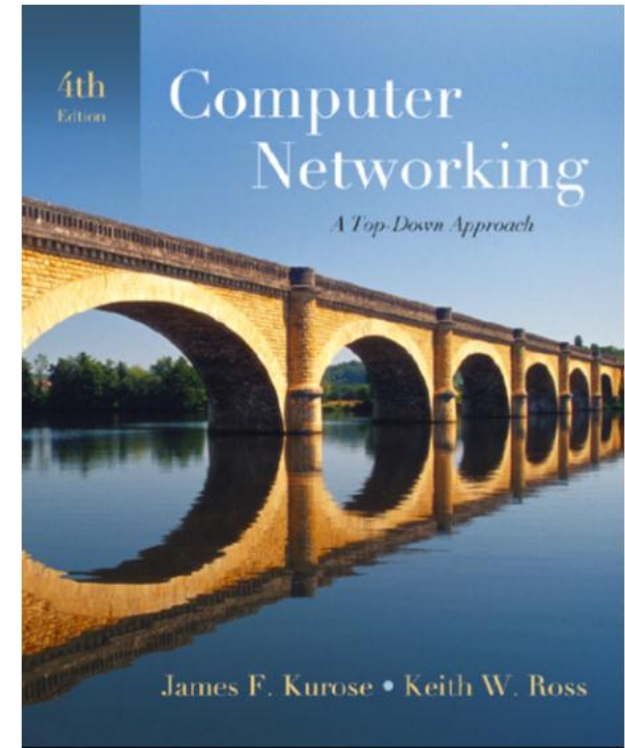
We're making these slides freely available to all (faculty, students, readers). They're in PowerPoint form so you can add, modify, and delete slides (including this one) and slide content to suit your needs. They obviously represent a *lot* of work on our part. In return for use, we only ask the following:

- ❑ If you use these slides (e.g., in a class) in substantially unaltered form, that you mention their source (after all, we'd like people to use our book!)
- ❑ If you post any slides in substantially unaltered form on a www site, that you note that they are adapted from (or perhaps identical to) our slides, and note our copyright of this material.

Thanks and enjoy! JFK/KWR

All material copyright 1996-2007

J.F Kurose and K.W. Ross, All Rights Reserved



*Computer Networking:
A Top Down Approach
4th edition.*

*Jim Kurose, Keith Ross
Addison-Wesley, July
2007.*

Chapter 3: Transport Layer

Our goals:

- ❑ understand principles behind transport layer services:
 - multiplexing/demultiplexing
 - reliable data transfer
 - flow control
 - congestion control
- ❑ learn about transport layer protocols in the Internet:
 - UDP: connectionless transport
 - TCP: connection-oriented transport
 - TCP congestion control

Chapter 3 outline

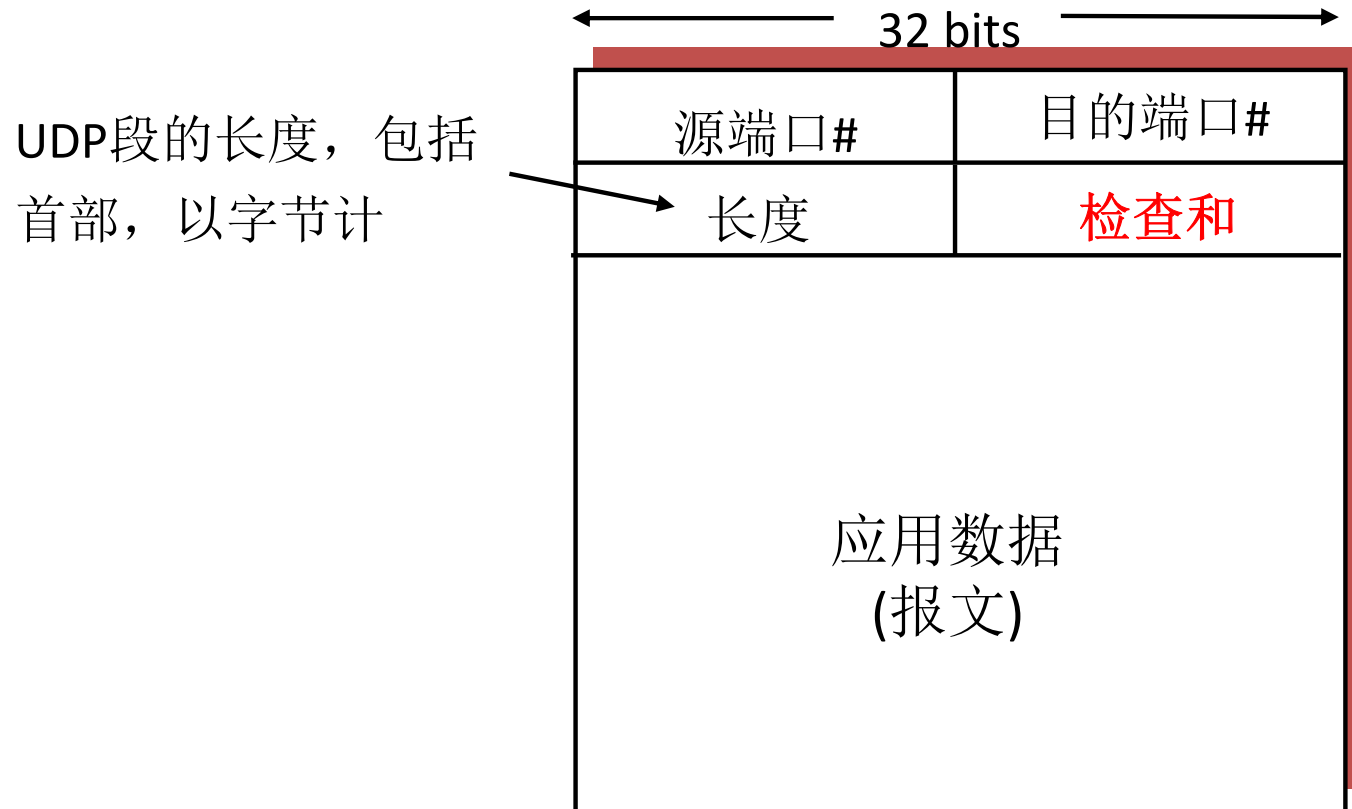
- ❑ 3.1 Transport-layer services
- ❑ 3.2 Multiplexing and demultiplexing
- ❑ 3.3 Connectionless transport: UDP
- ❑ 3.4 Principles of reliable data transfer
- ❑ 3.5 Connection-oriented transport: TCP
 - segment structure
 - reliable data transfer
 - flow control
 - connection management
- ❑ 3.6 Principles of congestion control
- ❑ 3.7 TCP congestion control

3.3 无连接运输：UDP

- “**尽力而为**”服务，UDP段可能：
 - 丢包
 - 对应用程序交付失序
- **无连接**：
 - 在UDP发送方和接收方之间无握手
 - 每个UDP段的处理独立于其他段
- **使用 UDP协议的原因**
 - ✓ 无连接创建(减少时延)
 - ✓ 简单：无连接
 - ✓ 段首部小
 - ✓ 无拥塞控制：UDP能够尽可能快地传输
- 应用常用于
 - 流式多媒体
 - DNS

经UDP的可靠传输：在应用层增加可靠性，实现特定的差错恢复！

UDP报文段格式



UDP 段格式

UDP检查和 (Checksum)

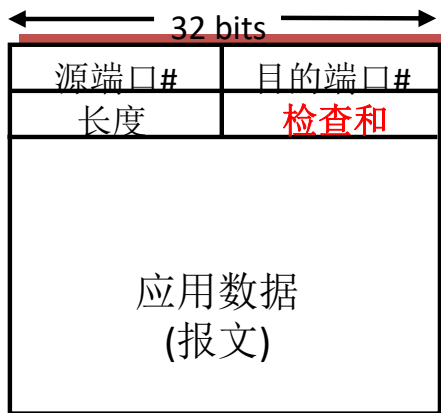
用于检查所传输的报文段中的比特差错 (如比特翻转)。

□ 发送方

- ✓ 将数据的每两个字节当作一个16位的整数，可分成若干整数
- ✓ 将所有16位的整数求和
- ✓ 对得到的和逐位取反，作为检查和，放在报文段首部，一起发送

□ 接收方

- ✓ 对接收到的数据按与发送方相同的方法求和
- ✓ 然后加上检查和
- ✓ 全“1”：收到的数据无差错
- ✓ 其中有“0”：收到的数据出现差错



例子

- **注意**
 - 当数字作加法时，**最高位的进位要回加到结果中。**
- 例，应用数据（报文）中有三个16 比特的字：

回卷

0110011001100000
0101010101010101
1000111100001100
<u>1010010101100000</u>
1010010101100000

和

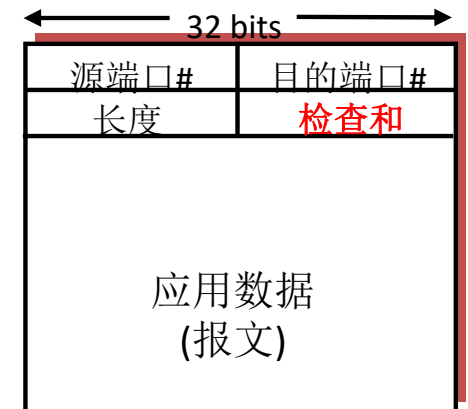
0100101011000010

发送方检查和（取反）

1011010100111101

接收方检测（用上述方法再加检查和）

1111111111111111



UDP抓包分析

```
> Frame 1570: 89 bytes on wire (712 bits), 89 bytes captured (712 bits) on interface 0
> Ethernet II, Src: 52:ce:c2:12:8b:ef (52:ce:c2:12:8b:ef), Dst: HuaweiTe_3e:8f:cf (94:db:da:3e:8f:cf)
> Internet Protocol Version 4, Src: 10.175.139.213, Dst: 123.151.78.110
> User Datagram Protocol, Src Port: 4015, Dst Port: 8000
> Data (47 bytes)
```

```
User Datagram Protocol, Src Port: 4015, Dst Port: 8000
```

```
Source Port: 4015
```

#源端口

```
Destination Port: 8000
```

#目标端口

```
Length: 55
```

#数据报长度

```
Checksum: 0x5690 [unverified]
```

#校验和

```
[Checksum Status: Unverified]
```

```
[Stream index: 171]
```

https://blog.csdn.net/weixin_43871760

```
▼ Data (47 bytes)
```

```
Data: 023803014b32e1310e73a002000000012e01000069022a59...
```

```
[Length: 47]
```

Chapter 3 outline

- ❑ 3.1 Transport-layer services
- ❑ 3.2 Multiplexing and demultiplexing
- ❑ 3.3 Connectionless transport: UDP
- ❑ 3.4 Principles of reliable data transfer
- ❑ 3.5 Connection-oriented transport: TCP
 - segment structure
 - reliable data transfer
 - flow control
 - connection management
- ❑ 3.6 Principles of congestion control
- ❑ 3.7 TCP congestion control

3.4 可靠数据传输的原理

- **可靠数据传输**的问题在运输层、链路层及应用层都会出现。
 - 最重要的10个网络主题中之一!

- ✓ 只考虑**单向数据传输**，数据传输从发送方到接收方
- ✓ 发送方和接收方可以来回交换控制分组
- ✓ 使用有限状态机 (FSM)来定义发送方和接收方

3.4.1 构造可靠的数据传输协议

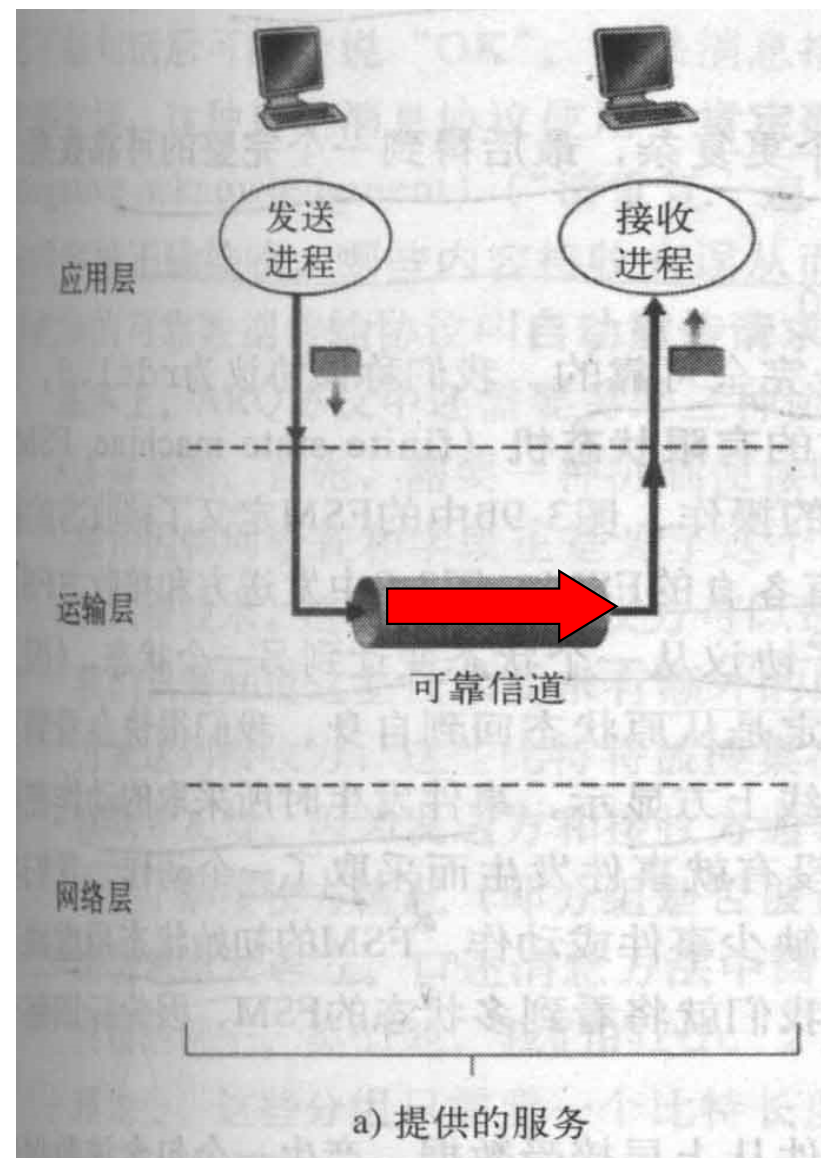
3.4.2 流水线可靠数据传输协议

3.4.3 Go-Back-N(GBN)

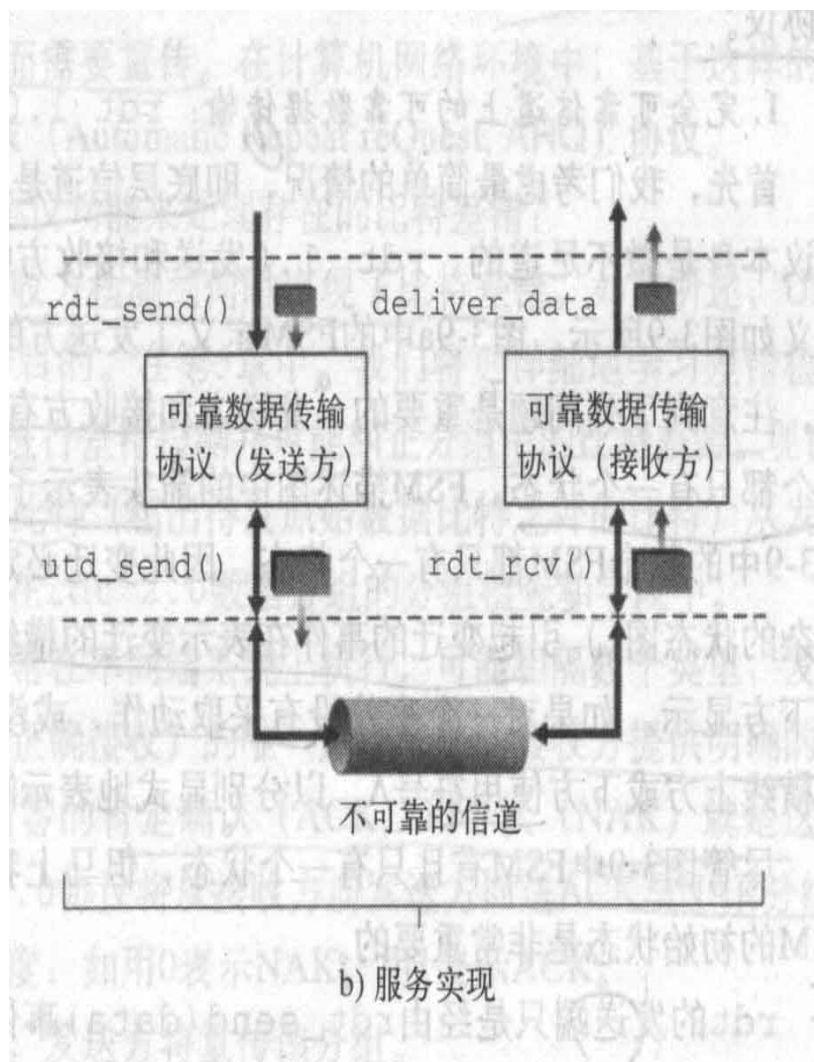
3.4.4 选择性重传 (SR)

可靠数据传输的框架

- 提供给上层的服务
 - ✓ 数据可以通过一条可靠的信道传输
 - ✓ 即传输的数据不出错、丢失，并按照发送顺序传送
- ✓ 由于现实存在的各种干扰，低层信道可能不可靠，通过使用可靠数据传输协议来保证可靠的数据传输
- ✓ 需要分别设计开发可靠数据传输协议的发送方和接收方



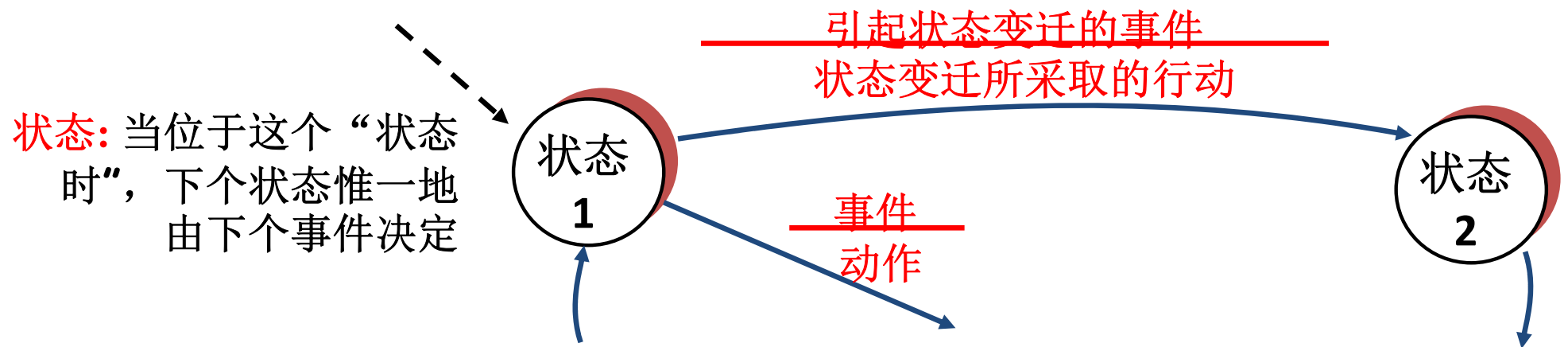
- **发送方**
- ✓ **udt_send()**用来发送分组给对方
(**udt**表示“不可靠数据传输”)
- ✓ 通过**rdt_send()**函数调用可靠数据传输协议的发送方 (**rdt**表示“可靠数据传输”协议)



- **接收方**
- ✓ 当一个分组从信道抵达时，调用**rdt_rcv()****接收**
- ✓ 调用**deliver_data()**将数据交付给上层

有限状态机 (FSM)

- ✓ **圆圈**表示状态
- ✓ **箭头**表示协议从一个状态变迁到另一个状态
- ✓ **横线上方**：表示引起变迁的事件
- ✓ **横线下方**：表示事件发生时所采取的动作
- ✓ **Λ** ：表示没有事件或未采取动作
- ✓ **虚线箭头**指向FSM的初始状态



3.4.1 构造可靠的数据传输协议

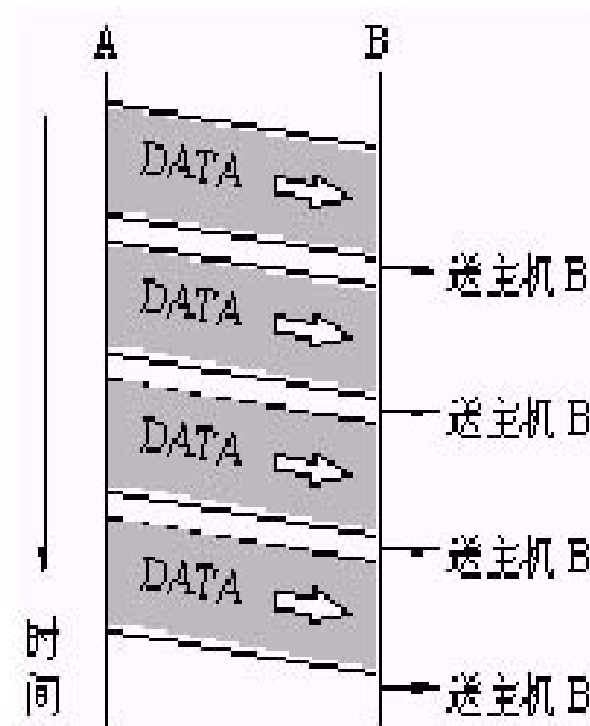
1. 完全可靠信道上的可靠数据传输
2. 具有比特差错信道上的可靠的数据传输
3. 具有比特差错的丢包信道上的可靠的数据传输
4. 停止等待协议流程

1、可靠信道上的可靠数据传输： rdt 1.0

假设

- ✓ 底层信道完全可靠，数据传送不出错不丢失
- ✓ 收发双方速率匹配：接收方接收数据的速率和发送方发送数据一样快

- ✓ 所有的分组都是从发送方流向接收方
- ✓ 接收方不需反馈信息给发送方，因为不会发生任何差错。

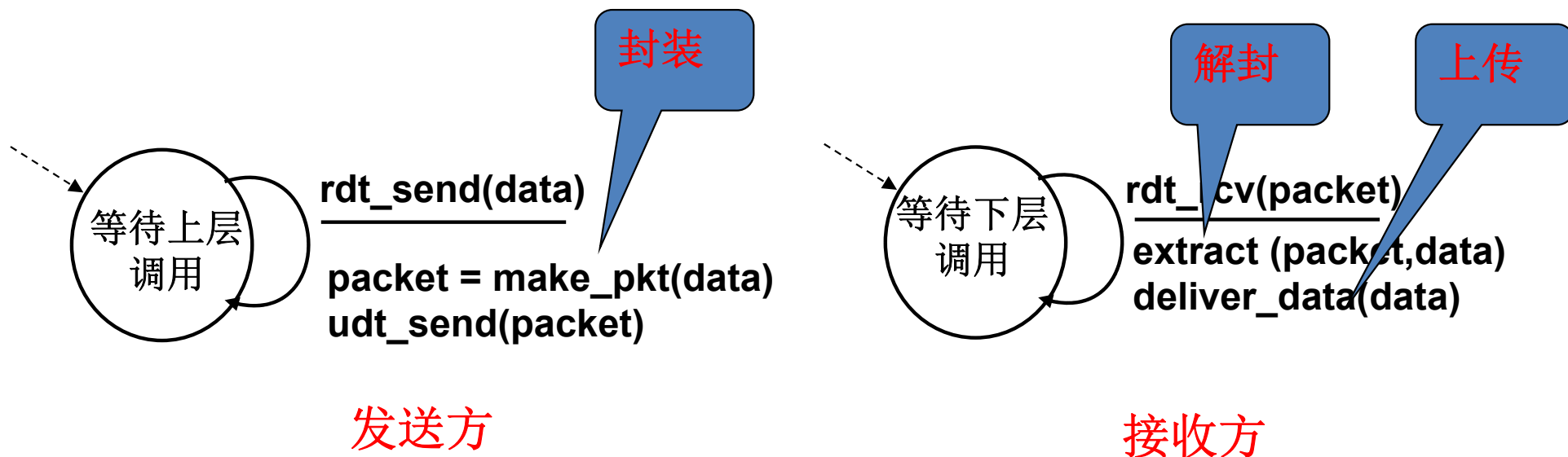


- **发送方：将数据发向底层信道**

从上层接受数据`rdt_send(data)` → 封装成分组`make_pkt(data)` → 将分组发送到信道中`udt_send(packet)`

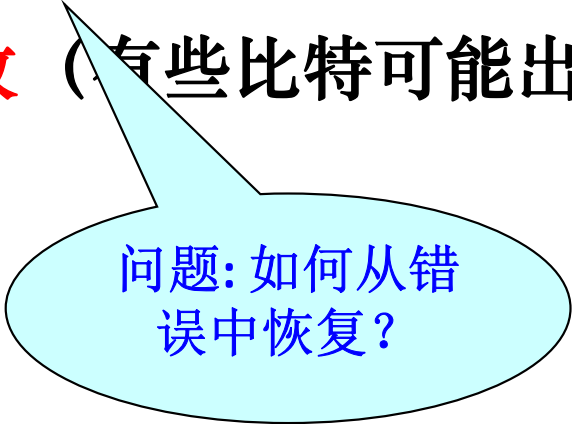
- **接收方：从底层信道读取数据**

从底层信道接收一个分组`rdt_rcv(packet)` → 解封取出数据`extract(packet, data)` → 数据传给上层`deliver_data(data)`



2、比特差错信道上的可靠数据传输： rdt2.0

- ✓ 信道不完全可靠，有可能产生比特错，但不丢包
- ✓ 所有分组按发送顺序被接收（有些比特可能出错）



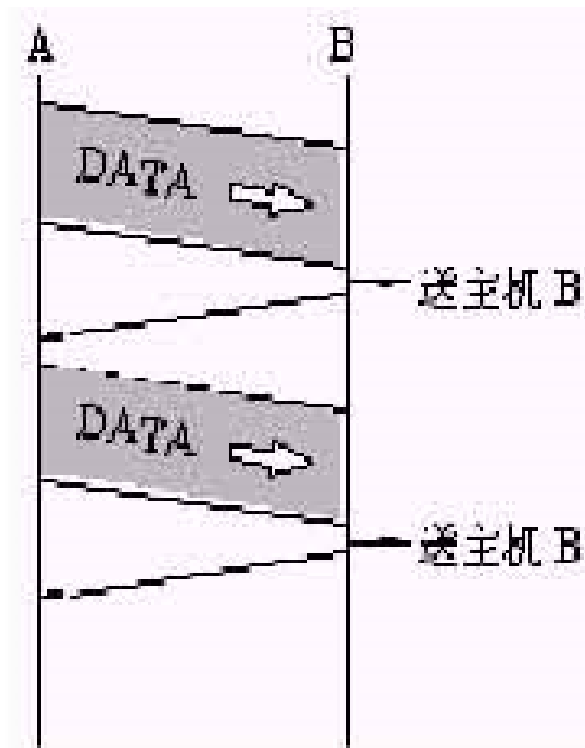
问题: 如何从错误中恢复?

类比例：打电话传递一条长信息，收听者：

- ✓ 听清每句话后说“OK”（肯定确认）；
- ✓ 听到不清楚的话，要求对方重复说“请重复一遍”（否定确认）。

肯定、否定确认作用

- ✓ 使发送方知道**哪些内容被正确接收**（肯定确认），**哪些有误需要重传**（否定确认）
- ✓ 基于重传机制的可靠数据传输协议称为**自动重传请求协议ARQ**(Automatic Repeat request)



ARQ协议处理比特差错机制

- 差错检测

- ✓ 使接收方检测出是否出现比特差错。如采用差错检测和纠错技术，使接收方能够进行差错检测并纠正。
- ✓ 需要给分组附加额外的比特一起发送。如rdt2.0协议，分组增加检验和checksum字段。

- 接收方反馈

- ✓ 使发送方知道发送的分组是否被正确接收。可以通过接收方回送肯定确认(ACK)和否定确认(NAK) 分组完成。

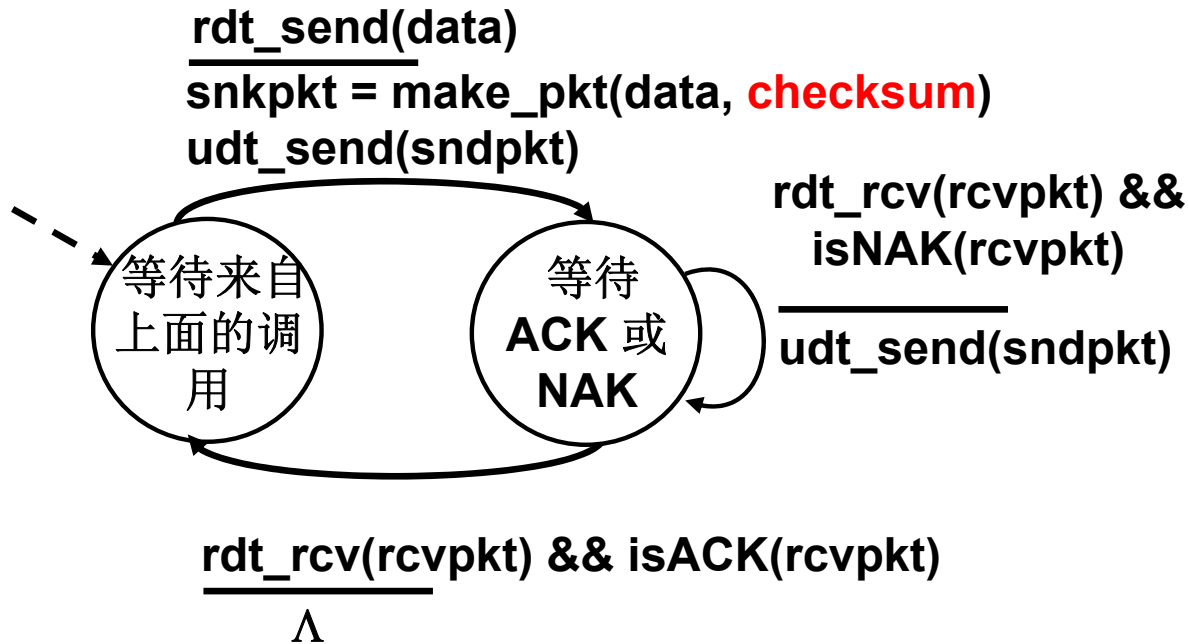
如rdt2.0协议，接收方将向发送方回送ACK（“1”比特）或NAK（“0”比特）分组。

- 重传

接收方收到有差错的分组时，通知发送方重传该分组。

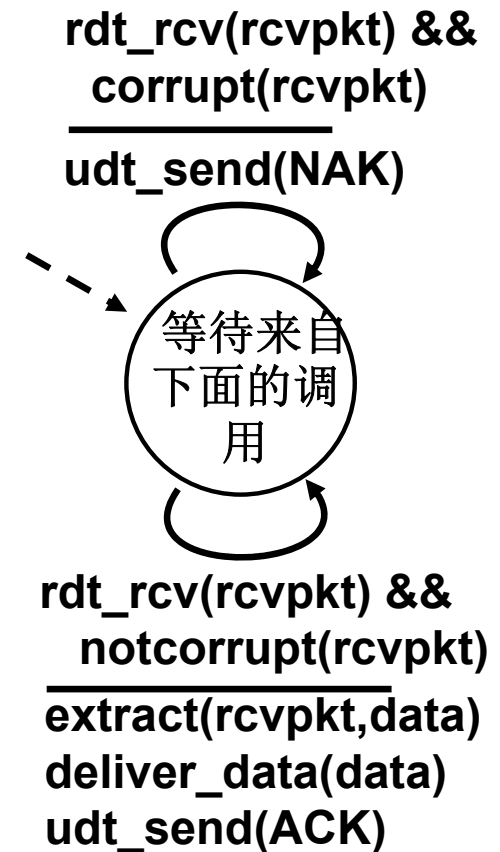
rdt2.0的FSM

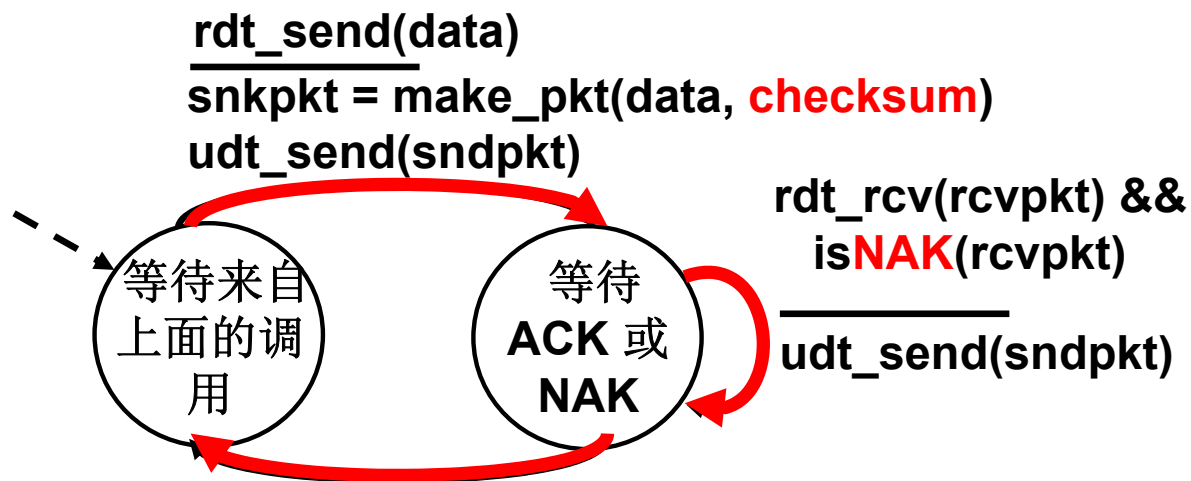
该数据传输协议采用了**差错检测**、**肯定确认与否定确认**及**重传**



发送方

接收方





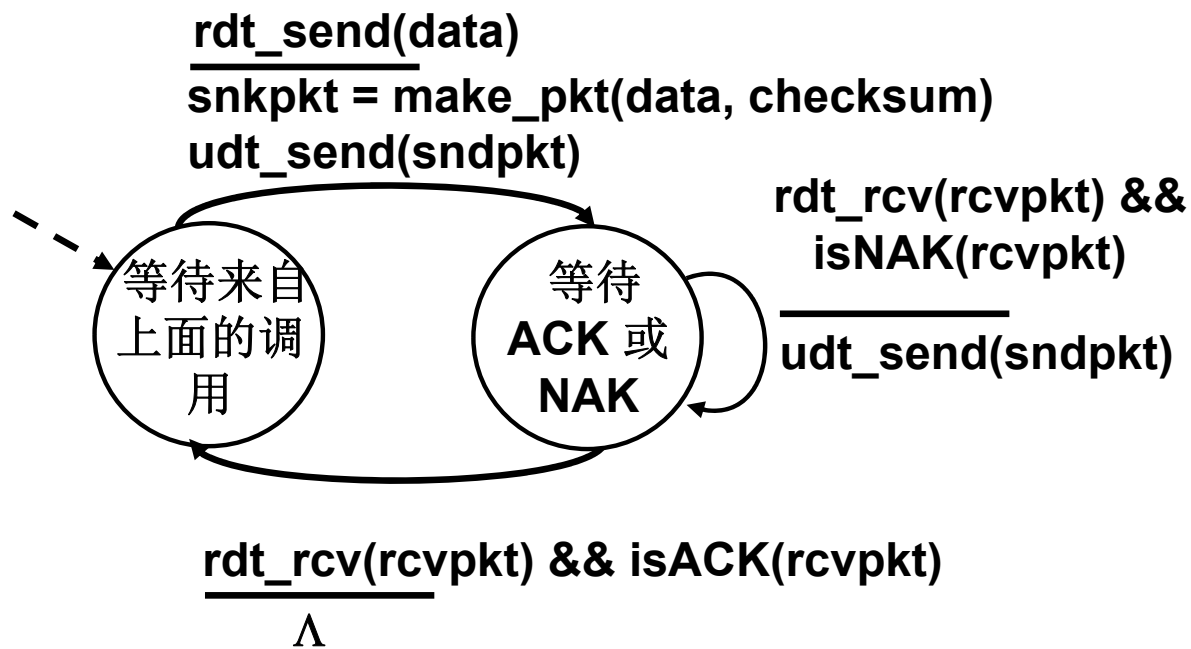
rdt_rcv(rcvpkt) && is**ACK**(rcvpkt)

Λ

发送方

等待上层数据 → 上层数据传来
 rdt_send(data) → 计算校验和，封装成分组
 make_pkt(data, checksum)
 → 发送分组 udt_send(sndpkt)

✓ 收到**ACK**分组 rdt_rcv(rcvpkt) && is ACK(rcvpkt) → 返回初始状态
 ✓ 收到**NAK**分组 rdt_rcv(rcvpkt) && is NAK(rcvpkt) → 重传上次发送的分组 udt_send(sndpkt) → 等待 ACK 或 NAK



发送方

说明

- ✓ 当发送方在等待ACK或NAK状态时，不能从上层得到数据，直到收到ACK离开该状态。
- ✓ 发送方在收到ACK分组之前，不会发送任何新数据，直到收到ACK分组为止。

rdt2.0协议称为 **停等协议**

停等 (stop-and-wait) 协议: 发送方发完一个分组后停止，等待对方回答。

接收方

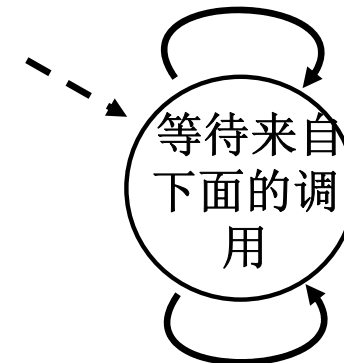
从底层接收一个分组，**检查校验和**

- ✓ **分组出错** `rdt_rcv(rcvpkt) && corrupt(rcvpkt) → 丢弃分组 → 返回NAK 分组`
`udt_send(NAK)`
- ✓ **分组无错** `rdt_rcv(rcvpkt) && notcorrupt(rcvpkt) → 从分组中取出数据`
`extract(rcvpkt, data) → 将数据传给上层`
`deliver_data(data) → 返回ACK 分组`
`udt_send(ACK)`

分组出错

`rdt_rcv(rcvpkt) && corrupt(rcvpkt)`

`udt_send(NAK)`

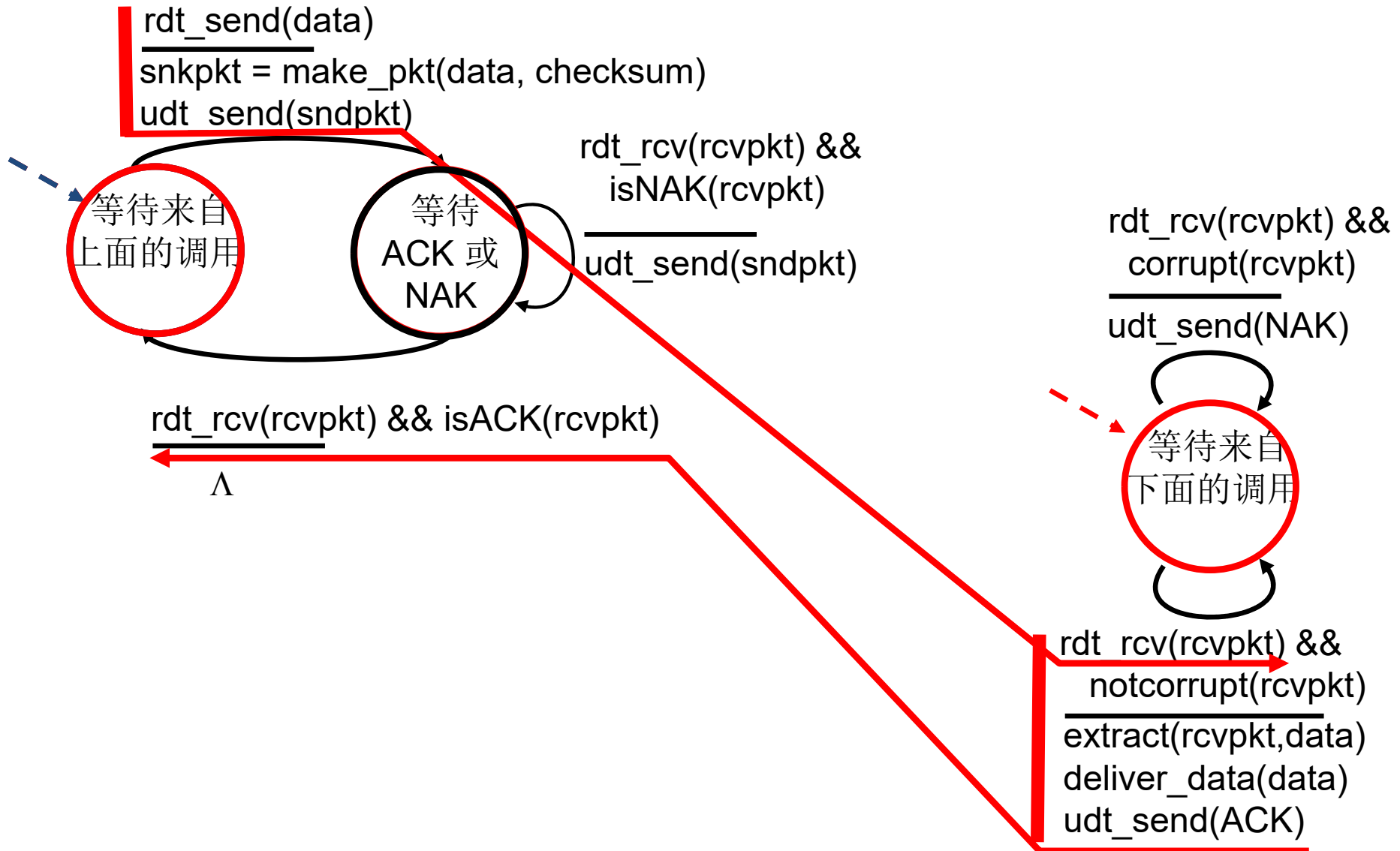


`rdt_rcv(rcvpkt) && notcorrupt(rcvpkt)`

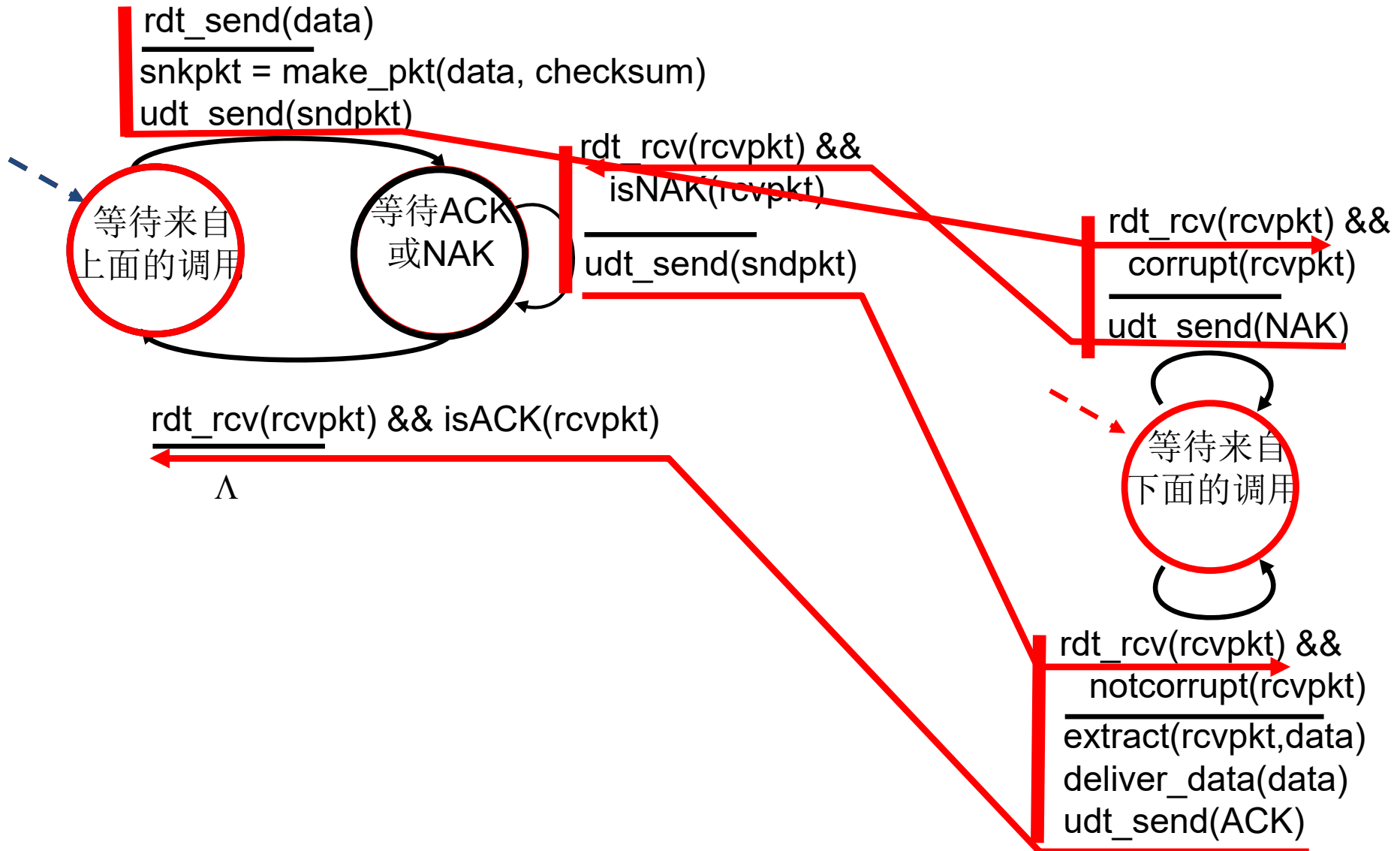
`extract(rcvpkt, data)`
`deliver_data(data)`
`udt_send(ACK)`

分组无错

rdt2.0: 无差错时的操作



rdt2.0: 有差错时的情况



rdt2.0协议特点

✓可以解决数据分组出错的问题

✓缺陷

若返回的ACK或NAK分组受损，发送方无法知道接收方是否正确接收了上一块数据

处理受损ACK和NAK的可能方法

- ✓ 收发双方互不理解对方回答的含义：对话不能正常进行；
- ✓ 增加足够的检验和比特：使接收方不仅可以检测差错，还可进行恢复。适用于会产生差错但不丢失分组的信道。
- ✓ 当发送方收到含糊不清的ACK或NAK分组时：只简单地重发当前数据分组。

新问题：

产生冗余分组(*duplicate packets*)：同一个分组收到两次。
接收方无法知道收到的是新的分组还是重传的分组。

解决冗余分组的方法

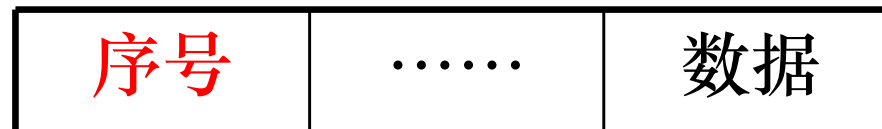
给分组添加一个序号字段

- ✓ **发送方：**给发送的数据分组编号，并将其序号放入“序号字段”

每发送一个新的分组“序号加1”

- ✓ **接收方：**通过检查序号，确定收到的分组是不是重复传送

按序号接收，若本次接收到的分组序号与前一次收到的分组的序号相同，即为重复分组，将该重复分组丢弃



序号位数选择

- 停等协议只需用一个比特，即“0”和“1”两种不同的序号
- 发送方的真实序号通过模2运算向前移动，可以区分前后相邻的两帧
- 0、1、0、1、.....

说明：

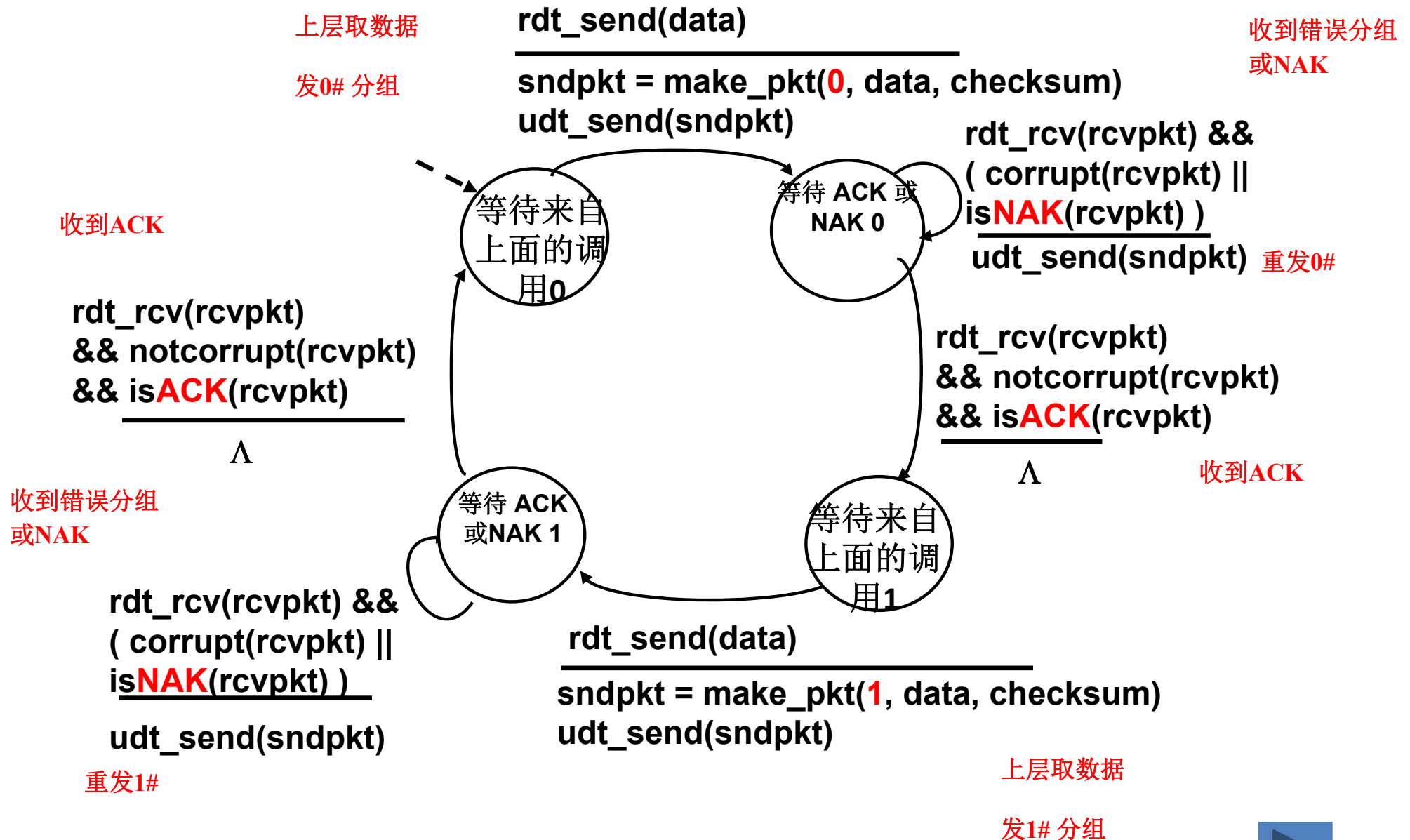
- ✓ ACK和NAK分组不需要指明要确认的序号，因为假设信道不丢失分组。
- ✓ 发送方知道所接收的ACK和NAK分组是对最近发送分组的响应。

rdt2.1协议FSM

rdt2.0改进，**可处理重复分组。**

- ✓ 指出正发送或准备接收的分组的**序号**。
- ✓ 发送或期望接收0号分组的状态中的动作与发送或期望接收1号分组的状态中的动作相似，只是序号处理方法不同。
- ✓ 使用从**接收方到发送方的肯定和否定确认**。

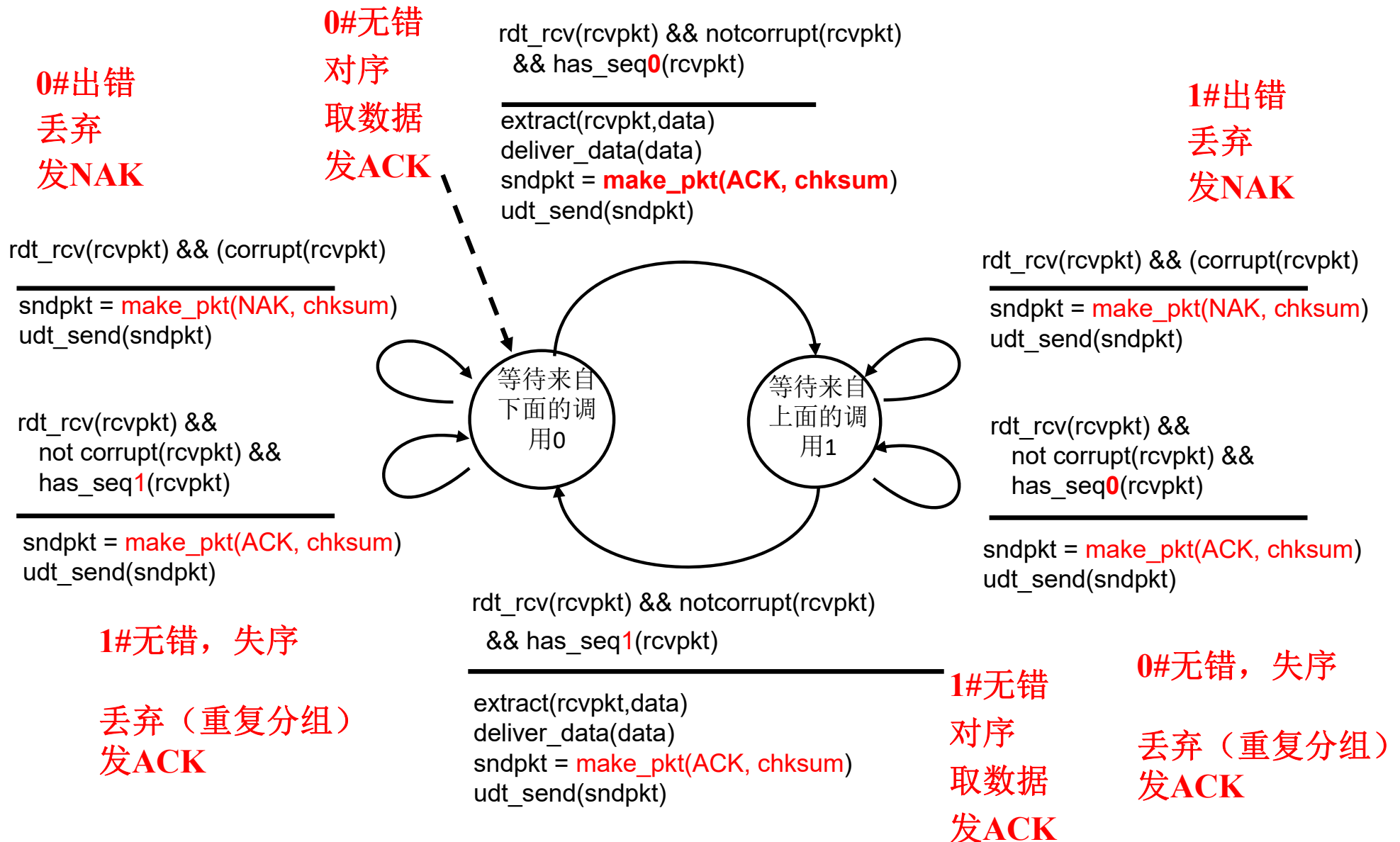
rdt2.1: 发送方, 处理受损的ACK/NAK



rdt2.1: 接收方

- ✓ 收到**乱序的分组**：是重复分组，丢弃，并回发一个肯定确认（ACK）
- ✓ 收到**受损的分组**：丢弃，并回发一个否定确认（NAK）

rdt2.1: 接收方



Rdt2.2

- ✓ 接收方收到受损的分组时，丢弃，不发送NAK，改为发送一个对前一个正确接收分组的ACK。
- ✓ 发送方接收到对同一个分组的两个ACK（接收冗余ACK）后，可知道接收方没有正确接收到跟在被确认两次的分组后面的分组。

rdt2.2: 发送方

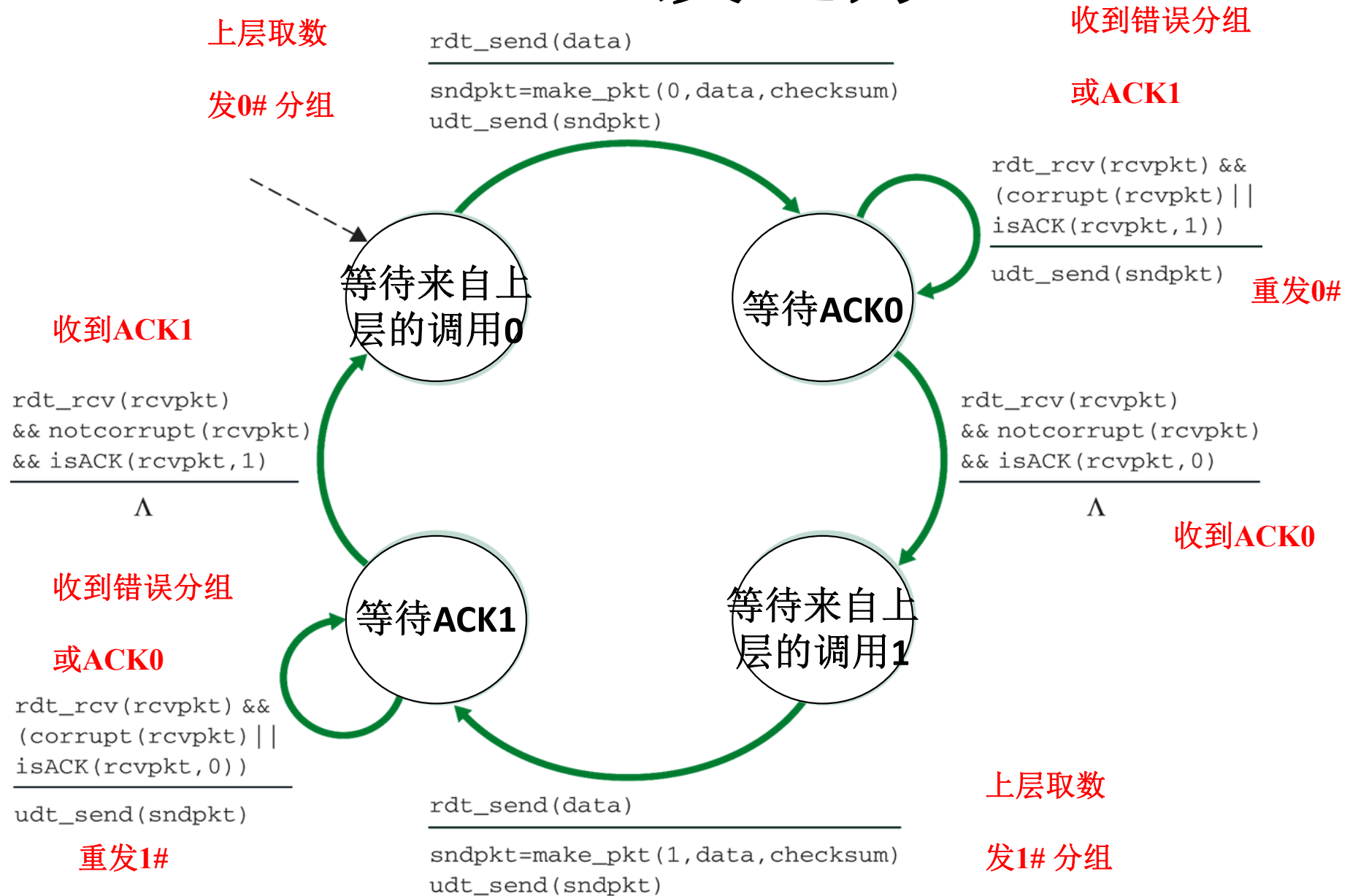


Figure 3.13 ♦ rdt2.2 sender

rdt2.2: 接收方

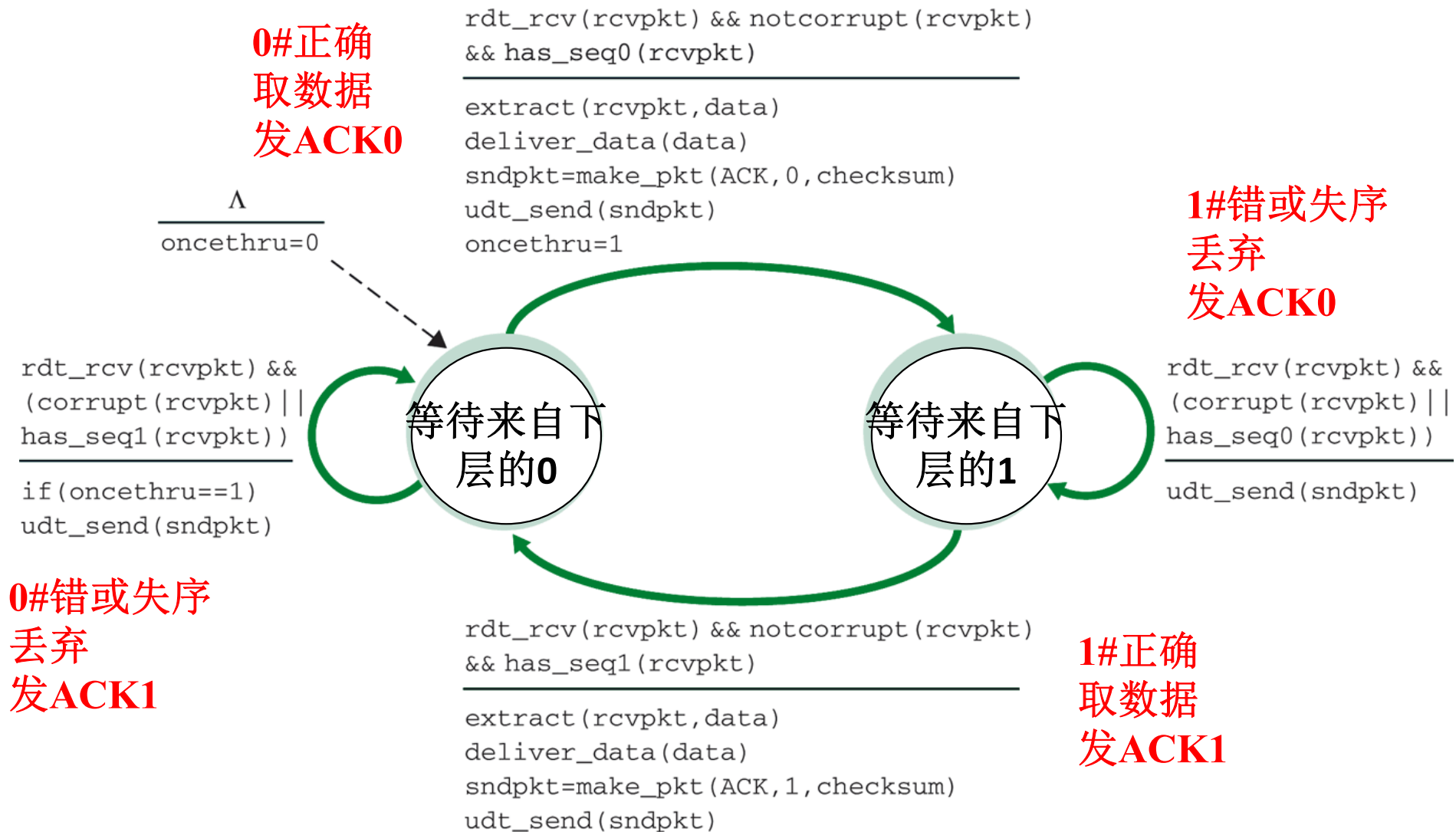


Figure 3.14 ♦ rdt2.2 receiver

3、具有比特差错的丢包信道上的可靠的数据传输：

rdt3.0

- ✓ 会出现**比特差错**
- ✓ 会**丢包**：发送的数据分组丢失，或接收方回发的ACK丢失
- 发送方如何发现**丢包以及丢包后如何处理**？
- ✓ **超时重发**：由发送方负责检测丢包和恢复，即
发送方发送一个数据分组后，等待一定时间，如果该段时间内没有收到ACK，则重传分组。
- ✓ **时间选择**：通常为“从发送方发出分组到收到接收方应答所需的平均时间”。
- ✓ **产生冗余分组**：一个分组在网络中经历了一个特别大的时延，使发送方超时重传（该数据分组或ACK并未丢失），可通过序号解决。

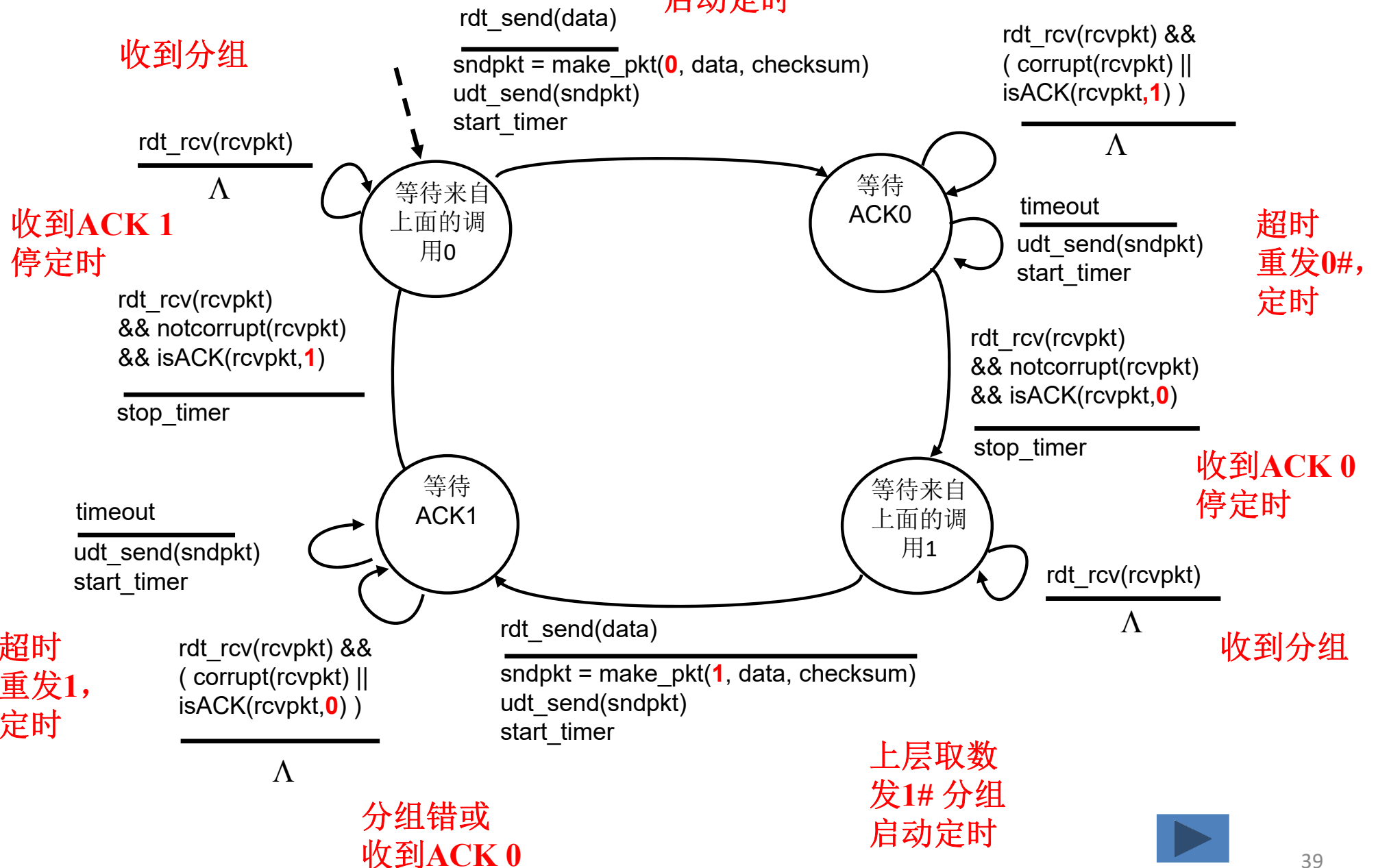


- **重传的实现：** 设置一个“**递减（倒）计数定时器**”，给定时间过期后，中断发送方
- **要求发送方**
 - ✓ 每发送一个分组(包括第一次和重发的分组)，启动一个定时器
 - ✓ 响应定时器中断(采取适当的动作)
 - ✓ 终止定时器

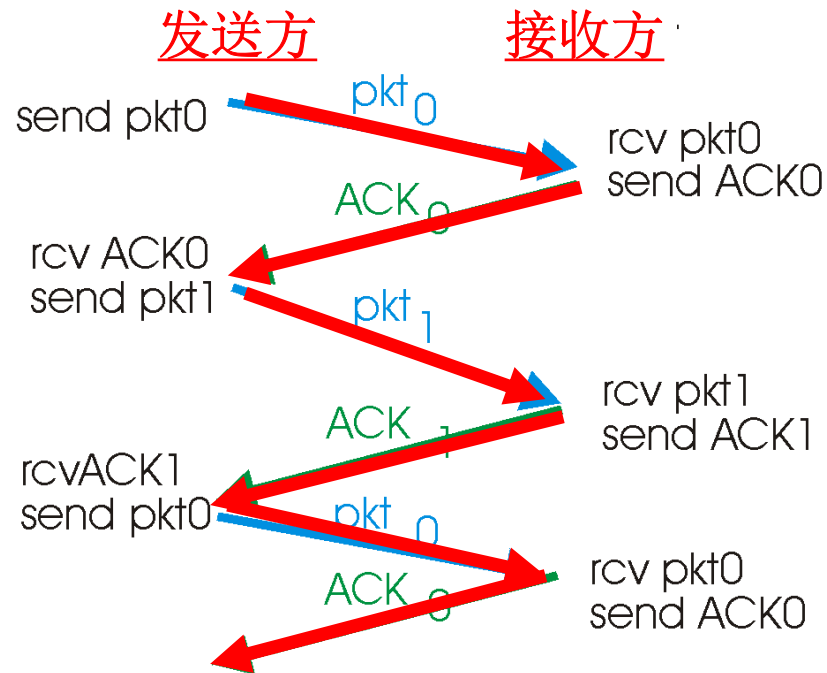
rdt3.0发送方

上层取数
发0# 分组
启动定时

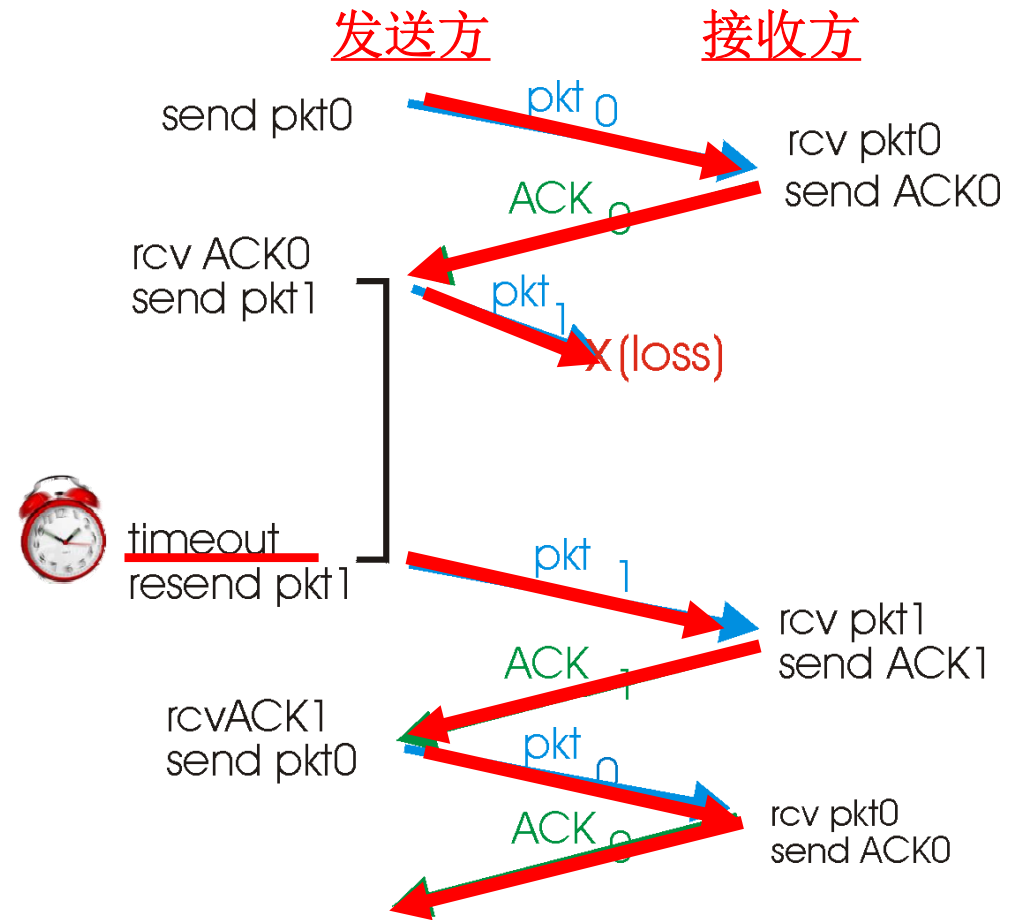
分组错或
收到ACK 1



rdt3.0 运行情况



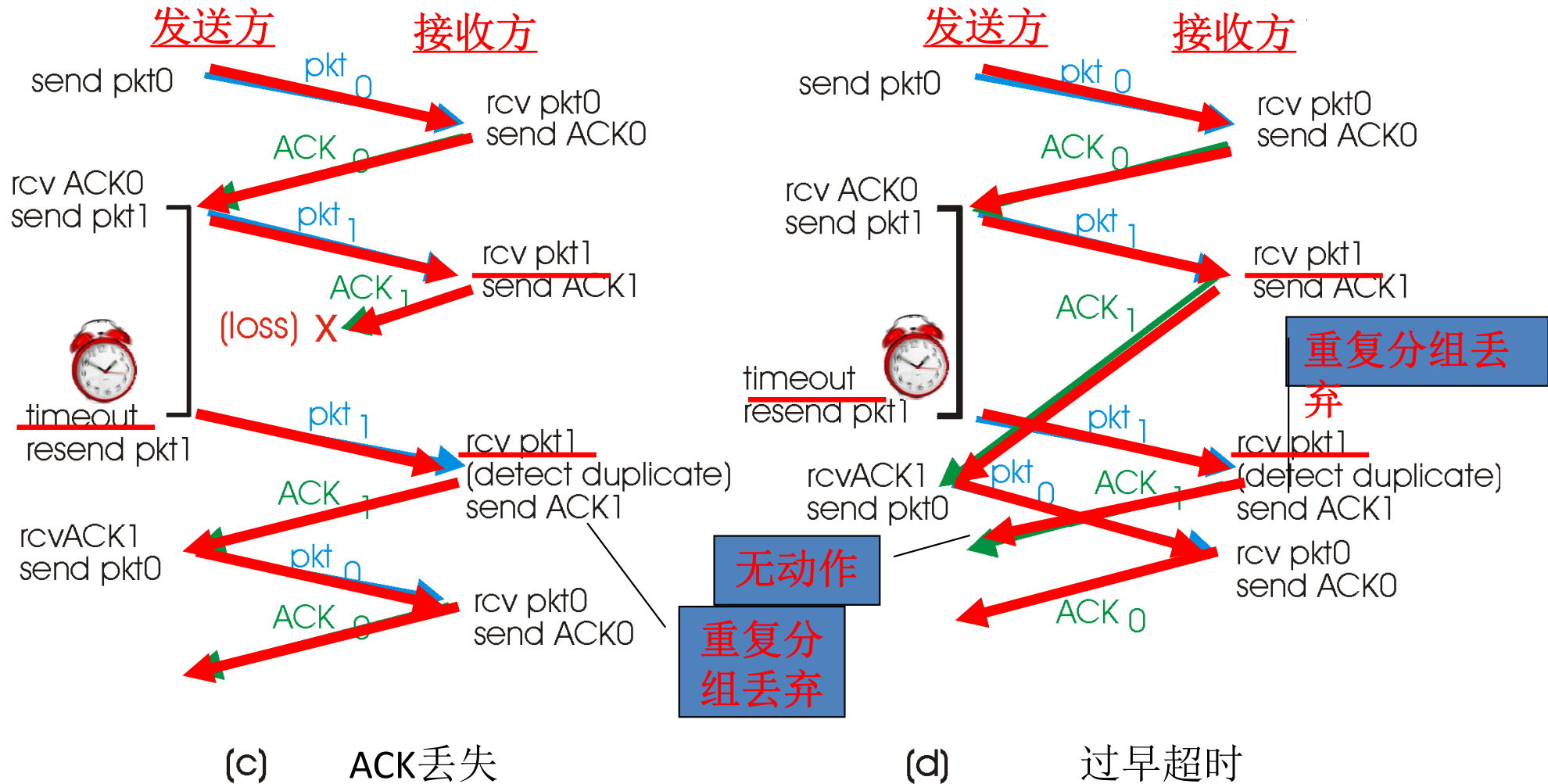
(a) 无丢包时的运行



(b) 分组丢失



rdt3.0运行情况



保证可靠数据传输协议的要点

检验和、序号、定时器、重传、肯定和否定确认



关于可靠

- 有没有100%的可靠？
- Checksum
- 各种协议

停等协议的性能:

能够正常工作，但效率不高

发送方(信道)利用率 U_{sender}

传输时延
 L/R

发送方将比特发送到信道的时间

发送总时间

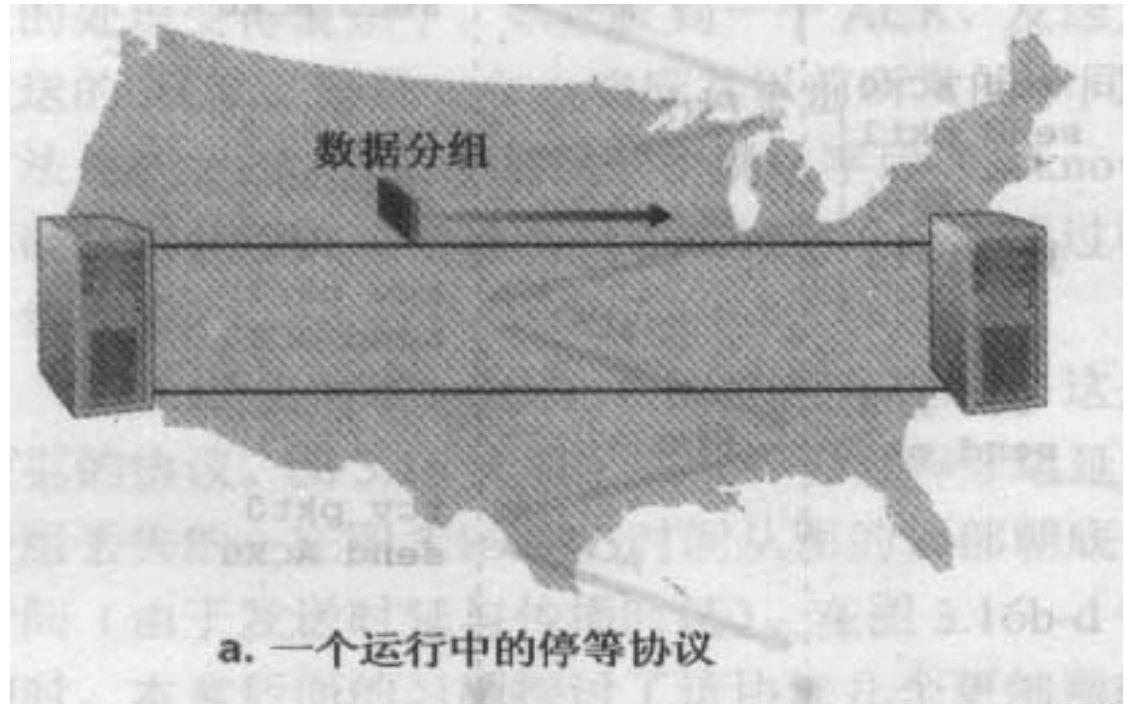
发送方从发送分组开始，到收到对方ACK分组，所需的时间

例，两台端主机之间进行理想的通信

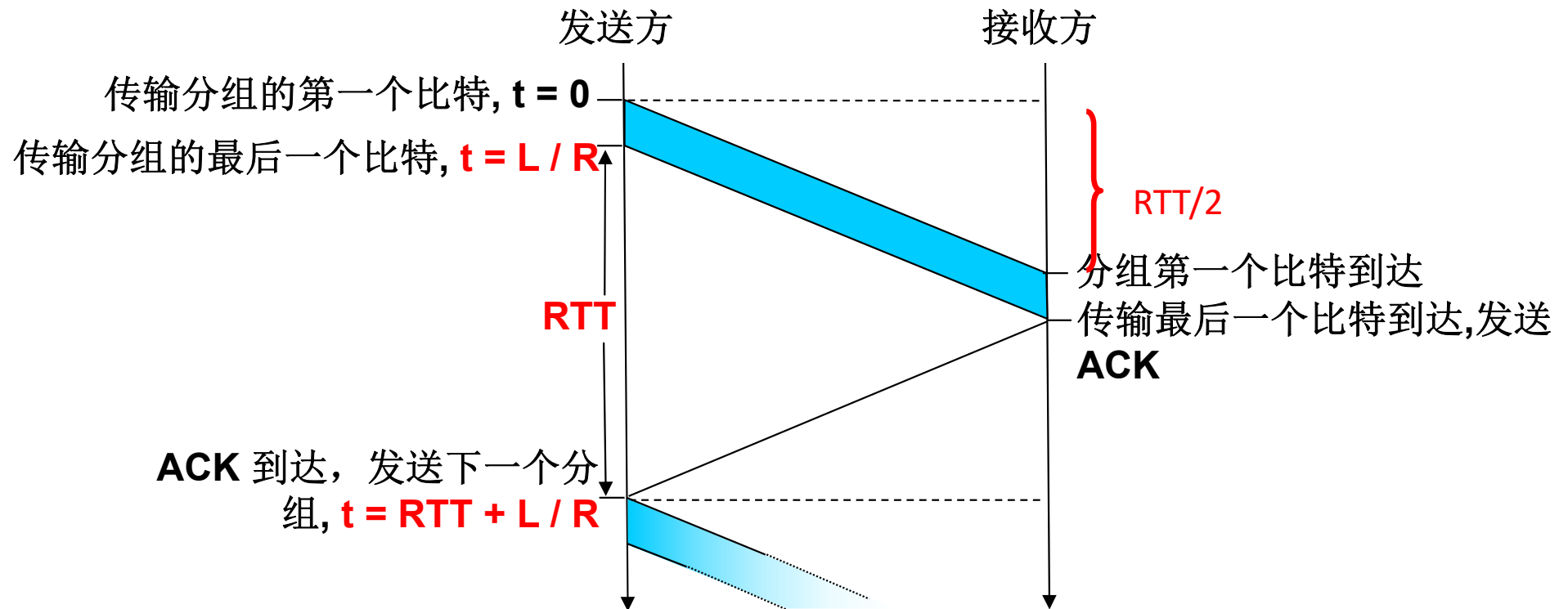
- ✓ 两个端系统之间的光速往返传播时延RTT约 30ms
- ✓ 信道传输速率 $R=1\text{Gb/s}(10^9\text{bit/s})$
- ✓ 分组长 $L = 1000$ 字节

传输时延:

$$t_{trans} = \frac{L}{R} = \frac{8000\text{bit}}{10^9\text{bit/s}} = 8\mu\text{s}$$



发送时间: $L/R + RTT = 30.008\text{ms}$

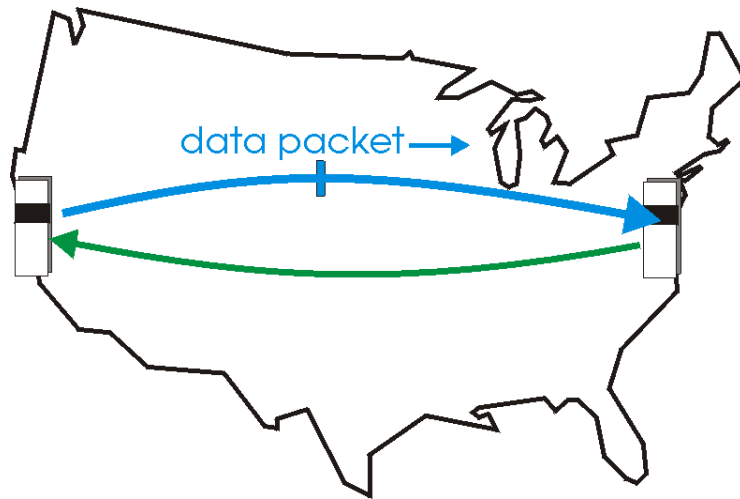


$$U_{\text{sender}} = \frac{L / R}{RTT + L / R} = \frac{0.008}{30.008} = 0.000267$$

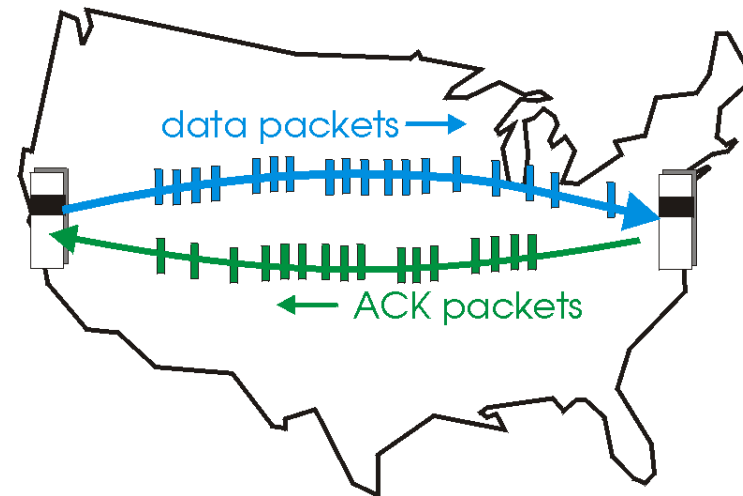
- ✓ 发送方只有**0.0267%**的时间忙
- ✓ 发送方在30.008ms内只能发送1000byte，有效的吞吐量仅为267kbit/s(即使有1Gbit/s的链路可用)

简单改进方法

允许发送方连续发送多个分组而无需等待确认，称为
流水线技术



(a) a stop-and-wait protocol in operation



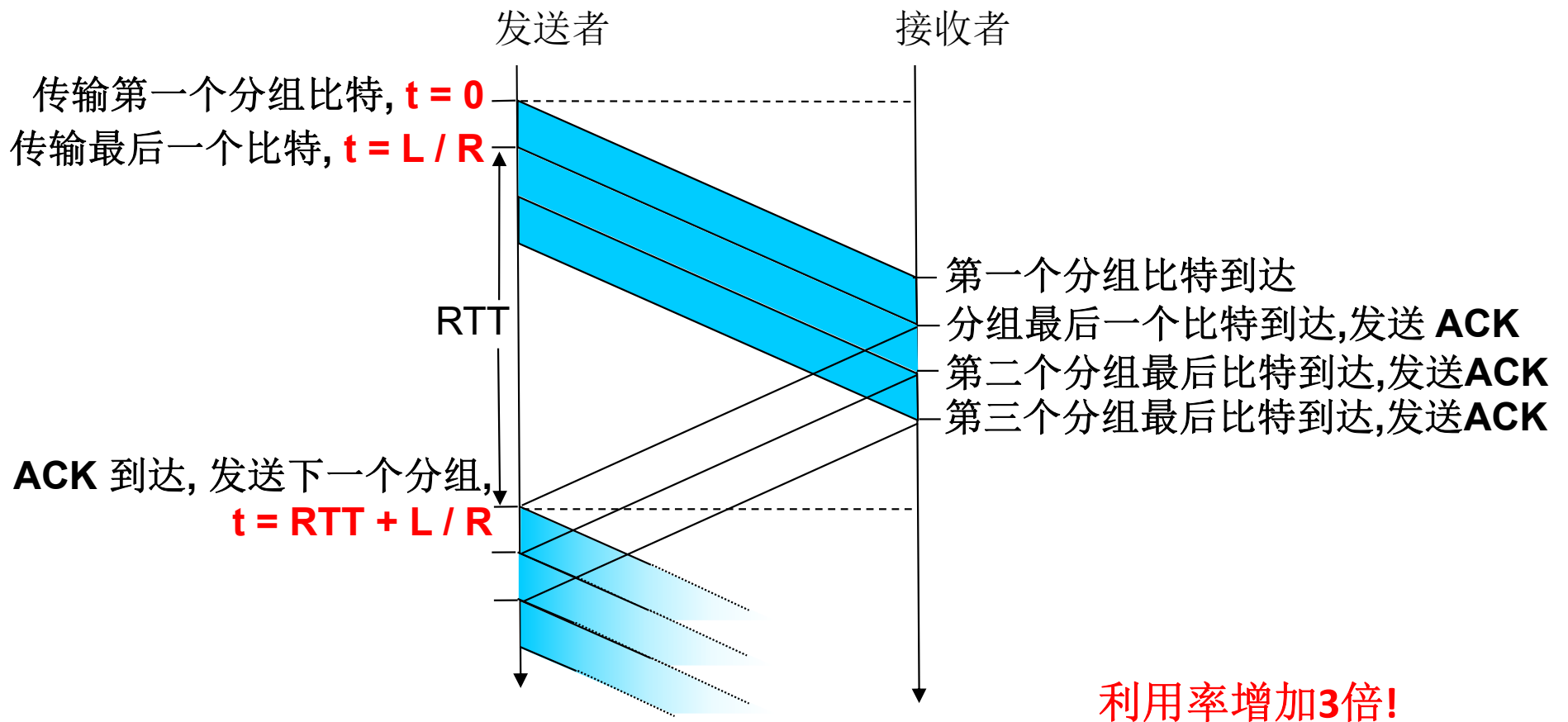
(b) a pipelined protocol in operation

流水线协议利用率

N 为连续发送分组个数

$$U_{sender} = \frac{L/R}{L/R + RTT} N$$

例：发送方在等待确认之前连续发送3个报文



$$U_{\text{sender}} = \frac{3 * L / R}{RTT + L / R} = \frac{0.024}{30.008} = 0.0008$$

P146

技术要点

- 增加序号范围：由于可连续发送多个分组，每个分组有唯一序号，可能有多个在传输中的未确认报文
 - 需要缓存多个分组
 - ✓ 发送方至少能缓冲已发送但没有确认的分组
 - ✓ 接收方缓存已正确接收的分组
- 序号范围和对缓冲的要求取决于数据传输协议处理丢失、差错及过度延时分组的方式
- 流水线协议类型
 - ✓ 回退N步 (Go-Back-N)
 - ✓ 选择性重传 (SR: selective repeat)

GBN基本思想

- **发送方：连续发送多个数据分组，停止等待**
 - ✓ 收到确认ACK，继续发送后面分组
 - ✓ 超时，未收到应答，**从出错分组开始重发**
- **接收方：按序号接收数据分组**
 - ✓ 正确：接收处理，发确认ACK
 - ✓ 出错：**将该分组及后面分组均丢弃，给最后一个按序接收的分组重新发送ACK**

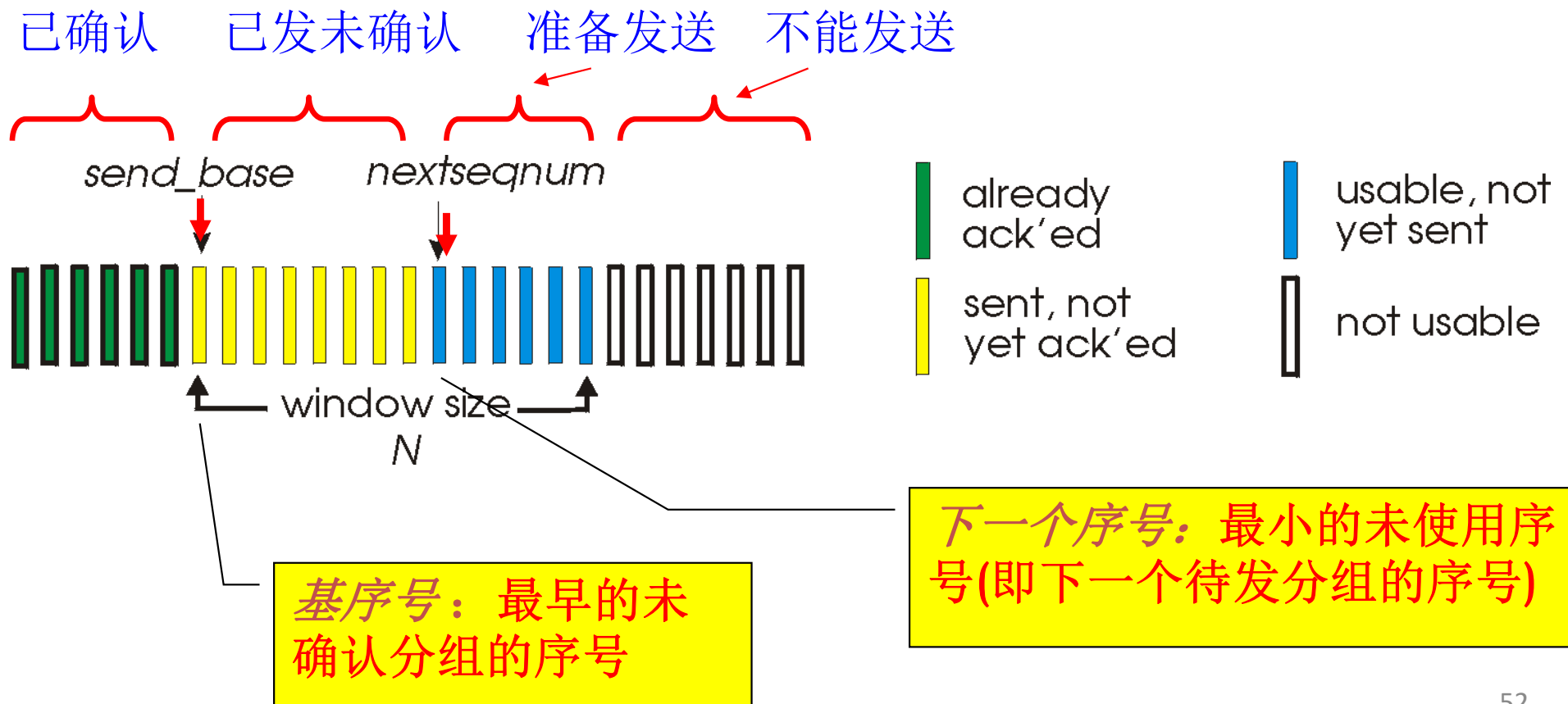
说明

- 连续发送的分组个数不能太多
 - ✓ 增加序号字段位数
 - ✓ 出错时，要重传很多数据分组，影响效率
- 连发个数受流水线中未确认的分组数限制，不能超过最大允许数 N
- 通过设置发送窗口和接收窗口分别控制连续发送和接收的分组个数

1、发送窗口P147

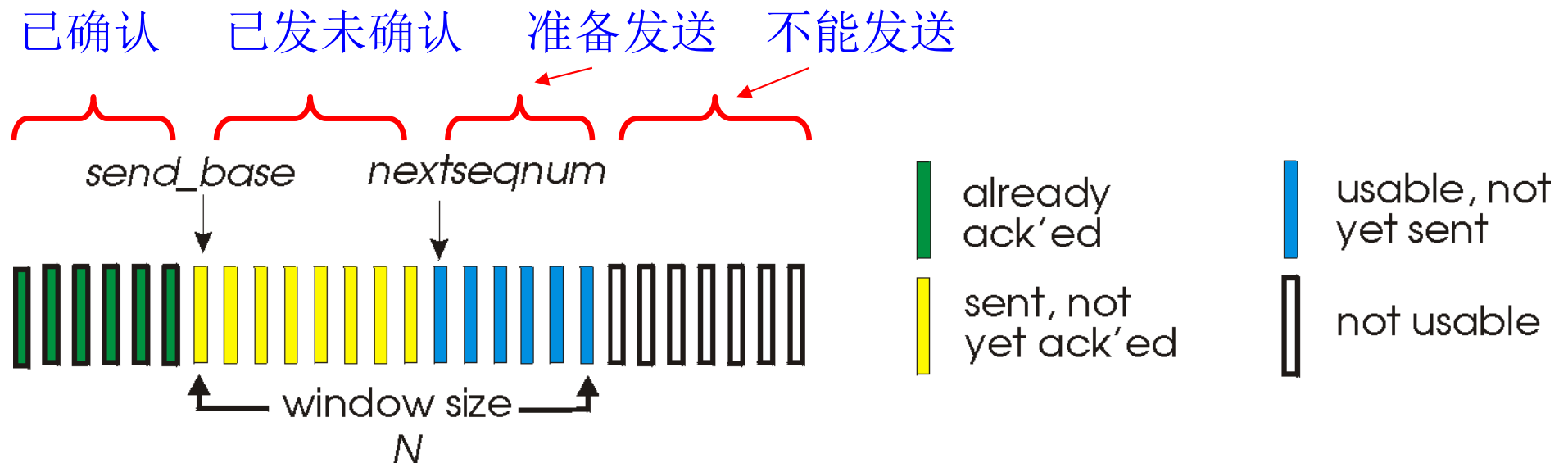
控制发送方连续发送的个数

实际是允许连续发送的序号表，只有落在发送窗口所含序号之内的，才能不等待应答就发送



序号范围分割成4部分:

- ✓ $[0, \text{base}-1]$: 已经发送并确认过的分组。
- ✓ $[\text{base}, \text{nextseqnum}-1]$: 已经发送但未被确认分组。
- ✓ $[\text{nextseqnum}, \text{base}+N-1]$: 用于将立即发送的分组。
- ✓ $\geq \text{base}+N$: 不能使用, 直到当前流水线中未被确认的分组得到确认(序号为base的分组)。

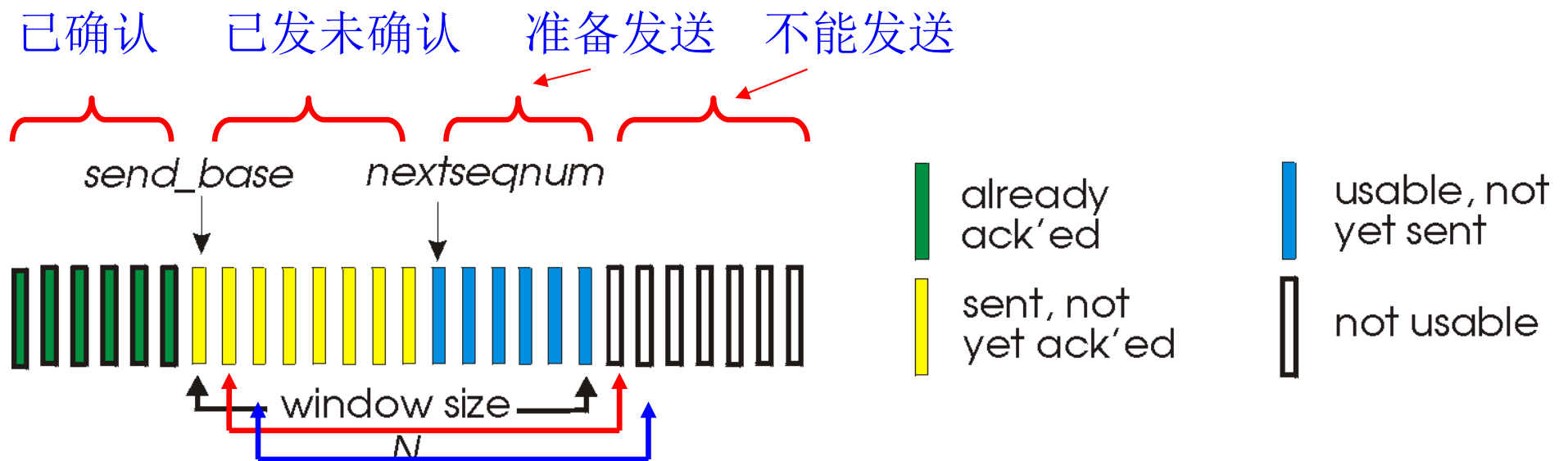


发送窗口大小 W_T

$W_T = N$ ，即序号范围，包括已发送未被确认的分组、以及准备发送的分组。

随着协议的运行，发送的分组陆续被确认，发送窗口在序号空间内向前滑动。

GBN协议常被称为**滑动窗口协议**(*sliding-window protocol*)。



2、序号取值范围

- 如果分组序号字段的位数是 k ，则序号范围是 $[0, 2^k-1]$
- 即序号在 $0 \sim 2^k-1$ 的范围循环使用
- $0, 1, \dots, 2^k-1, 0, \dots$
- 如：
- 停等协议序号字段的位数是 1 ，序号范围是 $[0, 1]$

3、发送方扩展FSM

GBN发送方响应三种事件

- **上层的调用**

上层调用rdt_send()→检查发送窗口是否已满（是否有N个已发送、但未被确认的分组）

- ✓ **窗口未满**：创建一个分组并将其发送，更新变量

- ✓ **窗口已满**：将数据返回给上层，以后再试

- **收到ACK**

对序号为 n 的分组的确认使用**累积确认**，即表明接收方已正确接收到序号为 n 及以前的所有分组

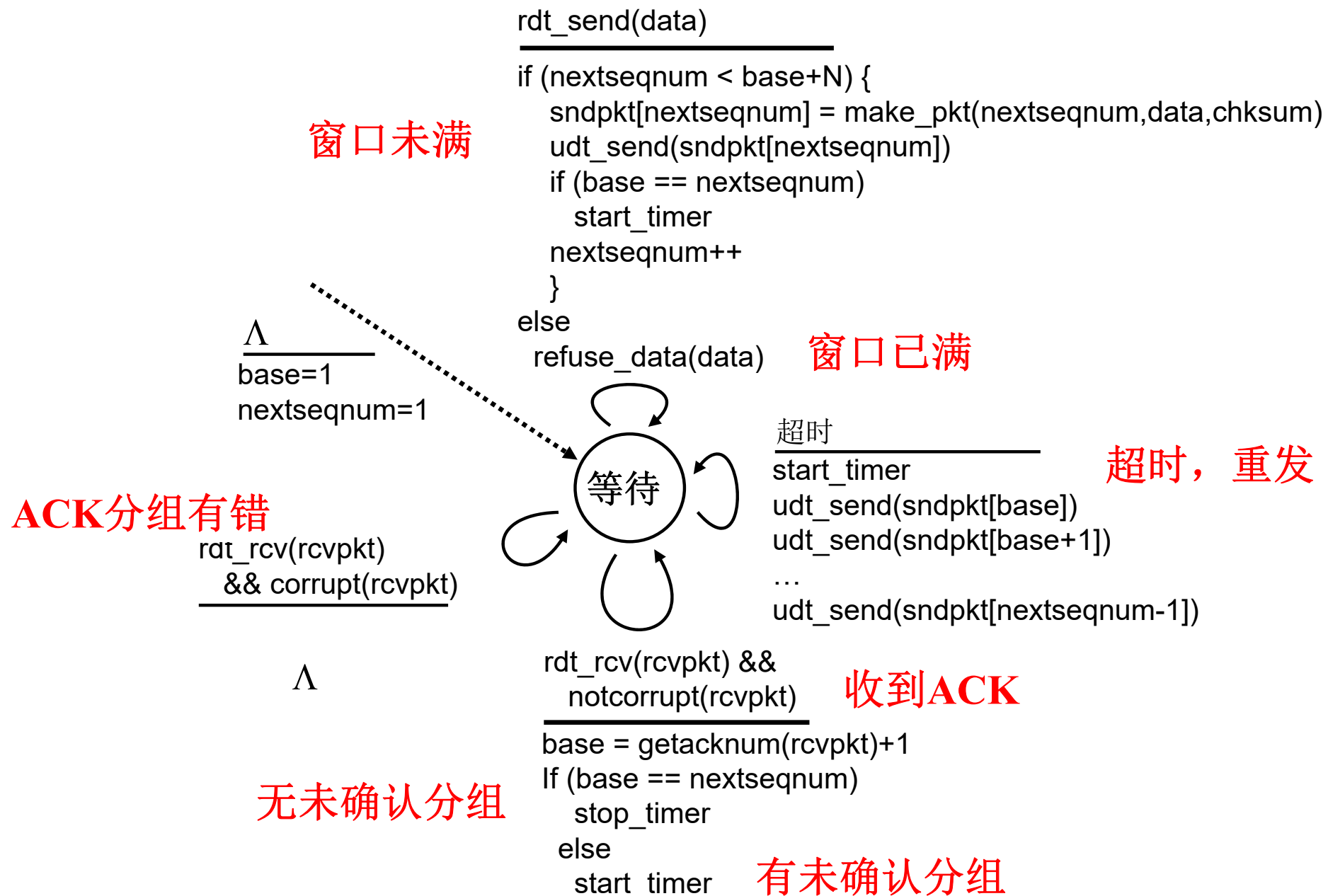
- **超时**：设置定时器处理“丢失数据或确认分组”的情况

只使用一个定时器，作为最早的已发送但未被确认的分组的定时

- ✓ 产生**超时**，发送方**重发**所有已发送过但还未被确认过的分组

- ✓ **收到一个ACK**，但仍有已发送但未被确认的分组，重新启动定时器；若没有未确认报文，终止定时器

GBN: 发送方扩展的 FSM



4、接收方

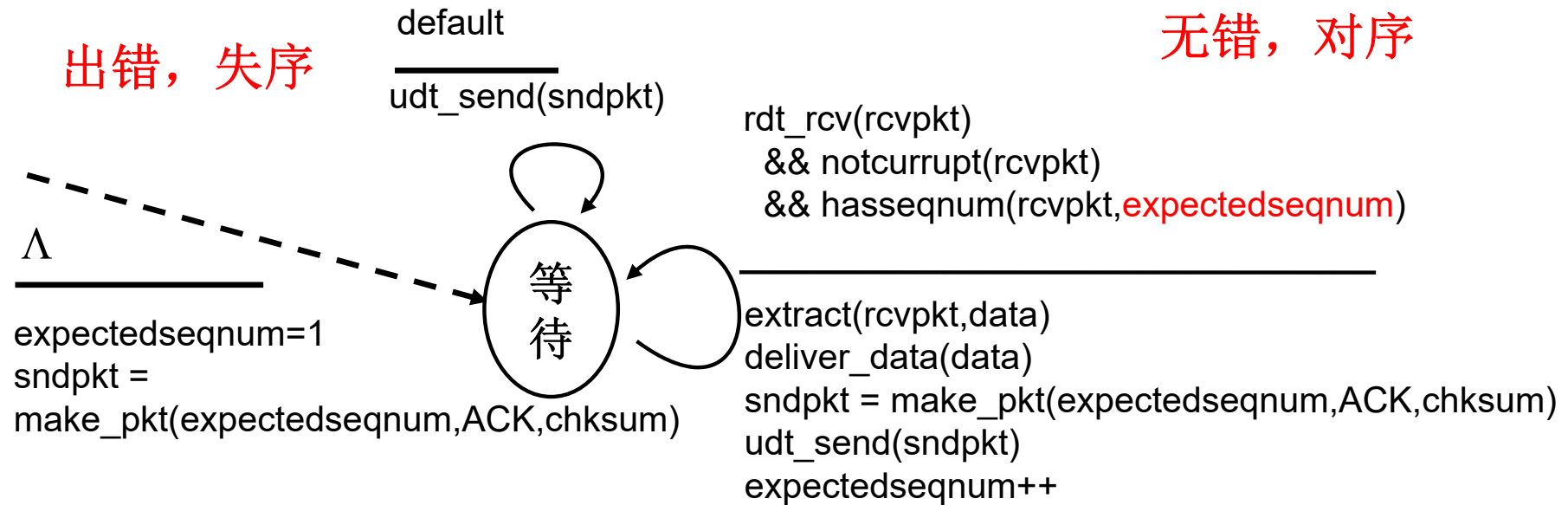
按序接收

接收方 $W_R=1$

要点

- ✓ 正确按顺序接收到序号为 n 的分组：将分组中的数据交付到上层，并回发一个 ACK
- ✓ 其他情况的分组：丢弃该分组，并为最近按序接收的分组重发 ACK
- ✓ 分组一次性交付给上层：分组 k 被交付， k 之前均已交付
- ✓ 丢弃所有失序分组：控制简单，接收方不需要缓存任何失序分组

GBN: 接收方扩展 FSM



- ✓ 接收方维护下一个按序接收的分组的序号，该值保存 expectedseqnum 变量中

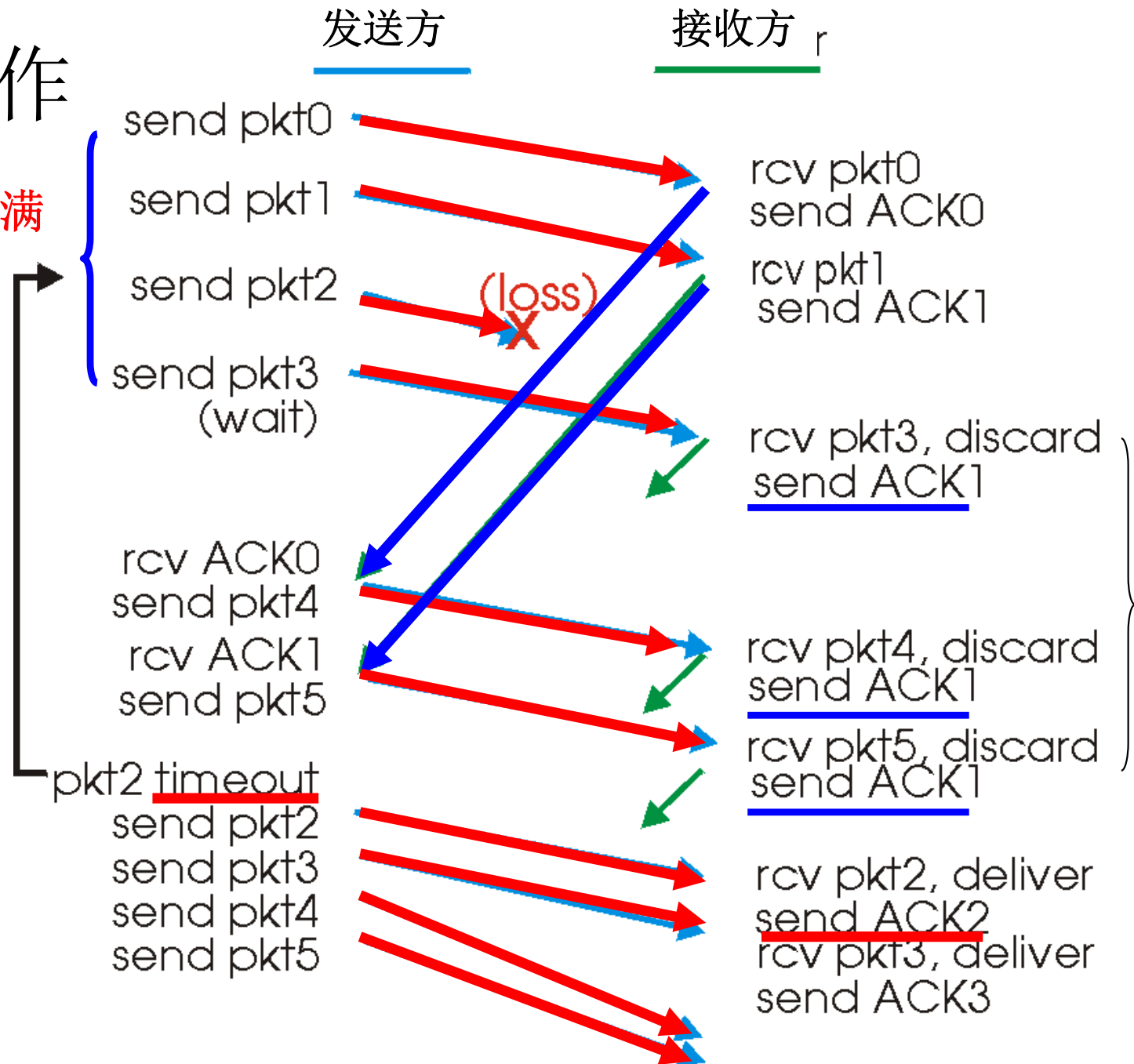
GBN 缺点: 可能需要很多的重传

例，窗口尺寸为4的GBN协议的运行

- **发送方：**连续发送4个分组后等待，直到一个或多个分组被确认
- ✓ 每接收到一个连续的ACK时，该窗口便向前滑动
- **接收方：**收到一个连续的正确分组，窗口向前滑动

GBN 操作

窗口满



丢弃

小结

- ✓ **GBN协议具体实现：**采用过程形式（实现在响应各种事件时要采取的动作）

如，发送方包括rdt_send()、定时器中断、 rdt_rcv()等

- ✓ **GBN协议综合可靠数据传输的全部技术：**使用序号、累积确认、检验和以及超时/重传操作等

5、发送窗口最大尺寸

若序号位数 k 位，GBN协议，发送窗口最大尺寸是 $W_T = 2^k - 1$ ，而不是 2^k

反例说明

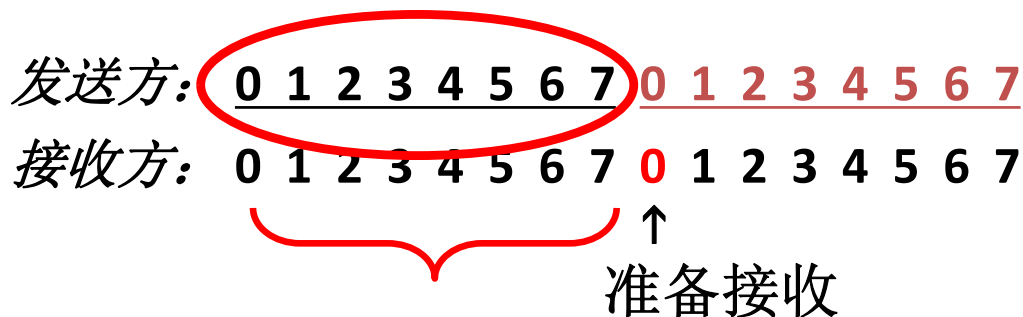
设序号 3 位，则序号空间为 $0\# \sim 7\#$ （共 8 个），其发送窗口最大尺寸是 $W_T = 7$ ，而不是 8

(1) $W_T = 8$

①发方发送0#~7#共8分组，停止等待应答

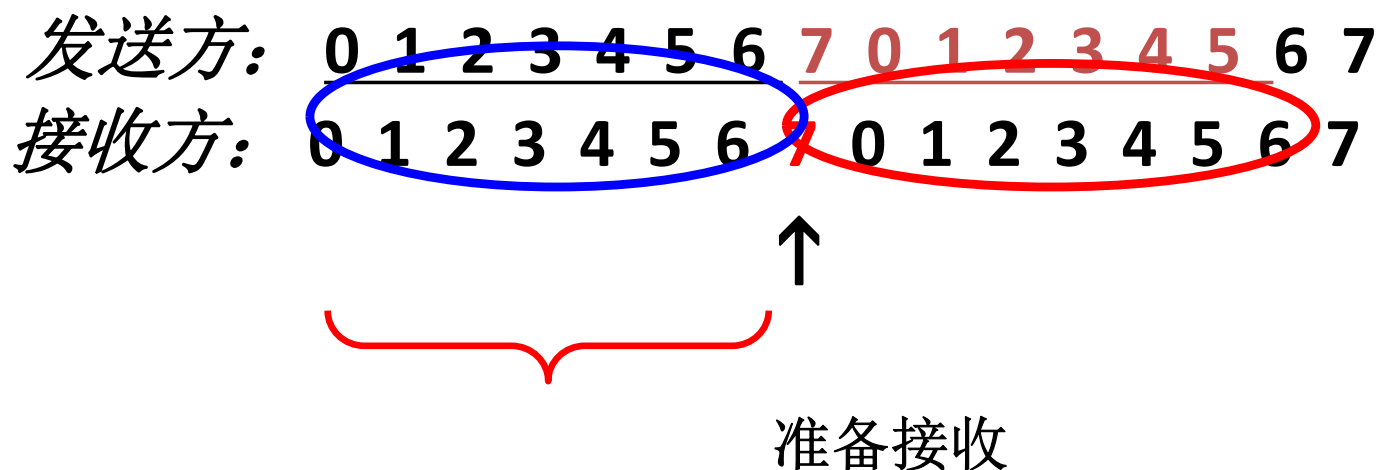
②收方全部正确收到，分别发应答，窗口序号变为0#
(准备收0#分组)

- ✓ 应答全部到达：发方接着发送8个新分组0#~7#，协议正确进行
- ✓ 应答全部丢失：发方超时，重新发送8个旧分组0#~7#，收方出现重复分组接收，协议错误



(2) $W_T = 7$

- ① 发方发送0#~6#共7分组，停止等待应答
- ② 收方全部正确收到，分别发应答，窗口序号变为7#
(准备收7#分组)
- ✓ 应答全部到达：发方接着发送7个新分组7#、0#~5#，协议正确进行
- ✓ 应答全部丢失：发方超时，重新发送7个旧分组0#~6#，与收方准备接收的分组序号不相同，拒绝接收，丢弃



3.4.4 选择性重传 (SR)

GBN 协议缺陷:

出错时，要重发出错及其后分组，当窗口大小和带宽时延积都很大时，堵塞管道，**影响效率**

改进方法:

发送方**只重传出错（丢失或受损）的分组**

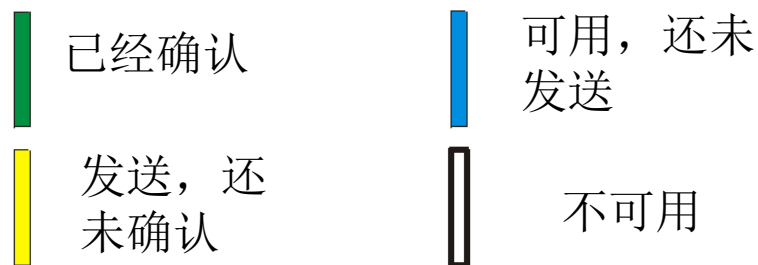
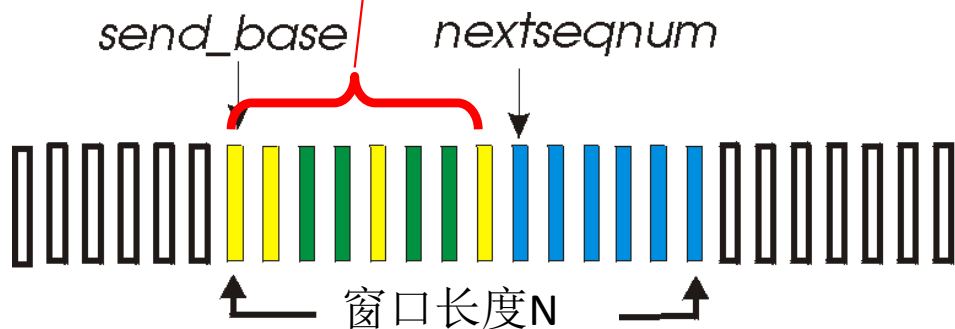
选择性重传 (SR)基本思想

- **发送方**：连发多个数据分组，停止等待
 - ✓ 收到确认ACK，继续发送后面分组
 - ✓ 超时,未收到应答，**只重发出错分组**
- **接收方**：**不按序号接收**数据分组
 - ✓ **正确**：接收，发确认ACK；如果是按序到达的则交付
 - ✓ **出错**：**丢弃该分组**，以后**正确分组放入缓存**，当出错分组正确收到后，按顺序一起交付

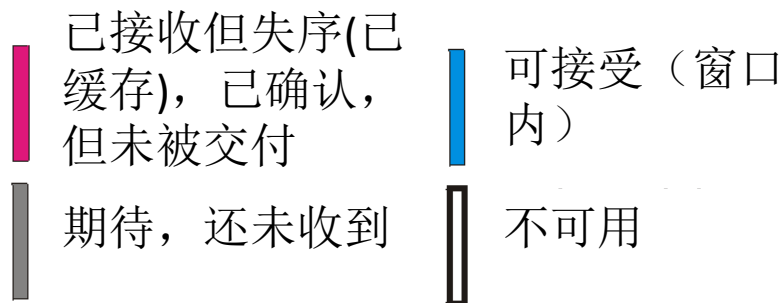
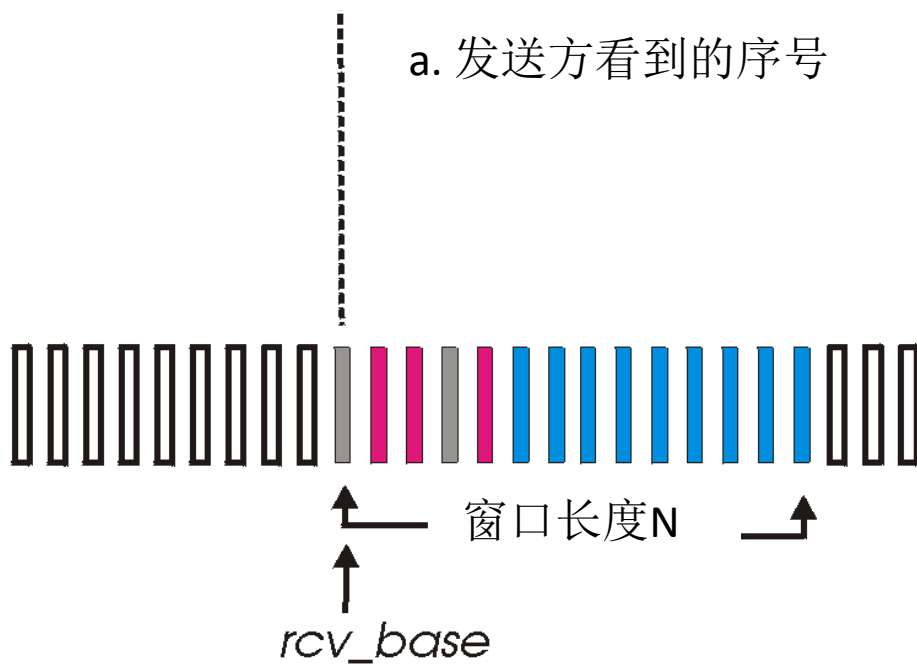
不按序接收。接收窗口大小 > 1 ，即**只要序号落在接收窗口内的正确分组都可接收。**

已发送

某个时刻的发送方、接收方窗口



a. 发送方看到的序号



b. 接收方看到的序号

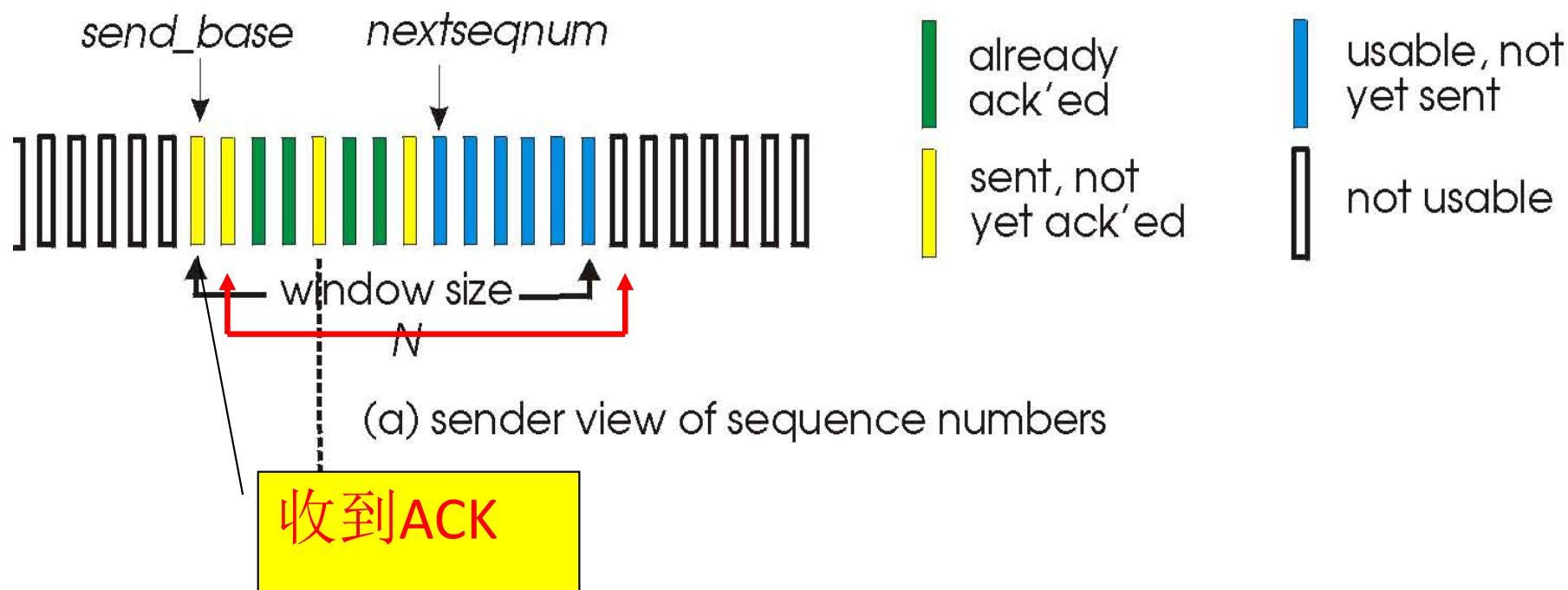
1、发送方的事件与动作

□ 从上层收到数据

当从上层接收到数据后，发送方检查下一个可用于该分组的序号

- ✓ 若序号在发送方窗口内，则将数据打包并发送
- ✓ 否则与GBN一样，将数据缓存，或将其返回给上层，以后再传
- 超时
 - 每个分组有自己的定时器，超时后只能发送一个分组

- **收到ACK:** 将被确认的分组标记为已接收(若该分组序号在窗口内)
- ✓ 如果该分组的序号等于发送基序号send_base, **则窗口基序号向前移动到具最小序号的未确认分组处**
- ✓ 窗口移动后, 仍有序号落在窗口内的未发送分组, 继续发送



2、接收方的事件与动作

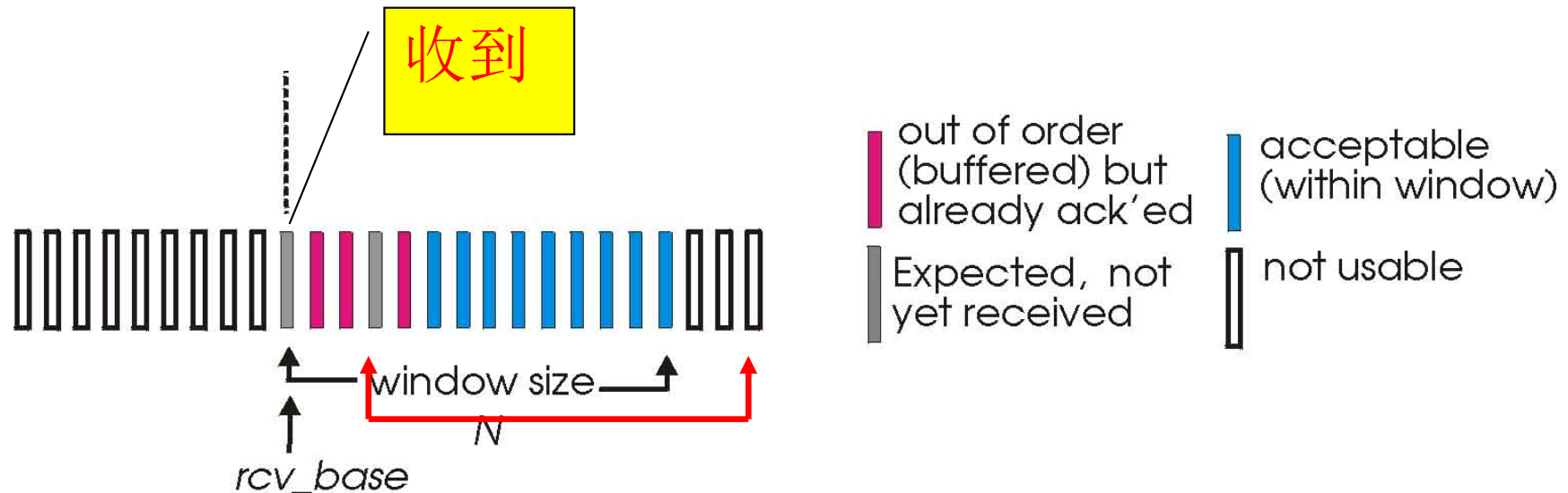
接收方接收正确分组不管其是否有序

失序分组先被缓存，直到所有丢失分组（序号更小的分组）被收到为止，才可**将一批分组按序交付给上层**

- 序号在接收窗口内的分组被正确接收:

收到的分组落在 $[rcv_base, rcv_base+N-1]$ 内, 接收并回发一个ACK

- ✓ 如果该分组以前没收到过, 则被缓存
- ✓ 如果该分组的序号**等于接收基序号**, 则该分组及已缓存的序号连续的分组交付 (起始于基序号) 给上层
- ✓ 然后, 接收窗口按交付的分组数量向前移动



(b) receiver view of sequence numbers

□ 收到序号在接收基序号以前的分组

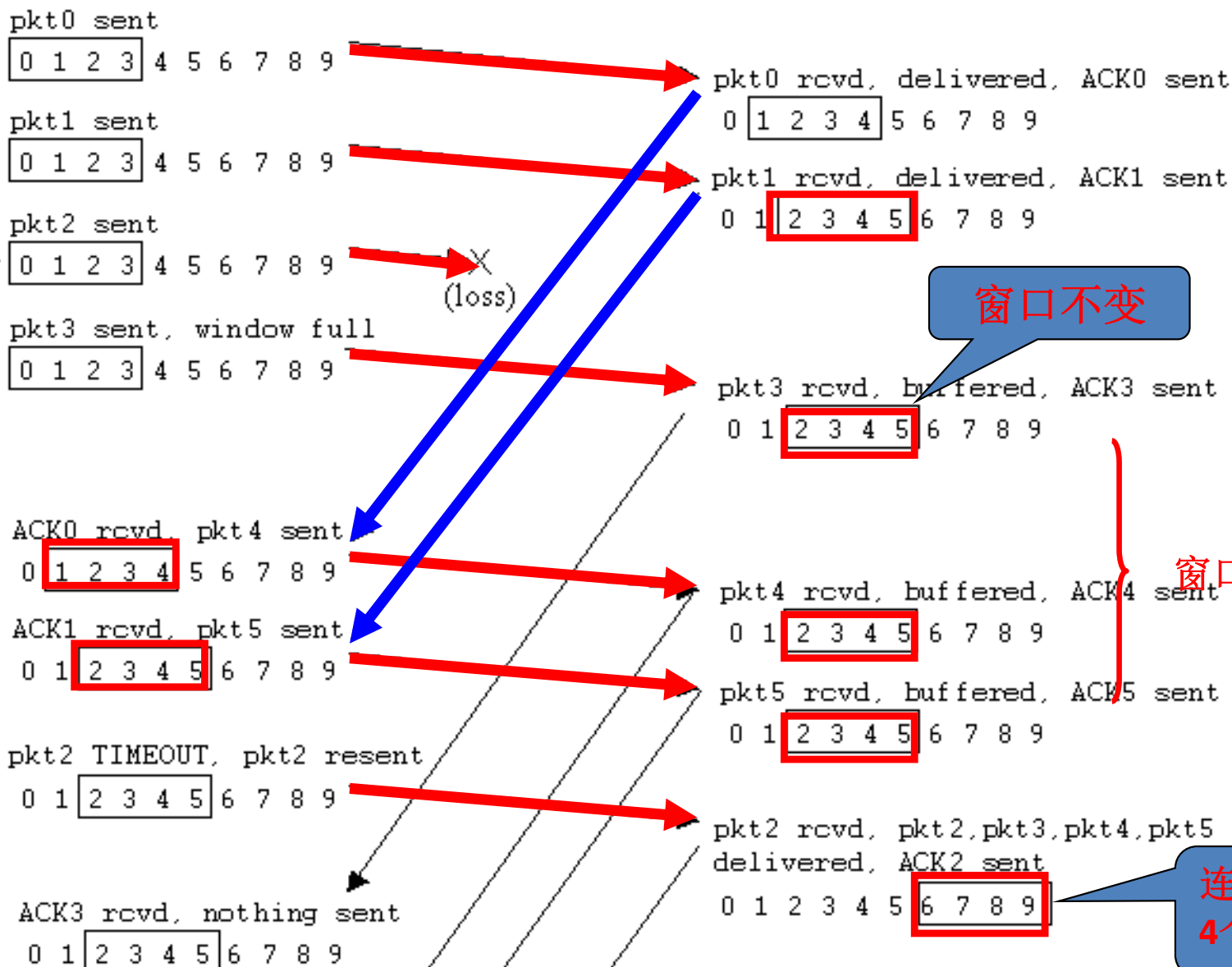
该分组是接收方以前已确认过的分组。生成一个ACK，并回发给发送方

如果接收方不确认，发送方窗口不能向前滑动

- 其他情况：忽略该分组

例：发送窗口、接收窗口大小均为4

连发4个分组



SR协议的发送方和接收方窗口不总一致。



3、窗口最大尺寸

若序号位数 k 位，SR协议，发送窗口和接收窗口尺寸最大是 2^{k-1} ，不是 2^k-1 。即序号空间一半。

反例说明：

设序号 2 位，则序号空间为 $0\# \sim 3\#$ ，发送窗口和接收窗口尺寸最大是 2 ，不是 3 。

$$(1) W_T = W_R = 3$$

- ①发方发送0#~2#共3分组，停止等待应答
- ②收方全部正确收到，发应答，窗口滑动，允许接收3#、0#、1#分组
- ✓ 应答全部到达：发方发送3个新分组3#、0#、1#，协议正确进行
- ✓ 应答全部丢失：发方超时，重新发送0#~2#共3旧分组，与准备接收序号重叠，出现重复分组接收，协议错误

发送方： 0 1 2 3 0 1 2 3

接收方： 0 1 2 3 0 1 2 3



准备接收



(2) $W_T = W_R = 2$

- ① 发方发送0#~1#共2分组，停止等待应答
- ② 收方全部正确收到，发应答，窗口滑动，允许接收2#~3#分组
- ✓ 应答全部到达：发方发送2个新分组2#~3#，协议正确进行
- ✓ 应答全部丢失：发方超时，重新发送2个旧分组0#~1#，序号不在接收窗口内，拒绝接收，丢弃

发送方：0 1 2 3 0 1 2 3

接收方：0 1 2 3 0 1 2 3



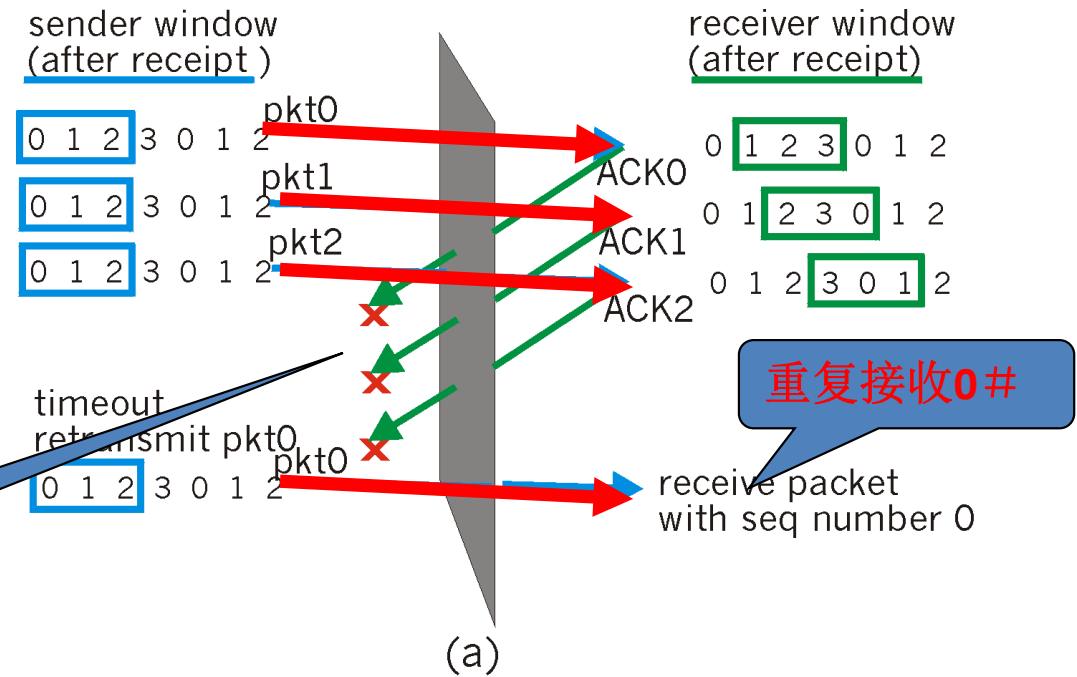
准备接收



例子:

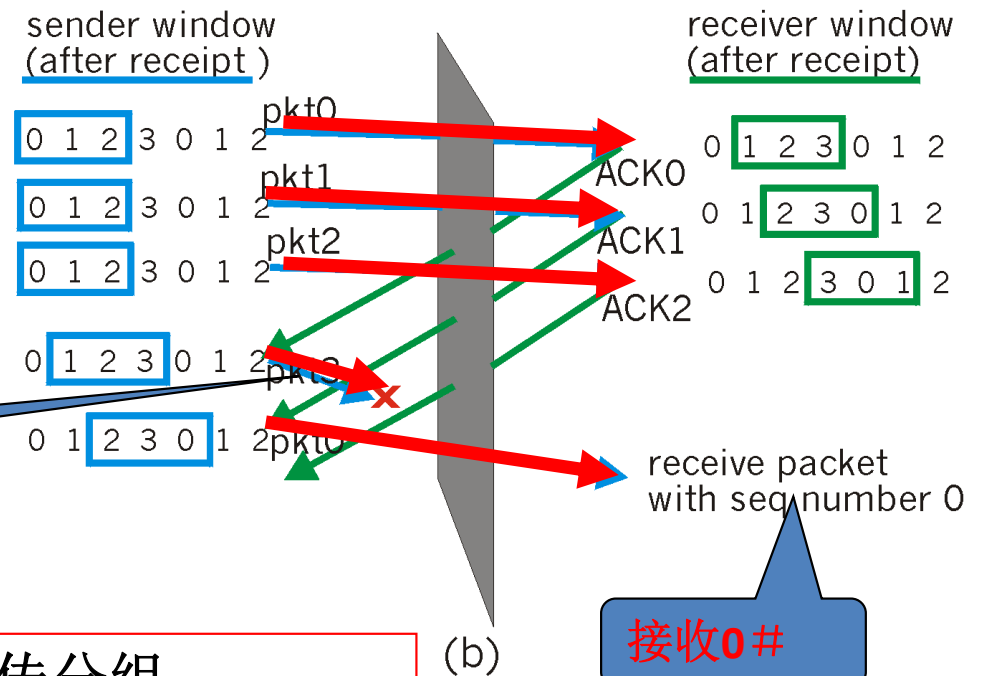
- 序号2位: 0, 1, 2, 3
- 窗口长度 = 3

应答丢失重发



窗口最大长度为2!

3# 丢失



接收方无法确定是新还是重传分组



4、三种协议比较

	停等协议	GBN协议	SR协议
发送方	发 1 个 重发出错分组	连发 <i>n</i> 个 从出错分组开始重发	连发 <i>n</i> 个 只重发出错分组
接收方	按序号接收 出错、重复分组丢弃	按序号接收 从出错分组开始丢弃	不按序号接收 只丢弃出错分组
窗口大小	$W_T = W_R = 1$	$1 < W_T \leq 2^k - 1$ $W_R = 1$	$1 < W_T, W_R \leq 2^{k-1}$

5、可靠传输机制及用途总结

- **校验和**：检测分组传输中的**比特错误**
- **定时器**：用于检测分组**超时及重传**分组
 - ✓ 分组或ACK丢失
 - ✓ 一个分组被延时但未丢失（过早超时），或ACK丢失，接收方可能会收到重复分组
- **序号**：对发送方向接收方**发送的数据分组按顺序编号**
接收方可检测分组丢失（序号间的空隙）或重复（相同序号的分组）

- **确认：**接收方用其告诉发送方某个分组或其组分组已被正确地接收

确认报文包含被确认的分组或多个分组的序号。可以是逐个的或累计

- **否认：**接收方告诉发送方某个分组尚未正确地接收。包含未被正确接收的分组的序号
- **窗口、流水线：**发送方可以连续发送序号落在发送窗口序号范围内的分组，可以不用逐个确认

存在问题

即使发送方或接收方的窗口中都没有包含 x ，但**可能会出现一个具有序号或确认号 x 的分组的旧拷贝**

- ✓ 为确保序号不被重新使用，需要发送方“确信”任何先前发送的序号为 x 的分组都不再在网络中为止
- ✓ **规定分组最长的寿命**：分组在网络中“生存”的时间不会超过某个固定的最长时间
如在TCP中假定为大约3分钟

作业

- 在**GBN**协议和**SR**协议中，假设序号空间的长度为 k ，为了避免出现错误接收冗余分组的问题，允许发送方窗口最大为多少？说明原因。
（提示：可以用反例说明）

谢谢大家！

Chapter 3

Transport Layer

A note on the use of these PowerPoint slides:

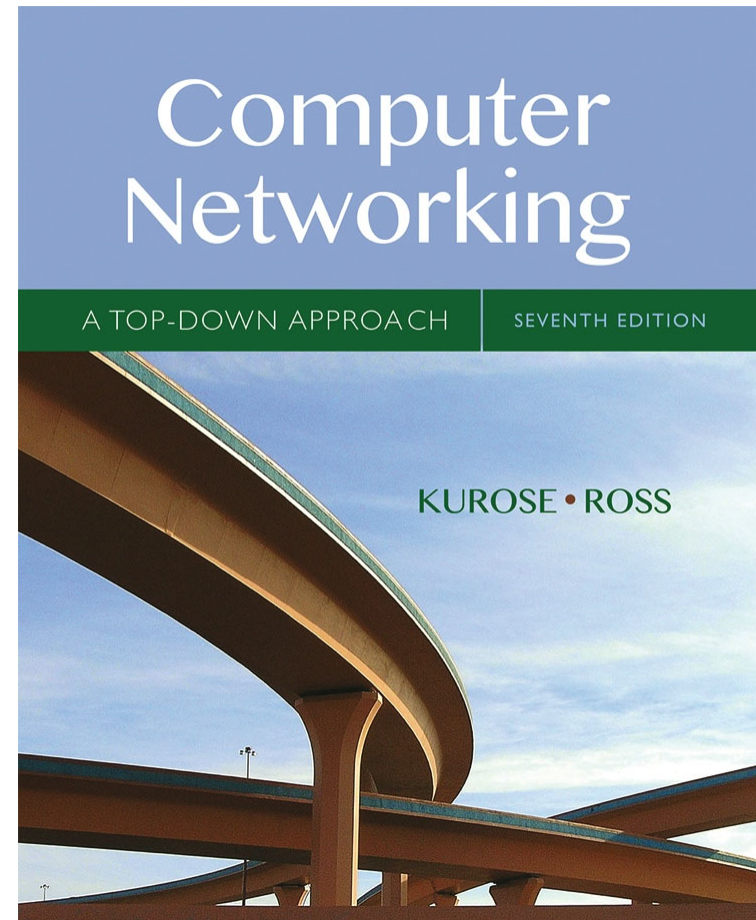
We're making these slides freely available to all (faculty, students, readers). They're in PowerPoint form so you see the animations; and can add, modify, and delete slides (including this one) and slide content to suit your needs. They obviously represent a *lot* of work on our part. In return for use, we only ask the following:

- If you use these slides (e.g., in a class) that you mention their source (after all, we'd like people to use our book!)
- If you post any slides on a www site, that you note that they are adapted from (or perhaps identical to) our slides, and note our copyright of this material.

For a revision history, see the slide note for this page.

Thanks and enjoy! JFK/KWR

All material copyright 1996-2020
J.F Kurose and K.W. Ross, All Rights Reserved



*Computer Networking: A
Top-Down Approach*
7th edition
Jim Kurose, Keith Ross
Pearson, 2016

3.5 面向连接的传输: TCP P152

采用：差错检测、重传、累积确认、定时、序号和确认序号。

3.5.1 TCP连接

3.5.2 TCP报文段结构 (+Telnet)

3.5.3 往返时间的估计与超时

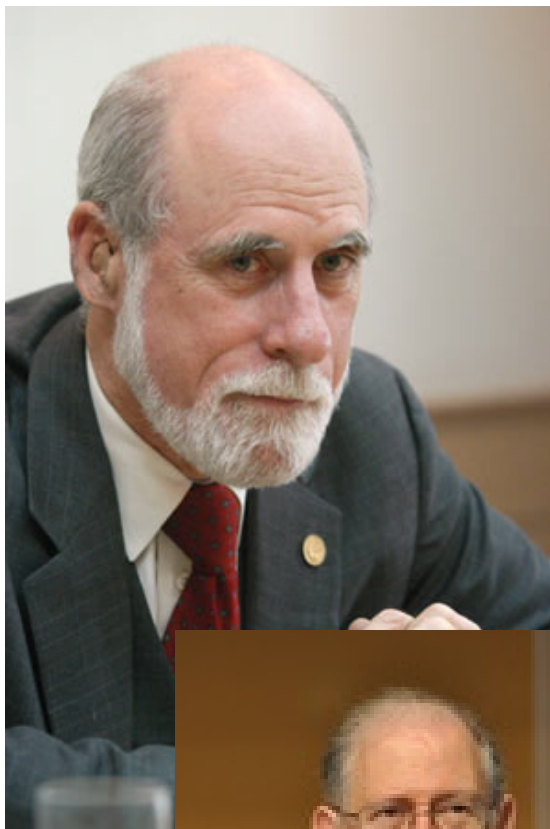
3.5.4可靠数据传输

3.5.5 流量控制

3.5.6 TCP连接管理

TCP/IP协议发明者

文顿·瑟夫



罗伯特·卡恩



- Vinton G. Cerf和Robert E. Kahn博士
- 在TCP/IP技术之前，已经有了互联技术(ARPA)，尽管它不足以支持10万台计算机的互联，但使用老的ARPA技术的人必然会抵抗TCP/IP技术。
- 他们用尽方法说服使用老技术的人接受新技术，和标准化组织ISO进行了长期斗争。直到1978年，才得到标准化认可，其实，当时标准化组织也推出了一套开放式互联的架构。从1983年到1993年，都在跟想使用标准化组织的互联架构的人做斗争。
- 1997年12月，克林顿总统颁发美国国家技术勋章
- 2004年，ACM颁发图灵奖
- 2005年11月，乔治布什总统颁发总统自由勋章

ARPA的信息处理技术办公室主任，他批准筹建了最终演变成互联网的政府计算机网络

互联网之父

鲍勃·泰勒

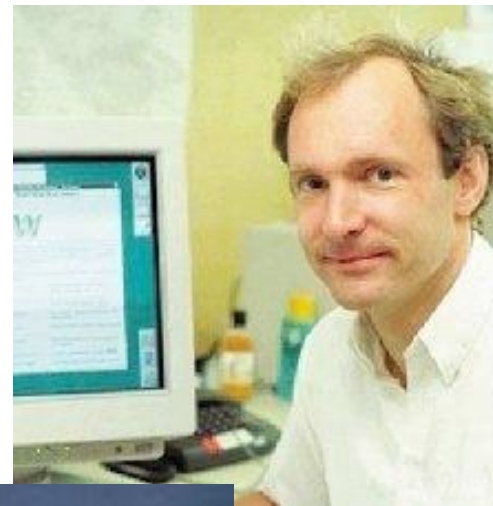


拉里·罗伯茨

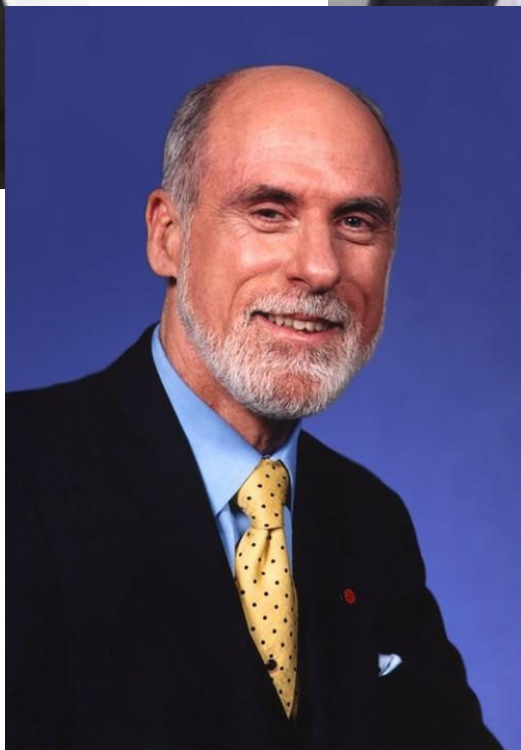


ARPANet项目技术负责人

蒂姆·伯纳斯·李



文顿·瑟夫



罗伯特·卡恩

3.5.1 TCP连接

- 面向连接:

- 在进行数据交换前, 初始化发送方与接收方状态, 进行握手(交换控制信息)
- 连接状态与端系统有关, 不为路由器所知

- 全双工:

- 同一连接上的双向数据流

- 点到点:

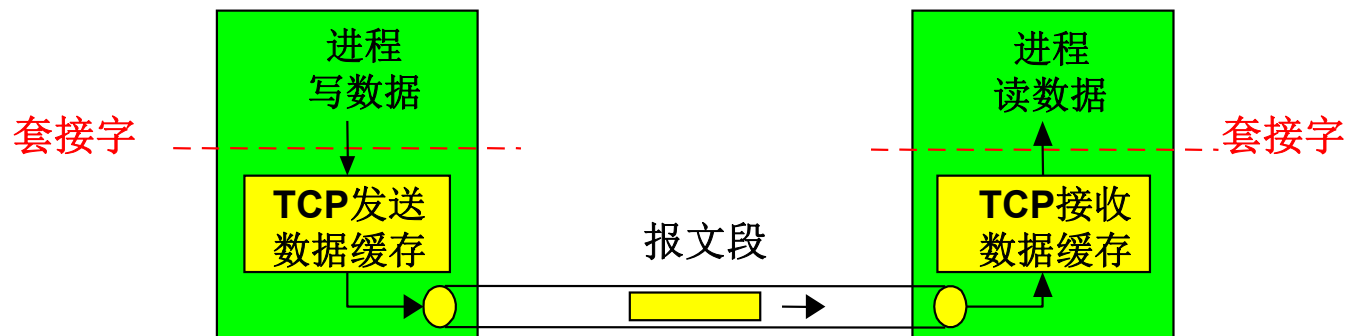
- 一个发送方和一个接收方之间的连接

- 流量控制:

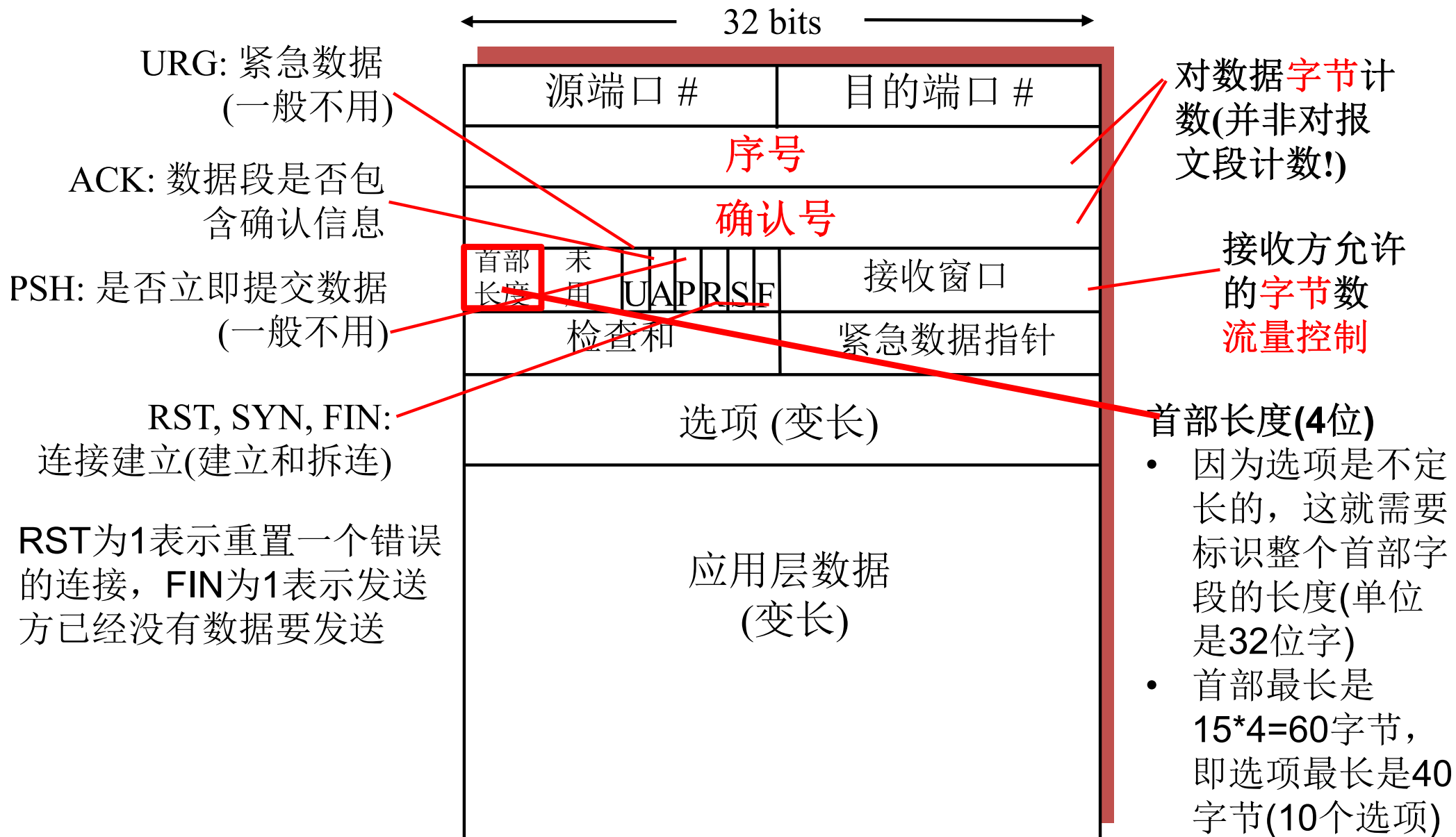
发送方发送速率不能超过接收方能力

- 拥塞控制:

网络拥塞时遏制发送方速率



3.5.2 TCP 报文段结构 P154

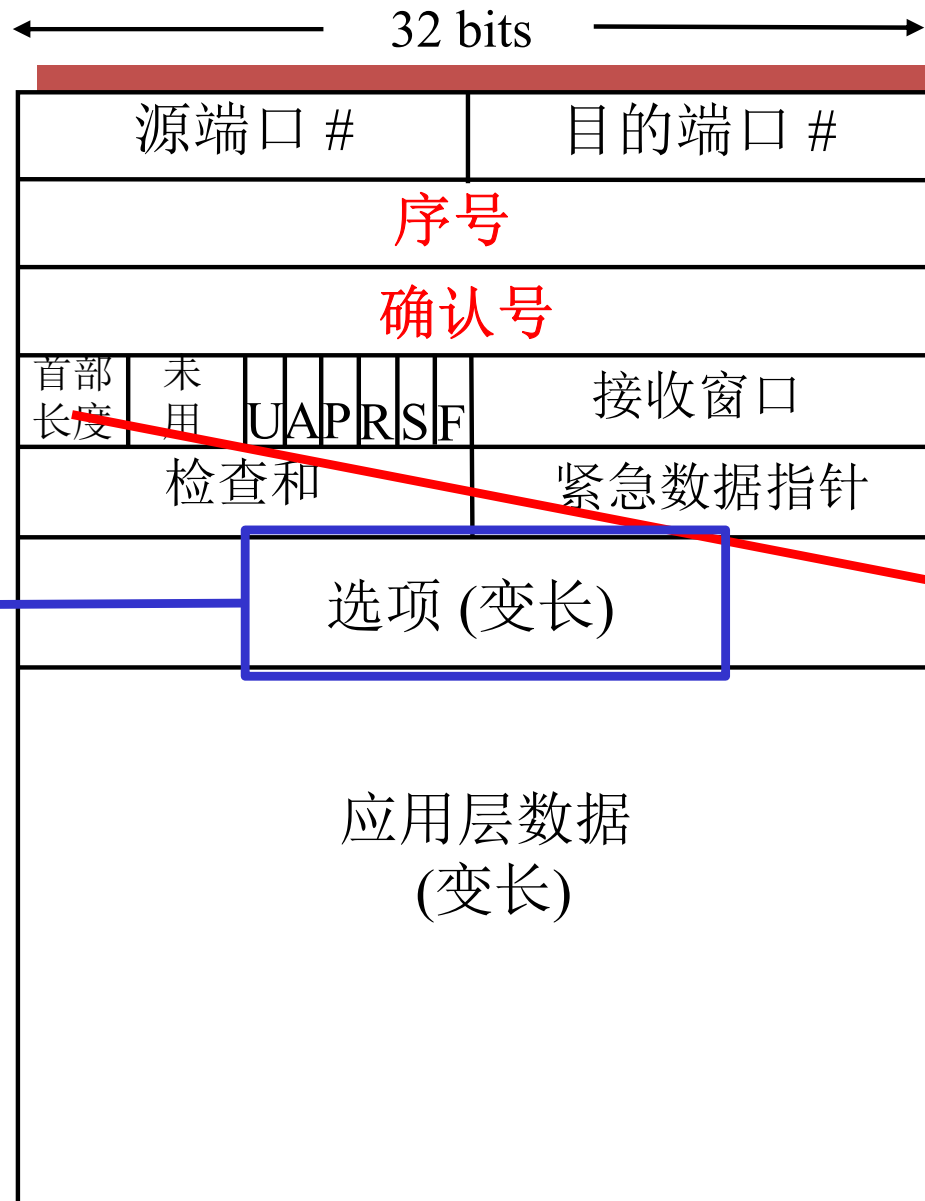


如果选项不是不是4字节的倍数怎么办呢?

3.5.2 TCP 报文段结构 P154

选项部分是为了适合复杂网络环境和更好地服务于应用层设计的包括

- 最大报文传输段 (MSS, Maximum Segment Size)
- 窗口扩大选项 (Window scaling)
- 选择确认选项 (SACK, Selective Acknowledgements)
- 时间戳选项 (timestamps)
- NOP(NO-Operation) 等



首部长度(4位)

- 因为选项是不定长的, 这就需要标识整个首部字段的长度(单位是32位字)
- 首部最长是 $15 \times 4 = 60$ 字节, 即选项最长是40字节(10个选项)

MSS-最大报文段长度 P153

- MTU: maximum transmission unit, 最大传输单元(最大链路层帧长度)
- 由硬件规定, 如以太网的MTU为1500字节
- MSS: maximum segment size, 最大报文段长度
- 为TCP数据包每次传输的最大数据分段大小
- MSS值为MTU值减去IPv4 Header (20 Byte) 和TCP header (20 Byte) 得到
- 最大为1460 (是一个典型值)
- 注意: MSS虽然叫最大报文段长度, 但实际上是指报文段里应用层数据的最大长度

```
Source port: groupwise (1677)
Destination port: http (80)
[Stream index: 0]
Sequence number: 0      (relative sequence number)
Header length: 32 bytes
```

[-] Flags: 0x002 (SYN)

000. = Reserved: Not set

```
...0 .... = Nonce: Not set
```

```
.... 0... .... = Congestion window Reduced (CWR): Not set
```

```
.....0..... = ECN-Echo: Not set
```

```
.....0. .... = Urgent: Not set
```

.... 0 = Acknowledgment: Not set

```

..... 0... = Push: Not set

```

```
.....0.. = Reset: Not set
```

```
.....1. = Syn: Set
```

```
.... 0 = Fin: Not set
```

```

window size value: 65535

```

```
[Calculated window size: 65535]
```

```
Checksum: 0x613b [validation disabled]
```

```
[Good Checksum: False]
```

```
[Bad Checksum: False]
```

Options: (12 bytes), Maximum segment size, No-Operation (NOP), window s

Maximum segment size: 1460 bytes

Kind: MSS size (2)

Length: 4

MSS value: 1460

No-Operation (NOP)

[-] window scale: 1 (multiply by 2)

Kind: window scale (3)

Length: 3

```
shift count: 1
```

[Multiplier: 2]

No-Operation (NOP)

No-Operation (NOP)

```

TCP SACK Permitted Option: True

```

Kind: SACK Permission (4)

Length: 2

```
0000 00 14 78 63 e8 96 00 1d 92 be a5 ca 08 00 45 00 ...XC... ..E.
0010 00 34 fb a6 40 00 80 06 eb 77 c0 a8 01 6b 77 4b ...4. @... ..kwk
0020 da 46 06 8d 00 50 3f 27 b4 32 00 00 00 00 80 02 ...F...P? ..2.
0030 ff ff 61 3b 00 00 02 04 05 b4 01 03 03 01 01 01 ...a;.....
0040 04 02
```

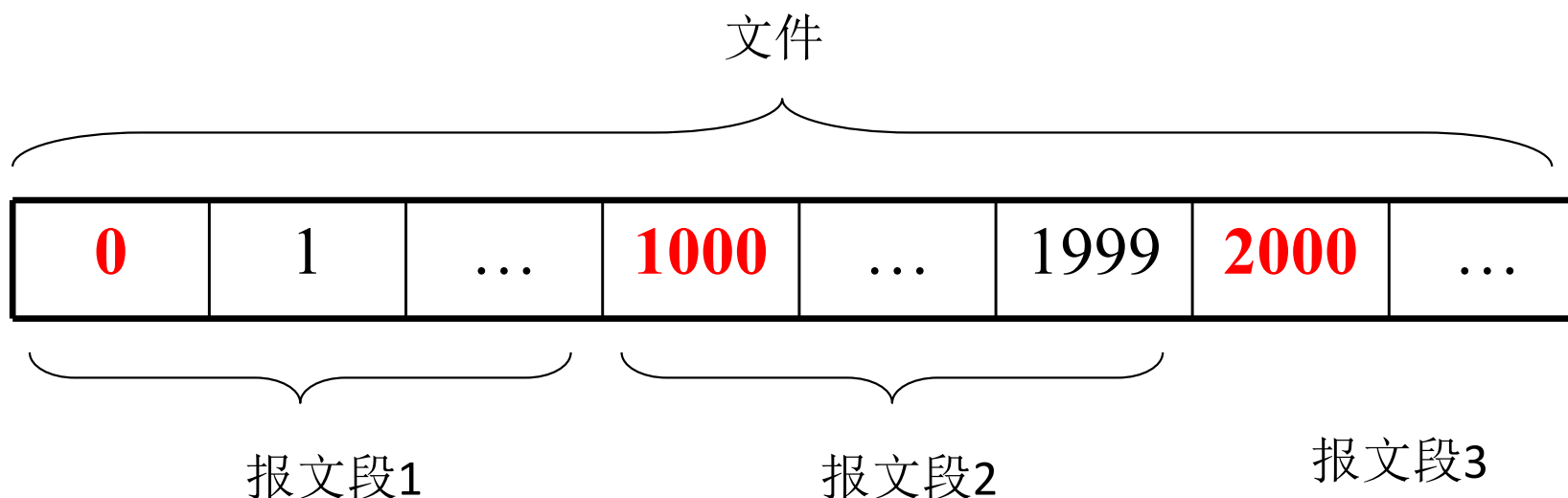
结合Wireshark捕获分组深入理解
TCP/IP协议栈之TCP协议(TCP报文
格式+三次握手实例)

<http://blog.chinaunix.net/uid-9112803-id-3212041.html>

TCP序号和确认号 P155

报文段是可靠、有序的字节流。

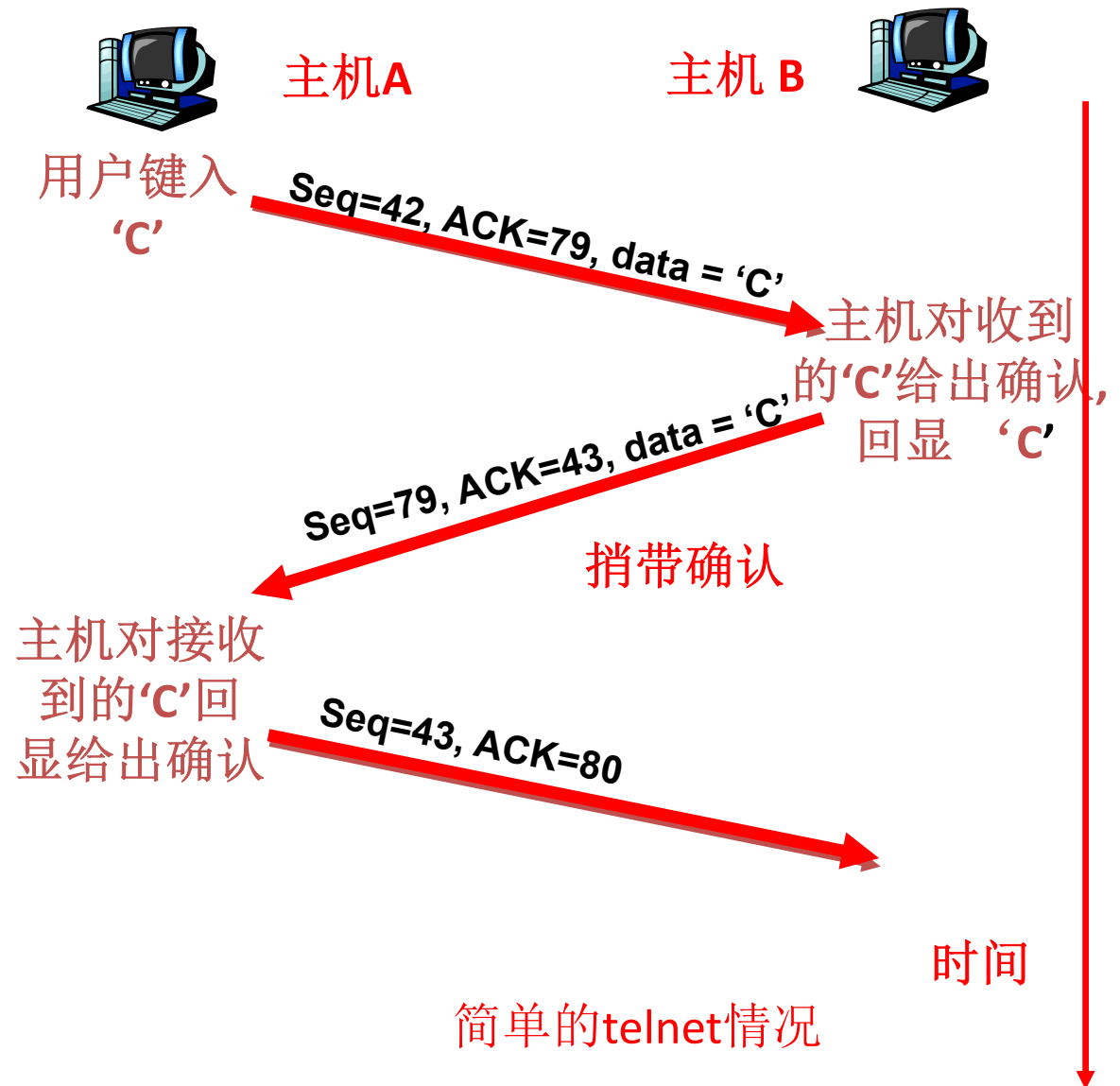
- 序号:
 - 报文段中第1个数据字节在字节流中的位置编号
- 确认号:
 - 期望从对方收到下一个字节的序号
 - 累计应答



例 P160

主机A: 起始序号42

主机B: 起始序号79



3.5.3 TCP Round Trip Time and Timeout P157

Q: how to set TCP timeout value?

- ❑ longer than RTT
 - but RTT varies
- ❑ too short: premature timeout
 - unnecessary retransmissions
- ❑ too long: slow reaction to segment loss

Q: how to estimate RTT?

- ❑ **SampleRTT**: measured time from segment transmission until ACK receipt
 - ignore retransmissions
- ❑ **SampleRTT** will vary, want estimated RTT "smoother"
 - average several recent measurements, not just current **SampleRTT**

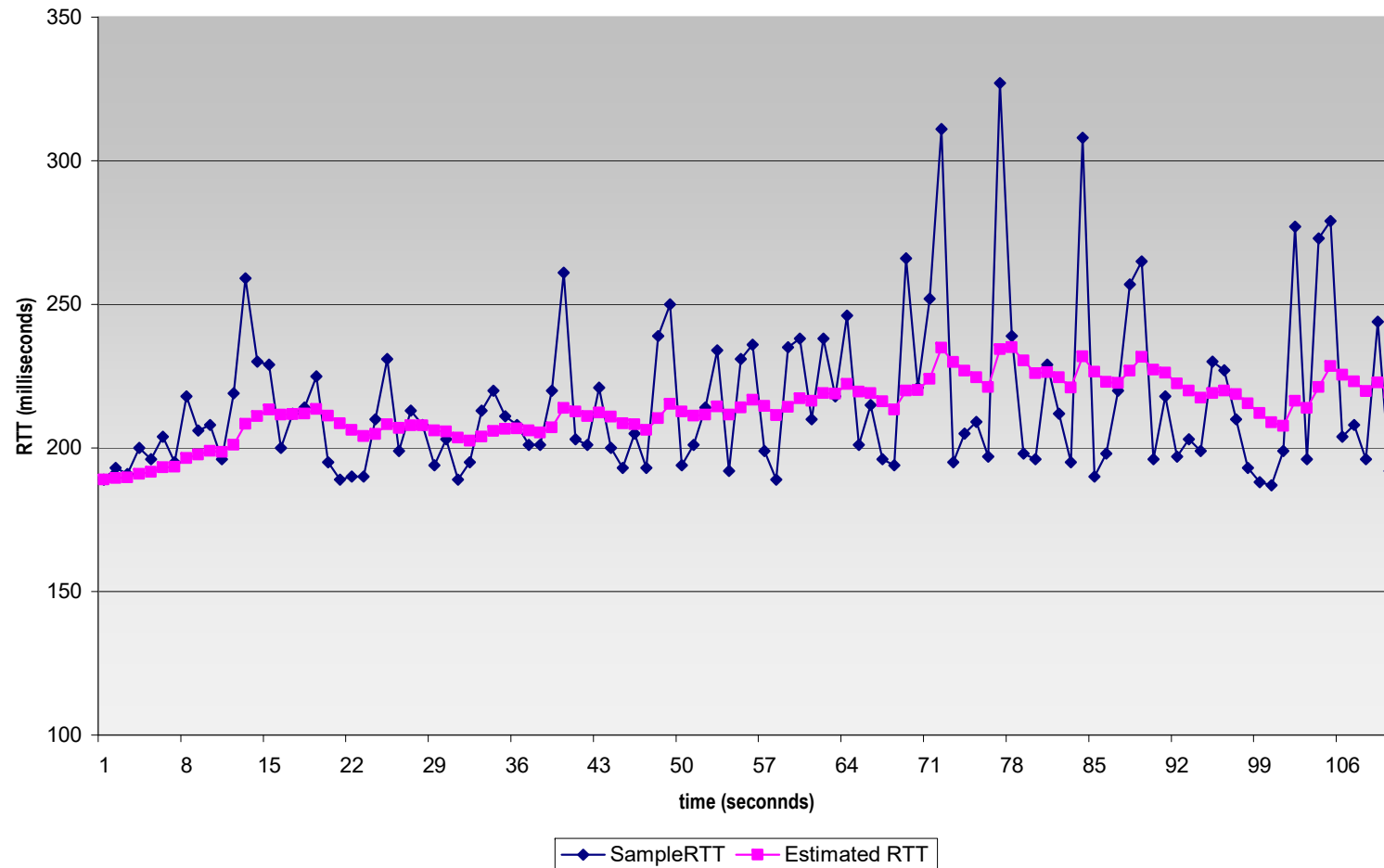
TCP Round Trip Time and Timeout

$$\text{EstimatedRTT} = (1 - \alpha) * \text{EstimatedRTT} + \alpha * \text{SampleRTT}$$

- ❑ Exponential weighted moving average 指数加权平均移动
- ❑ influence of past sample decreases exponentially fast
之前样本在更新过程中呈指数型快速衰减
- ❑ typical value: $\alpha = 0.125$

Example RTT estimation

RTT: gaia.cs.umass.edu to fantasia.eurecom.fr



$$\text{EstimatedRTT} = (1 - \alpha) * \text{EstimatedRTT} + \alpha * \text{SampleRTT}$$

typical value: $\alpha = 0.125$

TCP Round Trip Time and Timeout

- 预测RTT的变化
- DevRTT估算每次SampleRTT一般会偏离EstimatedRTT的程度
- first estimate of how much SampleRTT deviates from EstimatedRTT:

$$\text{DevRTT} = (1-\beta) * \text{DevRTT} + \beta * |\text{SampleRTT} - \text{EstimatedRTT}|$$

(typically, $\beta = 0.25$)

Then set timeout interval:

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 * \text{DevRTT}$$

例题

Suppose that TCP's current estimated values for the round trip time (estimatedRTT) and deviation in the RTT (DevRTT) are 310 msec and 50 msec, respectively (see Section 3.5.3 for a discussion of these variables). Suppose that the next three measured values of the RTT are 220, 240, and 300 respectively.

Compute TCP's new value of estimatedRTT , DevRTT , and the TCP timeout value after each of these three measured RTT values is obtained. Use the values of $\alpha = 0.125$ and $\beta = 0.25$.

Solution:

After the first RTT estimate is made:

$$\begin{aligned}\text{estimatedRTT} &= 0.875 \times 310 + 0.125 \times 220 = 298.75 \text{ msec} \\ \text{DevRTT} &= 0.75 \times 50 + 0.25 \times (\text{abs}(220 - 298.75)) = 57.1875 \text{ msec} \\ \text{TimeoutInterval} &= 298.75 + 4 \times 57.1875 = 527.5 \text{ msec}\end{aligned}$$

After the second RTT estimate is made:

$$\begin{aligned}\text{estimatedRTT} &= 0.875 \times 298.75 + 0.125 \times 240 = 291.40625 \text{ msec} \\ \text{DevRTT} &= 0.75 \times 57.1875 + 0.25 \times (\text{abs}(240 - 291.40625)) = 55.7421875 \text{ msec} \\ \text{TimeoutInterval} &= 291.40625 + 4 \times 55.7421875 = 514.375 \text{ msec}\end{aligned}$$

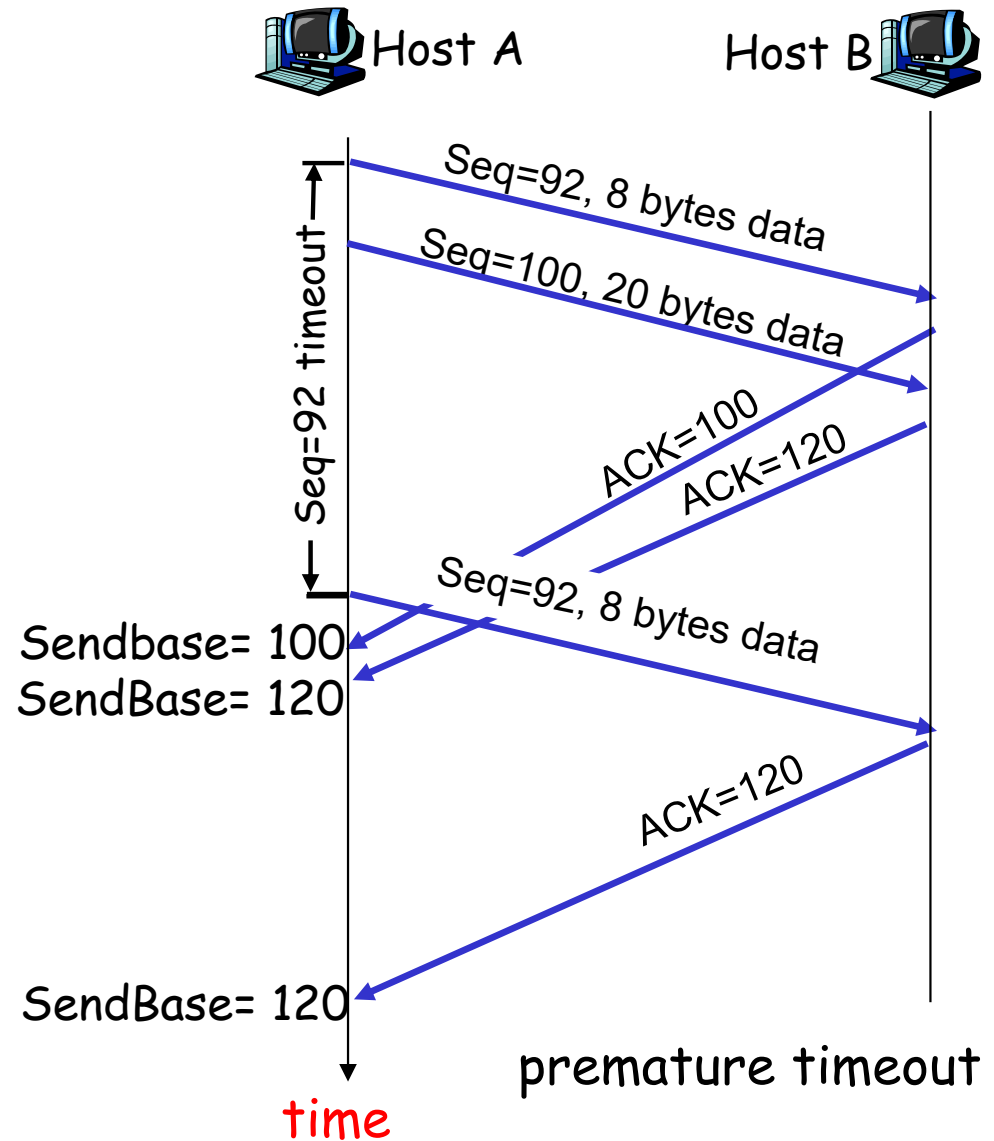
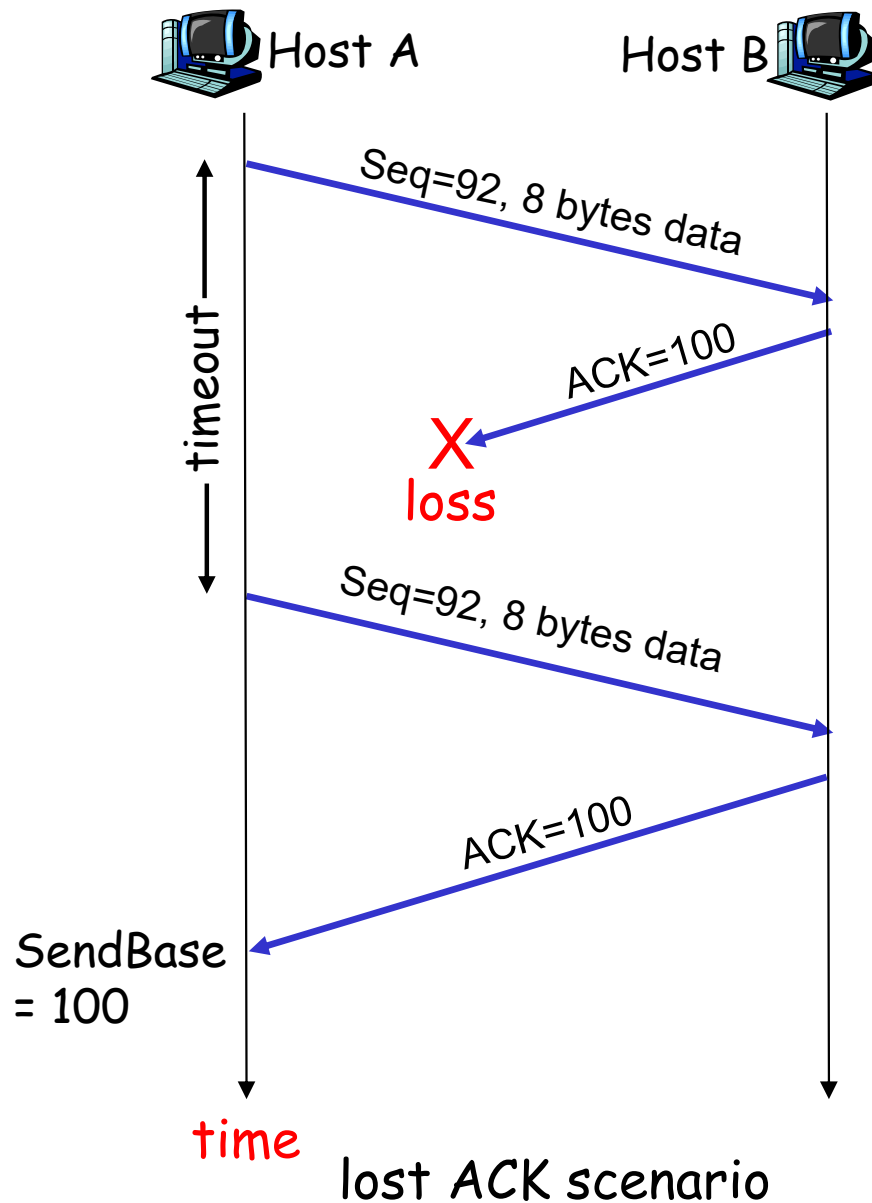
After the third RTT estimate is made:

$$\begin{aligned}\text{estimatedRTT} &= 0.875 \times 291.40625 + 0.125 \times 300 = 292.48046875 \text{ msec} \\ \text{DevRTT} &= 0.75 \times 55.7421875 + 0.25 \times (\text{abs}(300 - 292.48046875)) = 55.7421875 \text{ msec} \\ \text{TimeoutInterval} &= 292.48046875 + 4 \times 55.7421875 = 467.2265625 \text{ msec}\end{aligned}$$

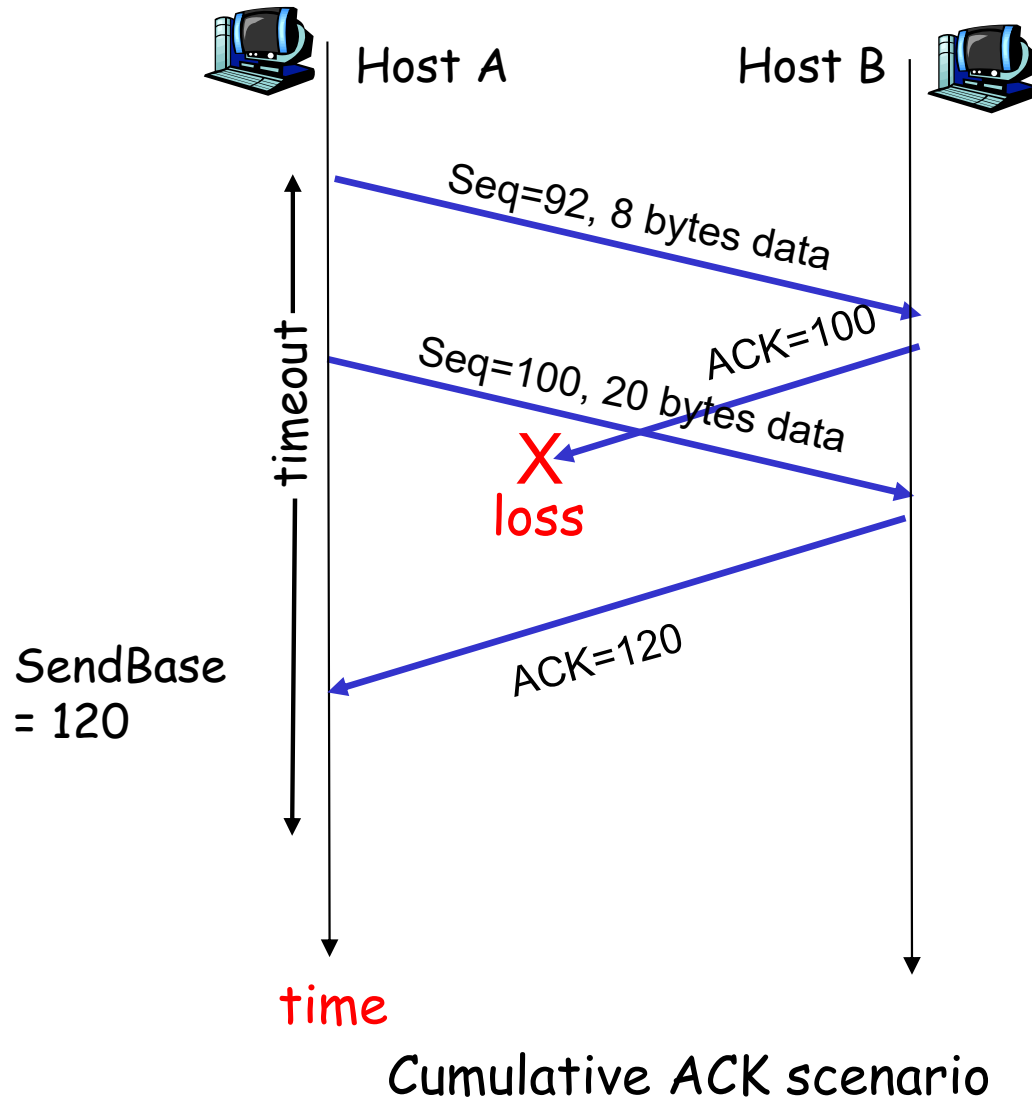
3.5.4 TCP reliable data transfer P159

- ❑ TCP creates rdt service on top of IP's unreliable service
- ❑ Pipelined segments
- ❑ Cumulative acks
- ❑ TCP uses single retransmission timer
- ❑ Retransmissions are triggered by:
 - timeout events
 - duplicate acks

TCP: retransmission scenarios



TCP retransmission scenarios (more)



问题：Host A收到ACK=120后
会不会重新发送Seq=100的那个分组？

产生TCP ACK的建议 [RFC 5681] P163

接收方事件

具有**期望序号**的**按序报文段**到达，所有在期望序号及以前的数据**都已经被确认** **0**

具有**期望序号**的**按序报文段**到达，**另一个按序报文段等待ACK传输**
0 1

比期望序号大的失序报文段到达，**检测出间隔**
0 1 4

能部分或完全填充接收数据间隔的报文段到达

0 1 4 0 1 2 4 0 1 3 4

接收方动作

延迟的**ACK**。对另一个按序报文段的到达最多等待**500ms**，**500ms**内没有达到，则发送一个**ACK**

立即发送单个累积**ACK**，以确认两个按序报文段
ACK=2

立即发送冗余**ACK**，指示下一个期待字节的序号（其为间隔的低端的序号）
ACK=2

倘若该报文段起始于间隔的低端，则立即发送**ACK**
ACK=3 ACK=2

Fast Retransmit

- ❑ Time-out period often relatively long:
 - long delay before resending lost packet
- ❑ Detect lost segments via duplicate ACKs.
 - Sender often sends many segments back-to-back
 - If segment is lost, there will likely be many duplicate ACKs.
- ❑ If sender receives 3 duplicate ACKs for the same data, it supposes that segment after ACKed data was lost:
 - fast retransmit: resend segment before timer expires

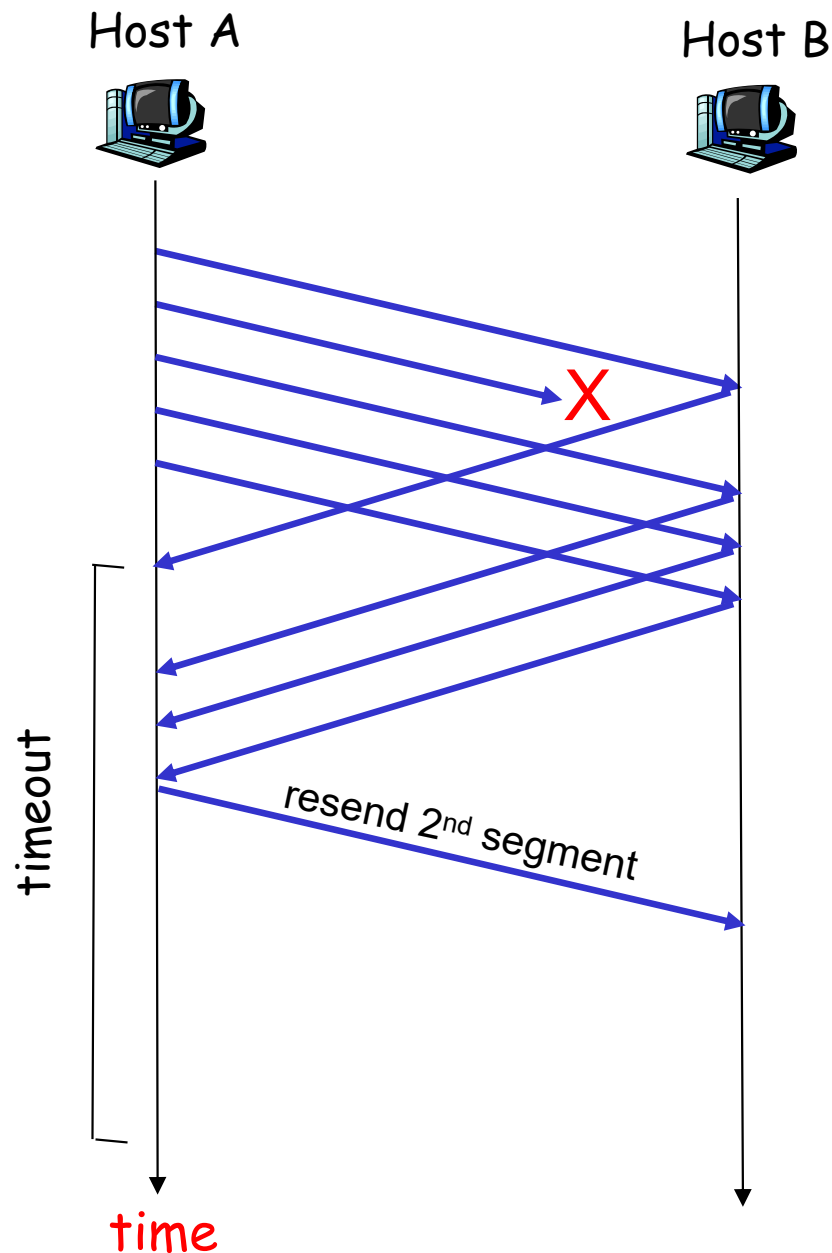
SACK

Fast Retransmit只解决了一个问题，就是快速重传的问题，它依然面临一个艰难的选择，就是重传之前的一个还是重传多个的问题。

对于示例来说，是重传#2呢还是重传#2，#3，#4，#5呢？因为发送端并不清楚冗余的3个ack(2)是谁传回来的？也许发送端发了20份数据，是#6，#10，#20传来的呢。

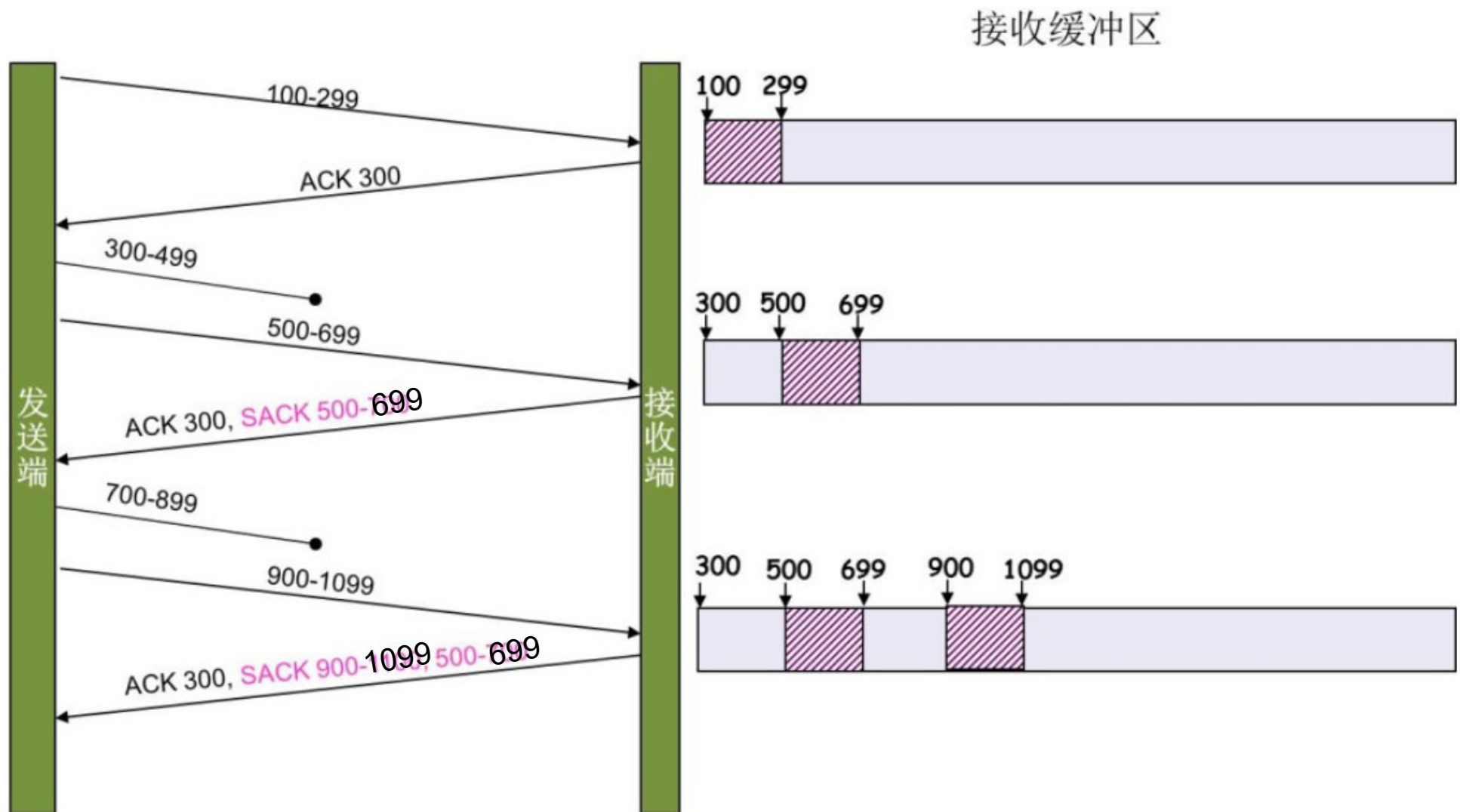
Selective Acknowledgment (SACK)

（参看[RFC 2018](#)），这种方式需要在TCP首部里加SACK，ACK还是Fast Retransmit的ACK，SACK则是汇报收到的数据碎片。



Resending a segment after triple duplicate ACK

SACK

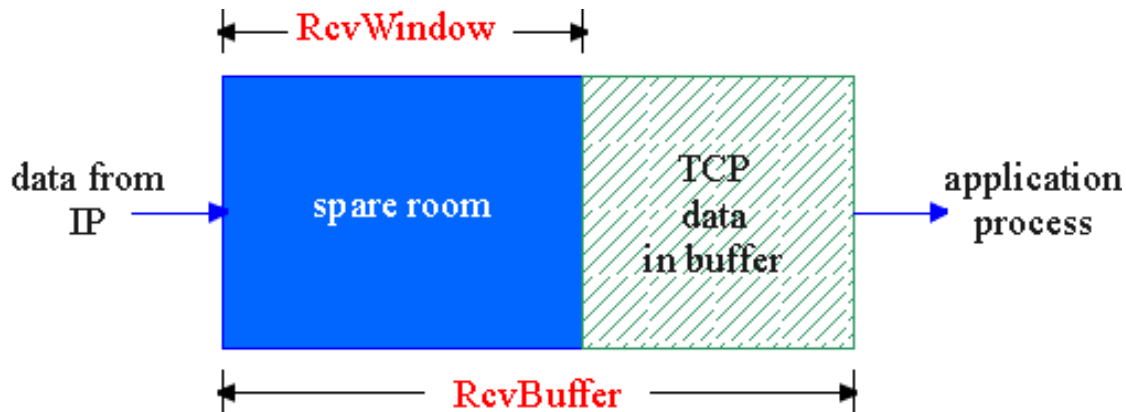


强烈推荐 TCP的那些事儿

<http://www.cppblog.com/weiyu/archive/2014/05/29/207145.html>

3.5.5 TCP Flow Control P164

- receive side of TCP connection has a receive buffer:



- app process may be slow at reading from buffer

flow control
sender won't overflow receiver's buffer by transmitting too much, too fast

- speed-matching service: matching the send rate to the receiving app's drain rate

问题: **RevWindow=0 ?**

发送方继续发送只有一个字节数据的报文段

3.5.6 TCP Connection Management P166

Recall: TCP sender, receiver establish "connection" before exchanging data segments

□ initialize TCP variables:

- seq. #s
- buffers, flow control info (e.g. RcvWindow)

□ *client*: connection initiator

```
Socket clientSocket = new  
Socket("hostname", "port  
number");
```

□ *server*: contacted by client

```
Socket connectionSocket =  
welcomeSocket.accept();
```

Three way handshake:

Step 1: client host sends TCP SYN segment to server

- specifies initial seq #
- no data

Step 2: server host receives SYN, replies with SYNACK segment

- server allocates buffers
- specifies server initial seq. #

Step 3: client receives SYNACK, replies with ACK segment, which may contain data

client state

LISTEN

SYNSENT

ESTAB

choose init seq num, x
send TCP SYN msg

received SYNACK(x)
indicates server is live;
send ACK for SYNACK;
this segment may contain
client-to-server data



server state

LISTEN

SYN RCVD

ESTAB

SYNbit=1, Seq=x

SYNbit=1, Seq=y
ACKbit=1; ACKnum=x+1

ACKbit=1, ACKnum=y+1

choose init seq num, y
send TCP SYNACK
msg, acking SYN

received ACK(y)
indicates client is live

Transmission Control Protocol, Src Port: groupwise (1677), Dst Port: http

Source port: groupwise (1677)

Destination port: http (80)

[Stream index: 0]

Sequence number: 0 (relative sequence number)

Header length: 32 bytes

Flags: 0x002 (SYN)

000. = Reserved: Not set

...0 = Nonce: Not set

.... 0... = Congestion window Reduced (CWR): Not set

.... .0.. = ECN-Echo: Not set

.... ..0. = Urgent: Not set

.... ...0 = Acknowledgment: Not set

.... 0... = Push: Not set

....0.. = Reset: Not set

+1. = Syn: Set

....0 = Fin: Not set

Window size value: 65535

[Calculated window size: 65535]

Checksum: 0x613b [validation disabled]

[Good checksum: False]

[Bad checksum: False]

Options: (12 bytes), Maximum segment size, No-Operation (NOP), window s

Maximum segment size: 1460 bytes

Kind: MSS size (2)

Length: 4

MSS value: 1460

No-Operation (NOP)

Window scale: 1 (multiply by 2)

Kind: window scale (3)

Length: 3

Shift count: 1

[Multiplier: 2]

No-Operation (NOP)

No-Operation (NOP)

TCP SACK Permitted option: True

Kind: SACK Permission (4)

Length: 2

```
0000 00 14 78 63 e8 96 00 1d 92 be a5 ca 08 00 45 00 ...XC...E.
0010 00 34 fb a6 40 00 80 06 eb 77 c0 a8 01 6b 77 4b .4..G..w..kWK
0020 da 46 06 8d 00 50 3f 27 b4 32 00 00 00 00 80 02 .F...P?...2....
0030 ff ff 61 3b 00 00 02 04 05 b4 01 03 03 01 01 01 ...a;.....
0040 04 02
```

```

Transmission Control Protocol, Src Port: http (80), Dst Port: groupwise
Source port: http (80)
Destination port: groupwise (1677)
[Stream index: 0]
Sequence number: 0      (relative sequence number)
Acknowledgment number: 1      (relative ack number)
Header length: 32 bytes
[-] Flags: 0x012 (SYN, ACK)
    000. .... = Reserved: Not set
    ...0 .... = Nonce: Not set
    .... 0... = Congestion Window Reduced (CWR): Not set
    .... .0.. = ECN-Echo: Not set
    .... ..0. = Urgent: Not set
    .... ...1 = Acknowledgment: Set
    .... .... 0... = Push: Not set
    .... .... .0.. = Reset: Not set
    [+]. .... ..1. = Syn: Set
    .... .... ...0 = Fin: Not set
Window size value: 65535
[Calculated window size: 65535]
[-] Checksum: 0x897a [validation disabled]
    [Good checksum: False]
    [Bad checksum: False]
[-] Options: (12 bytes), Maximum segment size, No-operation (NOP), No-operation (NOP)
    [-] Maximum segment size: 1452 bytes
        Kind: MSS size (2)
        Length: 4
        MSS value: 1452
        No-operation (NOP)
        No-operation (NOP)
        No-operation (NOP)
        No-operation (NOP)
    [+]. [Expert Info (warn/Protocol): 4 NOP in a row - a router may have removed them]
        No-operation (NOP)
        No-operation (NOP)
    [-] TCP SACK Permitted option: True
        Kind: SACK Permission (4)
        Length: 2
[-] [SEQ/ACK analysis]
    [This is an ACK to the segment in frame: 19]
    [The RTT to ACK the segment was: 0.041570000 seconds]

```

```

00 00 1d 92 be a5 ca 00 14 78 63 e8 96 08 00 45 00 ..XC...E.
10 00 34 fb a6 40 00 30 06 3b 78 77 4b da 46 c0 a8 .41.0.0. xwk.F..
20 01 6b 00 50 06 8d e8 80 f1 38 3f 27 b4 33 80 12 .k.P....8?..3..
30 ff ff 89 7a 00 00 02 04 05 ac 01 01 01 01 01 01 ...Z.....
40 04 02

```

ip.src==47.95.165.112 or ip.dst==47.95.165.112						
No.	Time	Source	Destination	Protocol	Length	Info
306	4.468765	192.168.1.101	47.95.165.112	TCP	66	64940 → 80 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM=1
307	4.469274	192.168.1.101	47.95.165.112	TCP	66	64941 → 80 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM=1
308	4.469631	192.168.1.101	47.95.165.112	TCP	66	64942 → 80 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM=1
362	4.517554	47.95.165.112	192.168.1.101	TCP	66	80 → 64939 [SYN, ACK] Seq=0 Ack=1 Win=14600 Len=0 MSS=1460 SACK_PERM=1 WS=1
363	4.517666	192.168.1.101	47.95.165.112	TCP	54	64939 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0

4. 服务器gaia.cs.umass.edu向客户端发送的SYNACK报文段序号是多少？在该报文段中，确认标识（Acknowledgement）的值是多少Gaia.cs.umass.edu服务器是如何决定确认标识（Acknowledgement）值的？在该报文段中，是用什么来标示该报文段是SYNACK报文段的？

参考答案：服务器向客户端发送的SYNACK报文段序号是0，确认标识是1。

Sequence number: 0 (relative sequence number)

Acknowledgement number: 1 (relative ack number)

服务器在syn报文段的序号上增加了1，这个值作为确认值。

在报文段用，通过设置ACK和SYN标识位来识别这一个SYN-ACK片段的。

Flags: 0x12 (SYN, ACK)

000. = Reserved: Not set

...0 = Nonce: Not set

.... 0... = Congestion Window Reduced (CWR): Not set

.... .0.. = ECN-Echo: Not set

.... ..0. = Urgent: Not set

.... ...1 = Acknowledgement: Set

.... 0... = Push: Not set

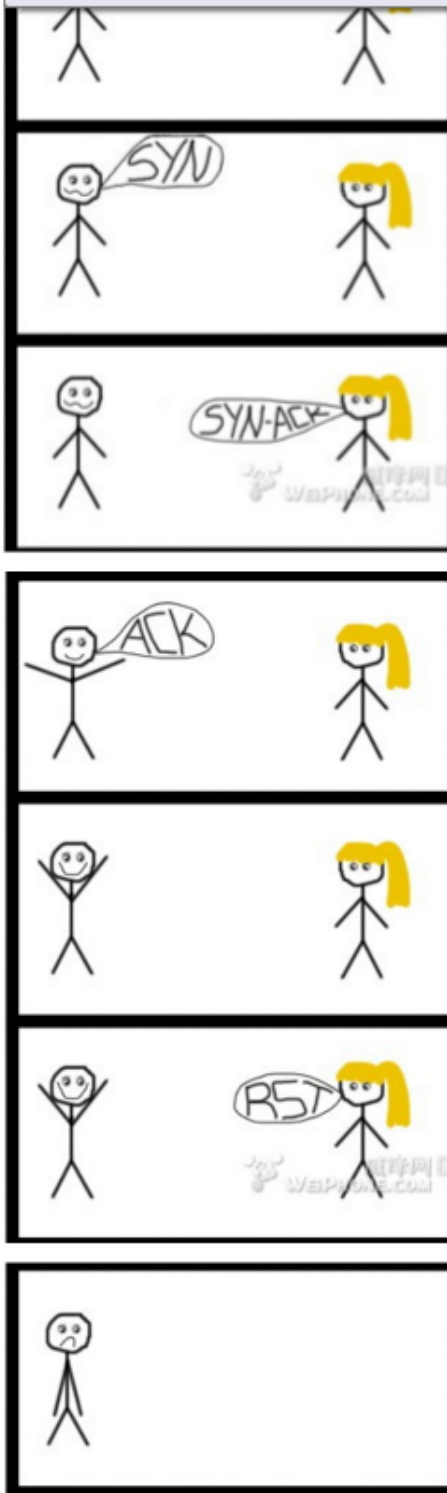
....0.. = Reset: Not set

⊕1. = Syn: Set

....0 = Fin: Not set

Window size: 5840

趣图：谈恋爱就像TCP连接...



RST标示复位、用来异常的关闭连接。

SYN洪泛攻击

- ❑ 客户端发送**SYN**报文段
- ❑ 服务器分配并初始化连接变量和缓存，并发**SYNACK**进行响应，并等待来自客户的**ACK**
- ❑ 客户端不进行第三步的确认
- ❑ 服务器将在一分多钟后，终止半开连接并回收资源
- ❑ 很多这种行为就形成**SYN洪泛攻击**
- ❑ 是**DoS**攻击的一种

TCP Connection Management (cont.)

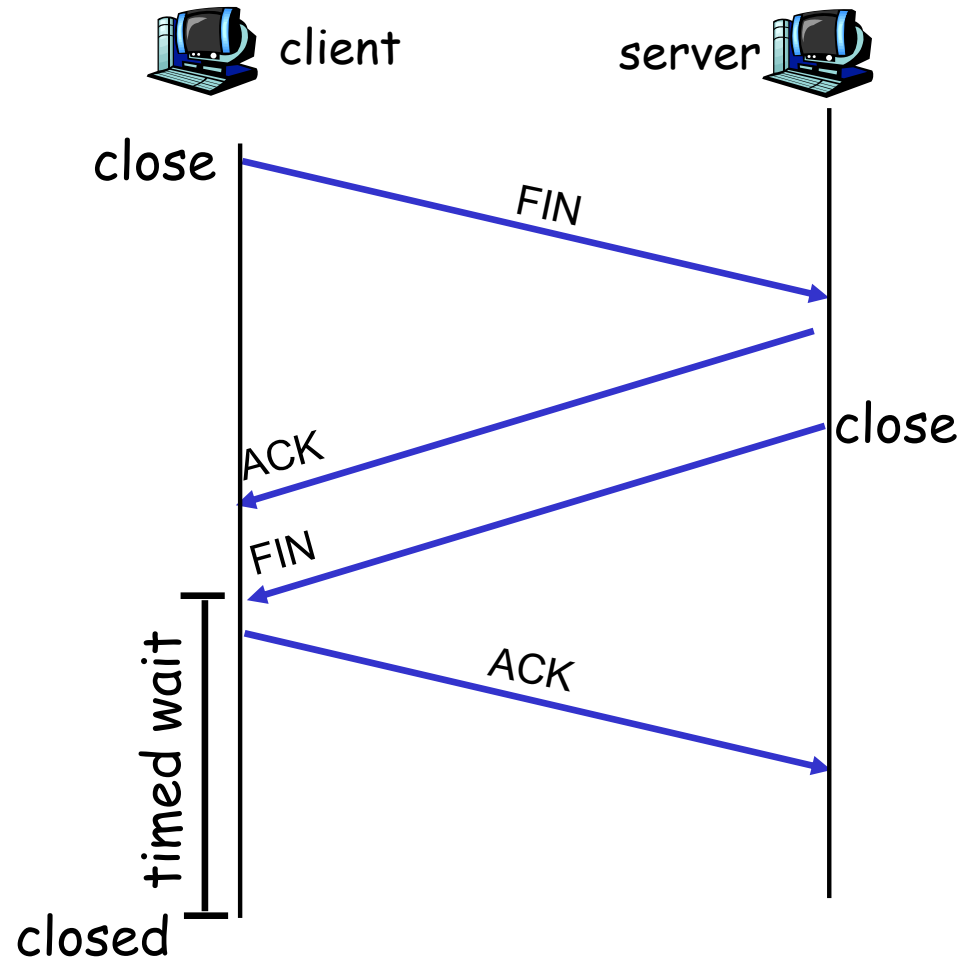
Closing a connection:

client closes socket:

```
clientSocket.close();
```

Step 1: client end system
sends TCP FIN control
segment to server

Step 2: server receives
FIN, replies with ACK.
Closes connection, sends
FIN.



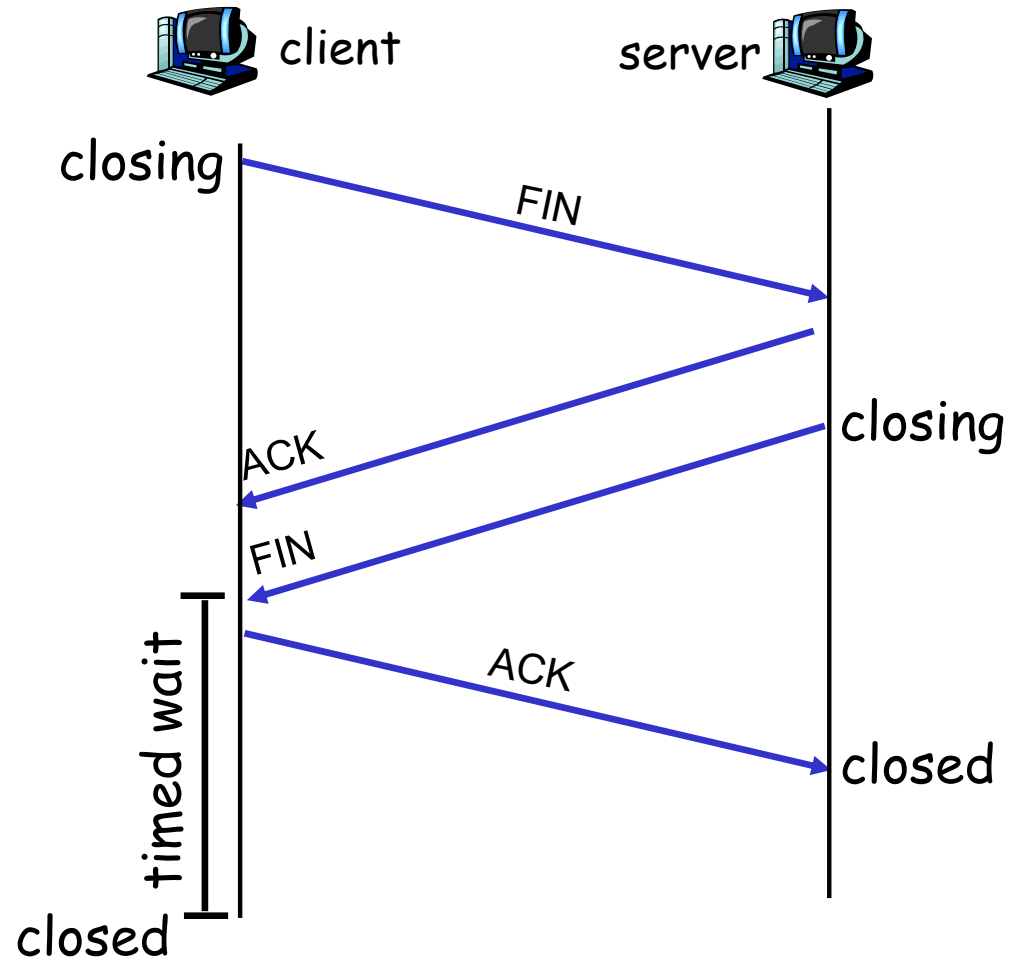
TCP Connection Management (cont.)

Step 3: client receives FIN,
replies with ACK.

- Enters "timed wait" -
will respond with ACK
to received FINs

Step 4: server, receives
ACK. Connection closed.

Note: with small
modification, can handle
simultaneous FINs.



实验分析

- ❑ 观察浏览器建立连接的过程
- ❑ 观察浏览器拆除连接的过程
- ❑ 建议标准与实际实现的差别



TCP对接收方产生ACK的建议

接收方事件

接收方动作

具有期望序号的按序报文段到达，所有在期望序号及以前的数据都已经被确认

延迟的ACK。对另一个按序报文段的到达最多等待500ms，500ms内没有达到，则发送一个ACK

具有期望序号的按序报文段到达，另一个按序报文段等待ACK传输

立即发送单个累积ACK，以确认两个按序报文段
ACK=2

比期望序号大的失序报文段到达，检测出间隔

立即发送冗余ACK，指示下一个期待字节的序号（其为间隔的低端的序号）
ACK=2

能部分或完全填充接收数据间隔的报文段到达

倘若该报文段起始于间隔的低端，则立即发送ACK

0 1 4 0 1 2 4 0 1 3 4

ACK=3 ACK=2

查看 ACK 是哪个报文段引起的响应？

89	2.189763	128.119.245.12	192.168.1.107	TCP	60	80 → 65162 [ACK] Seq=1 Ack=43898 Win=117120 Len=0
▼ Transmission Control Protocol						
Source Port: 80						
Destination Port: 65162						
[Stream index: 3]						
[TCP Segment Len: 0]						
Sequence number: 1 (relative sequence number)						
Acknowledgment number: 43898 (relative ack number)						
Header Length: 20 bytes						
➤ Flags: 0x010 (ACK)						
Window size value: 915						
[Calculated window size: 117120]						
[Window size scaling factor: 128]						
➤ Checksum: 0x60d6 [validation disabled]						
Urgent pointer: 0						
▼ [SEQ/ACK analysis]						
[This is an ACK to the segment in frame: 51]						
[The RTT to ACK the segment was: 0.364851000 seconds]						
[iRTT: 0.370706000 seconds]						

接收窗口大小 $RevWin$ $rwin$

- ❑ 发送方接收窗口
- ❑ 接收方接收窗口

```
Window size value: 4162  
[Calculated window size: 1065472]  
[Window size scaling factor: 256]
```

谢谢大家！

Chapter 3

Transport Layer

A note on the use of these PowerPoint slides:

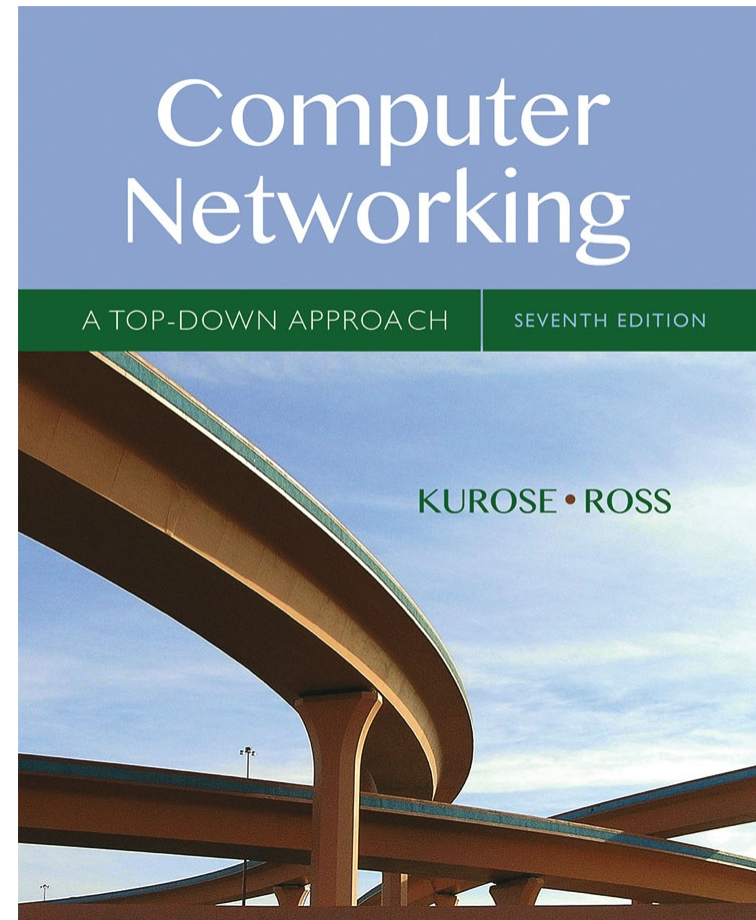
We're making these slides freely available to all (faculty, students, readers). They're in PowerPoint form so you see the animations; and can add, modify, and delete slides (including this one) and slide content to suit your needs. They obviously represent a *lot* of work on our part. In return for use, we only ask the following:

- If you use these slides (e.g., in a class) that you mention their source (after all, we'd like people to use our book!)
- If you post any slides on a www site, that you note that they are adapted from (or perhaps identical to) our slides, and note our copyright of this material.

For a revision history, see the slide note for this page.

Thanks and enjoy! JFK/KWR

All material copyright 1996-2020
J.F Kurose and K.W. Ross, All Rights Reserved



*Computer Networking: A
Top-Down Approach*
7th edition
Jim Kurose, Keith Ross
Pearson, 2016

Chapter 3 outline

- ❑ 3.1 Transport-layer services
- ❑ 3.2 Multiplexing and demultiplexing
- ❑ 3.3 Connectionless transport: UDP
- ❑ 3.4 Principles of reliable data transfer
- ❑ 3.5 Connection-oriented transport: TCP
 - segment structure
 - reliable data transfer
 - flow control
 - connection management
- ❑ 3.6 Principles of congestion control
- ❑ 3.7 TCP congestion control

3.6 拥塞控制原理 P170

- **拥塞**：太多的源发送太多太快的数据，使网络来不及处理。
- 表现：
 - **丢包**（路由器缓冲区溢出）
 - **长时延**（路由器缓冲区中排队）
- 对丢失的分组重传，不能解决网络拥塞问题。
- **网络拥塞时需要遏制发送方。**

网络中的10大主要问题之一！

本节内容

- ✓ 网络拥塞如何从网络层得到的服务中反映出来
- ✓ 如何避免网络拥塞，或对其作出反应

3.6.1 拥塞原因与开销（代价）

3.6.2 拥塞控制方法

3.6.3 网络辅助拥塞控制例

3.6.1 拥塞原因与代价

- 通过分析三种拥塞的情况，了解：
 - ✓ **拥塞原因**（当分组到达速率接近/超过链路容量时）
 - ✓ **拥塞的代价**（分组排队时延非常大，甚至被丢包）

网络拥塞丢弃分组的代价

- 当一个分组沿一条路径被丢弃时，每个上游路由器用于转发该分组到丢弃该分组而使用的传输容量最终被浪费掉了，即做“无用功”。

如，当有一个分组在第二跳路由器上被丢弃时，第一跳路由器所做的将分组转发到第二跳路由器的工作就是无用的。

- 如果路由器只是简单地丢弃分组，则网络中的情况逐步变差。
- 一种改进：当路由器选择一个分组进行发送时，**优先考虑已经历过一定数量的上游路由器的分组。**

3.6.2 拥塞控制方法P175

根据网络层是否为运输层拥塞控制提供明确的帮助，分为两种：

- **端到端拥塞控制：** 网络层没有为运输层拥塞控制提供明确的支持。

端系统必须通过对网络行为的观察 (如分组丢失与时延) 来推断网络中是否存在拥塞。

总体来说，TCP拥塞控制是端到端的

- **网络辅助的拥塞控制：** 网络层构件 (如路由器)向发送方提供关于网络拥塞状态的明确反馈信息
 - ✓ **直接反馈：** 路由器直接通过阻塞分组告知发送方
 - ✓ **经由接收方的反馈：** 路由器对从发送方流向接收方的分组中的某个字段进行标记或更新，来指示是否拥塞
 - 当接收方收到有拥塞标记的分组后，就通知发送方网络发生了拥塞（至少要经过一个完整的往返时间）

最近IP和TCP也能够选择性地实现网络辅助拥塞控制

Chapter 3 outline

- ❑ 3.1 Transport-layer services
- ❑ 3.2 Multiplexing and demultiplexing
- ❑ 3.3 Connectionless transport: UDP
- ❑ 3.4 Principles of reliable data transfer
- ❑ 3.5 Connection-oriented transport: TCP
 - segment structure
 - reliable data transfer
 - flow control
 - connection management
- ❑ 3.6 Principles of congestion control
- ❑ 3.7 TCP congestion control

3.7 TCP 拥塞控制 P176

- 主要是端到端控制 (没有网络辅助)
- 由发送方根据感知到的拥塞程度，来限制其发送流量的速率。
- ✓ 无拥塞：增加发送速率
- ✓ 有拥塞：降低发送速率
- 问题：
 - ✓ 如何限制发送速率
 - ✓ 如何感知拥塞
 - ✓ 拥塞时如何改变发送速率

限制发送速率

- **设置拥塞窗口CWin:** 对发送方向网络的发送进行限制
- ✓ 限制发送方发送但还未被确认的数据量outstanding data, 限制了这个数据量, 也就间接限制了发送方的发送速率

TCP Congestion Control: details

如何感知拥塞？

- sender limits transmission:

$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{CWin}$$

- Roughly,

$$\text{rate} = \frac{\text{CWin}}{\text{RTT}} \text{ Bytes/sec}$$

- CWin is dynamic, function of perceived network congestion

How does sender perceive congestion?

- loss event = timeout or 3 duplicate acks (哪个更严重？)
- TCP sender reduces rate (CWin) after loss event 降低速率
- 以上是拥塞情况，正常情况的感知是确认的到达 (ACK到达)
- 正常，则增加发送速率

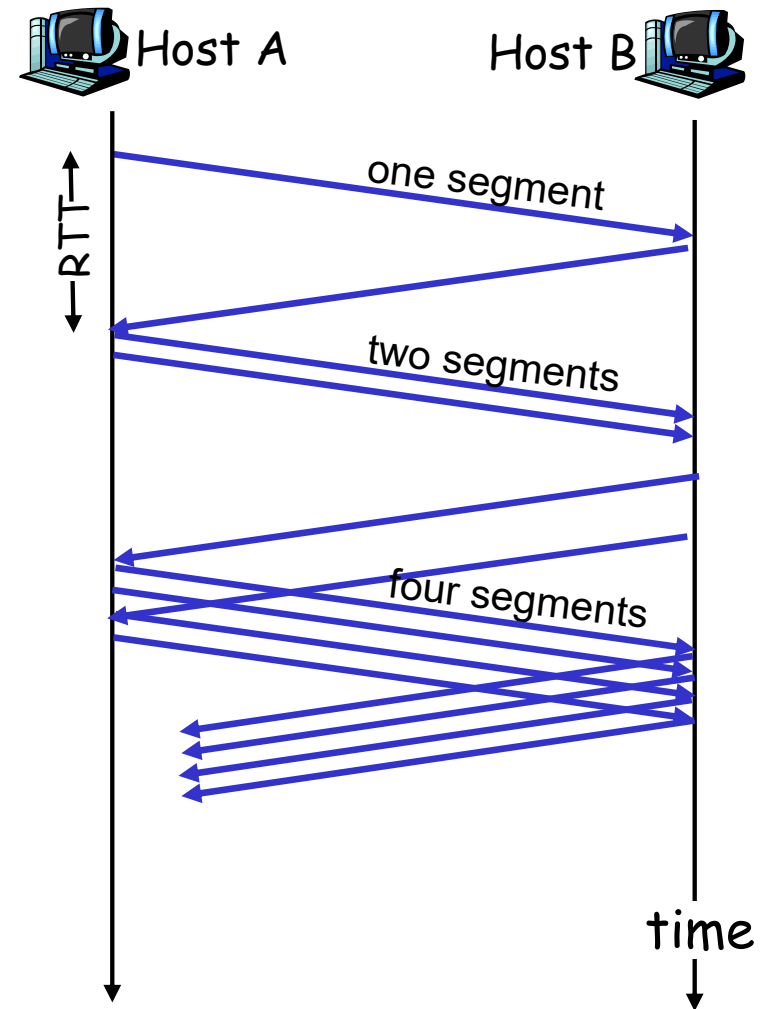
拥塞控制算法

- 慢启动
- 拥塞避免
- 快速恢复

- 超时事件后的保守机制

TCP Slow Start

- ❑ When connection begins, increase rate exponentially until first loss event:
 - double `CWin` every RTT
 - done by incrementing `CWin` for every ACK received
- ❑ Summary: initial rate is slow but ramps up exponentially fast
- ❑ 初始速率很慢但增长很快



Refinement

Q: When should the exponential increase switch to linear?

什么时候从指数型增加到线性增加？

也即从 慢启动 到 拥塞避免

□ 超过 **Threshold** 阈值---很重要的变量

Refinement

其他情况？

□ At loss event

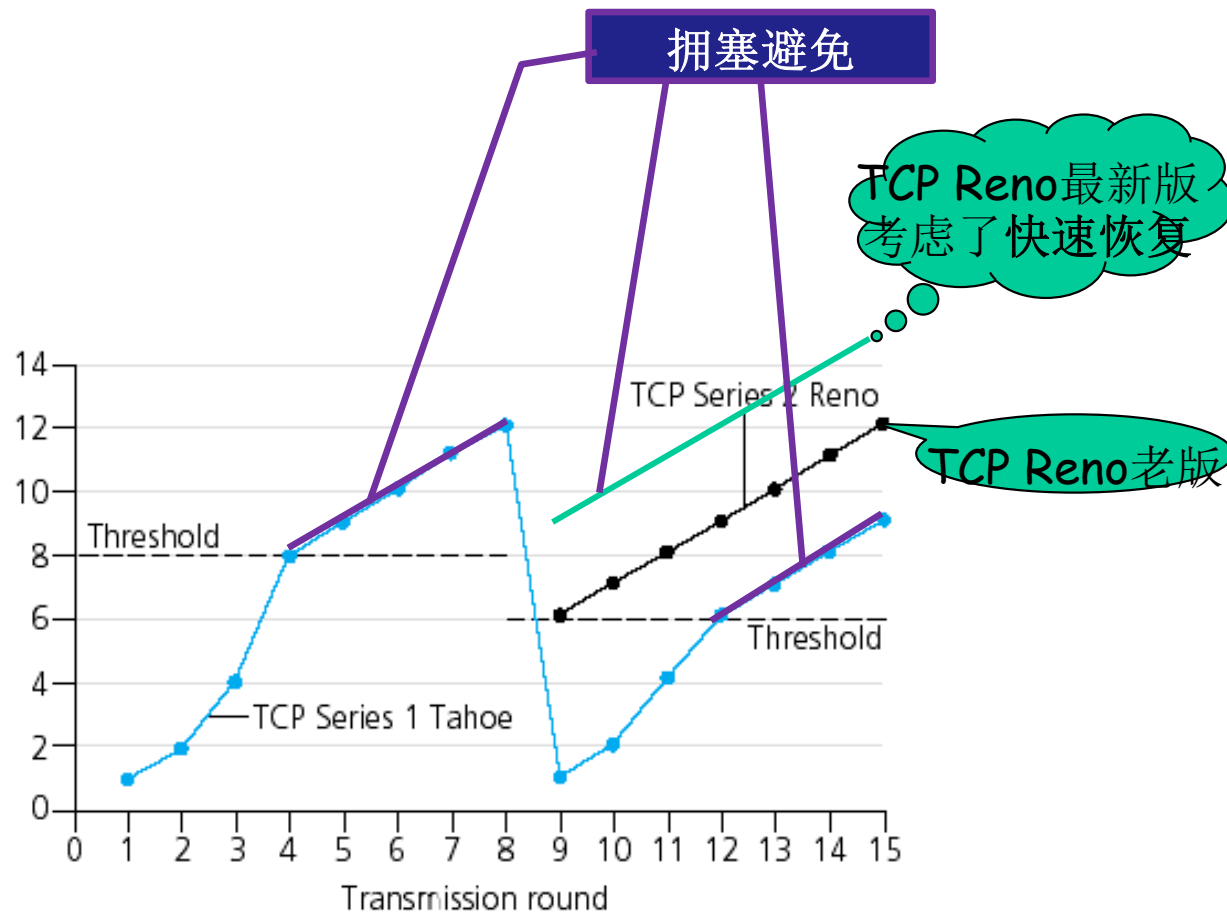
○ After timeout event

- CWin is set to 1 MSS
- window then grows exponentially
- to a threshold, then grows linearly 阈值会设为超时前 CWin的一半

○ After 3 duplicate acks

- Threshold is set to 1/2 of CWin just before loss event 阈值都设为超时前 Cwin的一半
- Cwin会设为(超时前 Cwin/2)+3

Refinement



TCP拥塞控制又被称为AIMD控制方式

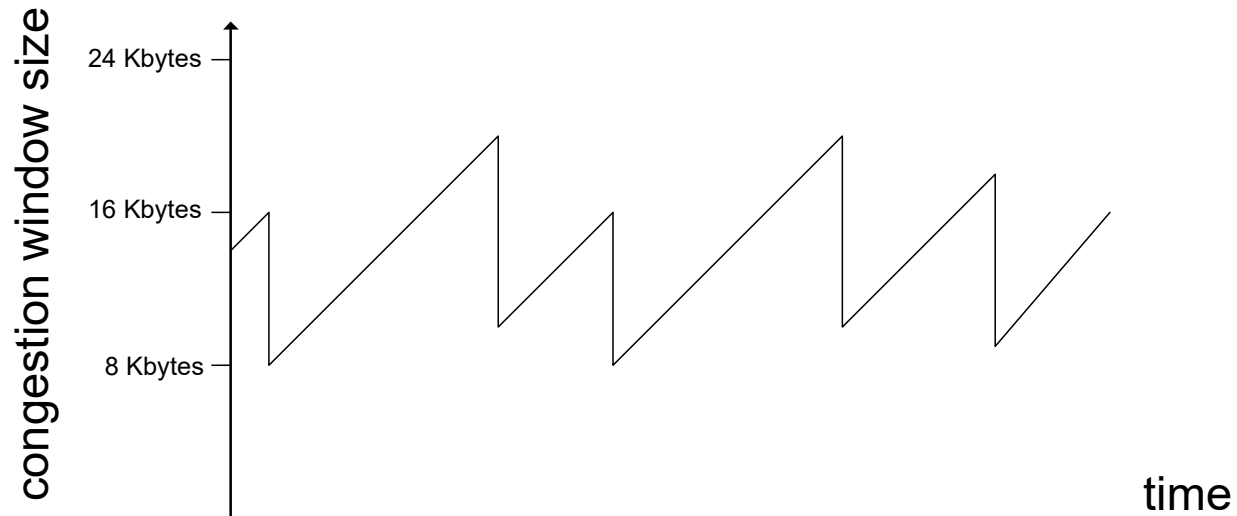
AIMD: Additive-Increase, Multiplicative-Decrease

“加性增，乘性减”的控制方式

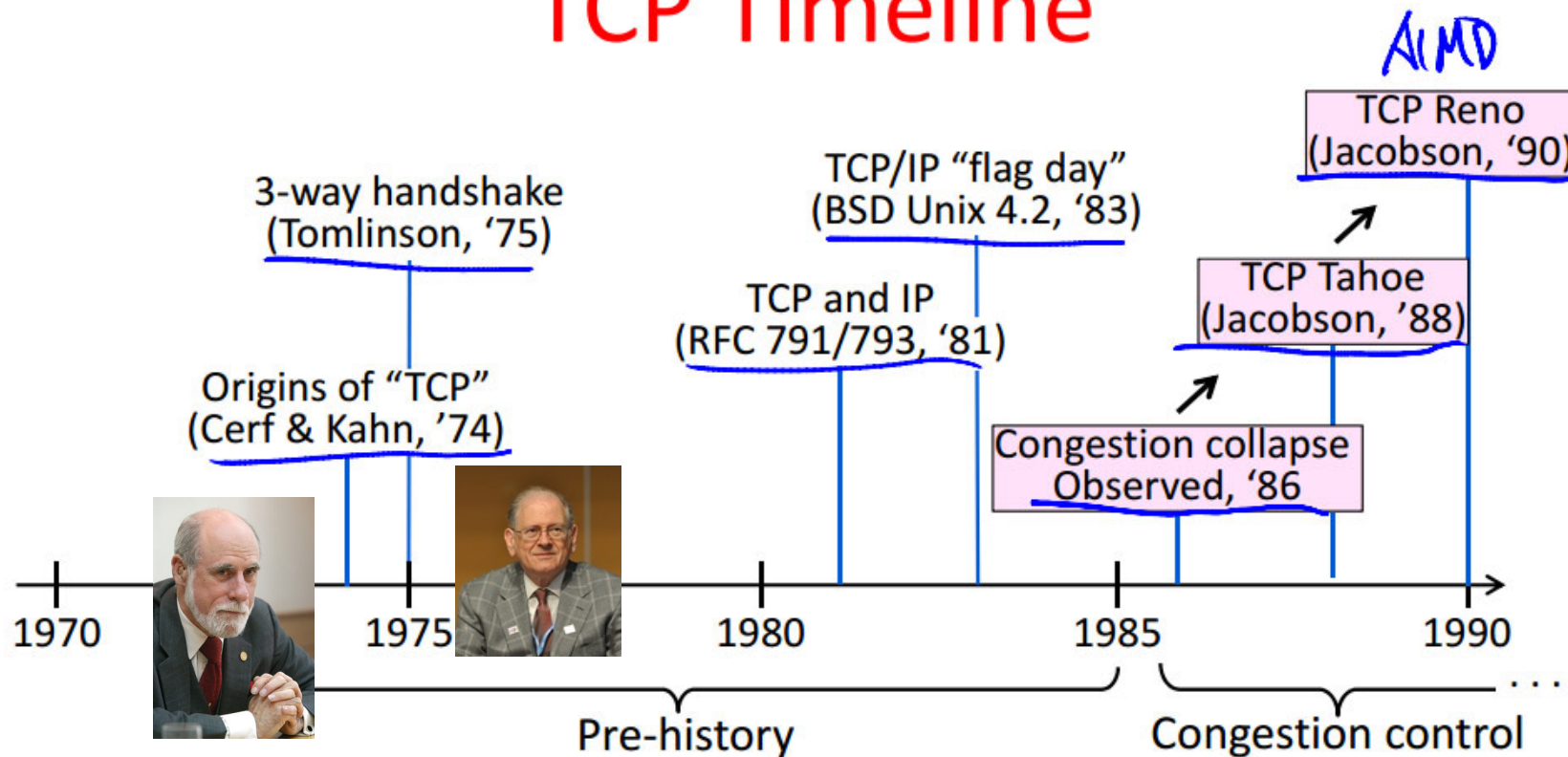
TCP congestion control: additive increase, multiplicative decrease

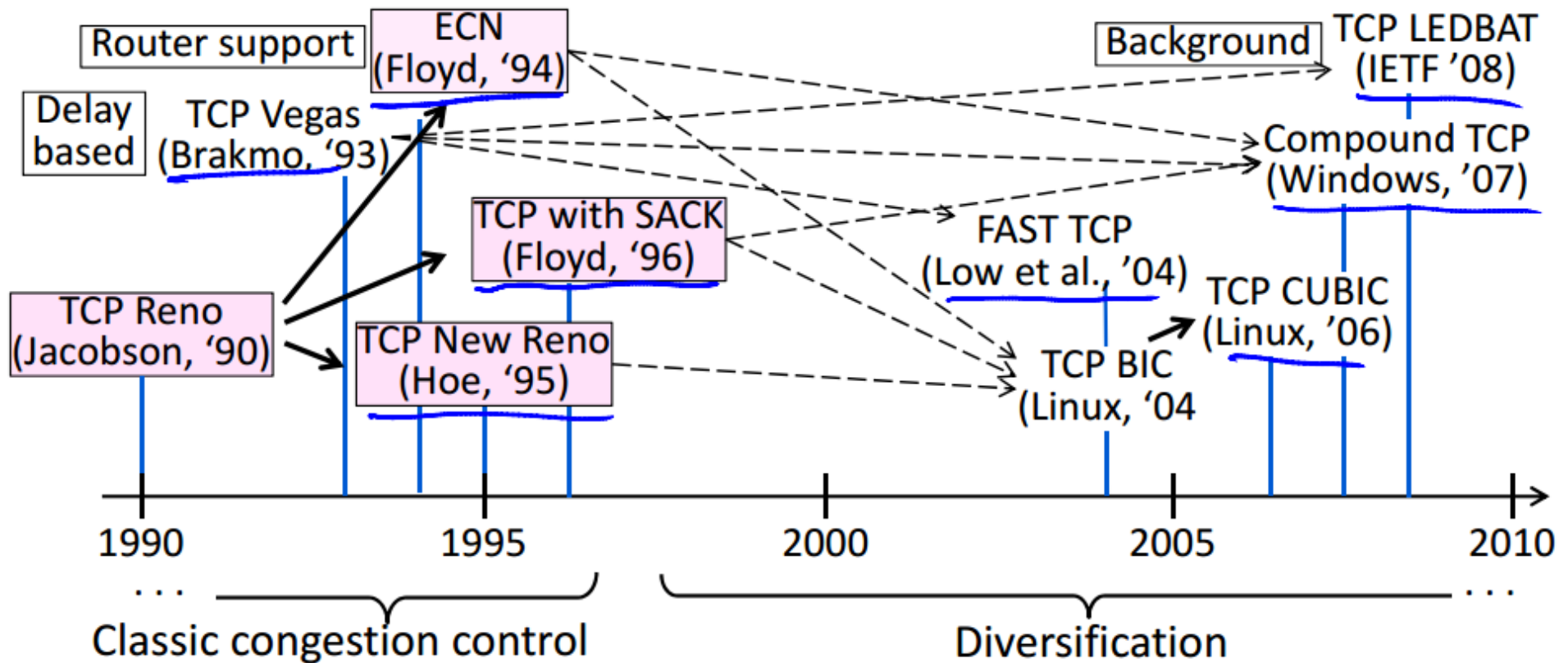
- **Approach:** increase transmission rate (window size), probing for usable bandwidth, until loss occurs
 - **additive increase:** increase **CWin** by 1 MSS every RTT until loss detected
 - **multiplicative decrease:** cut **CWin** in half after loss (大概)

Saw tooth behavior: probing for bandwidth



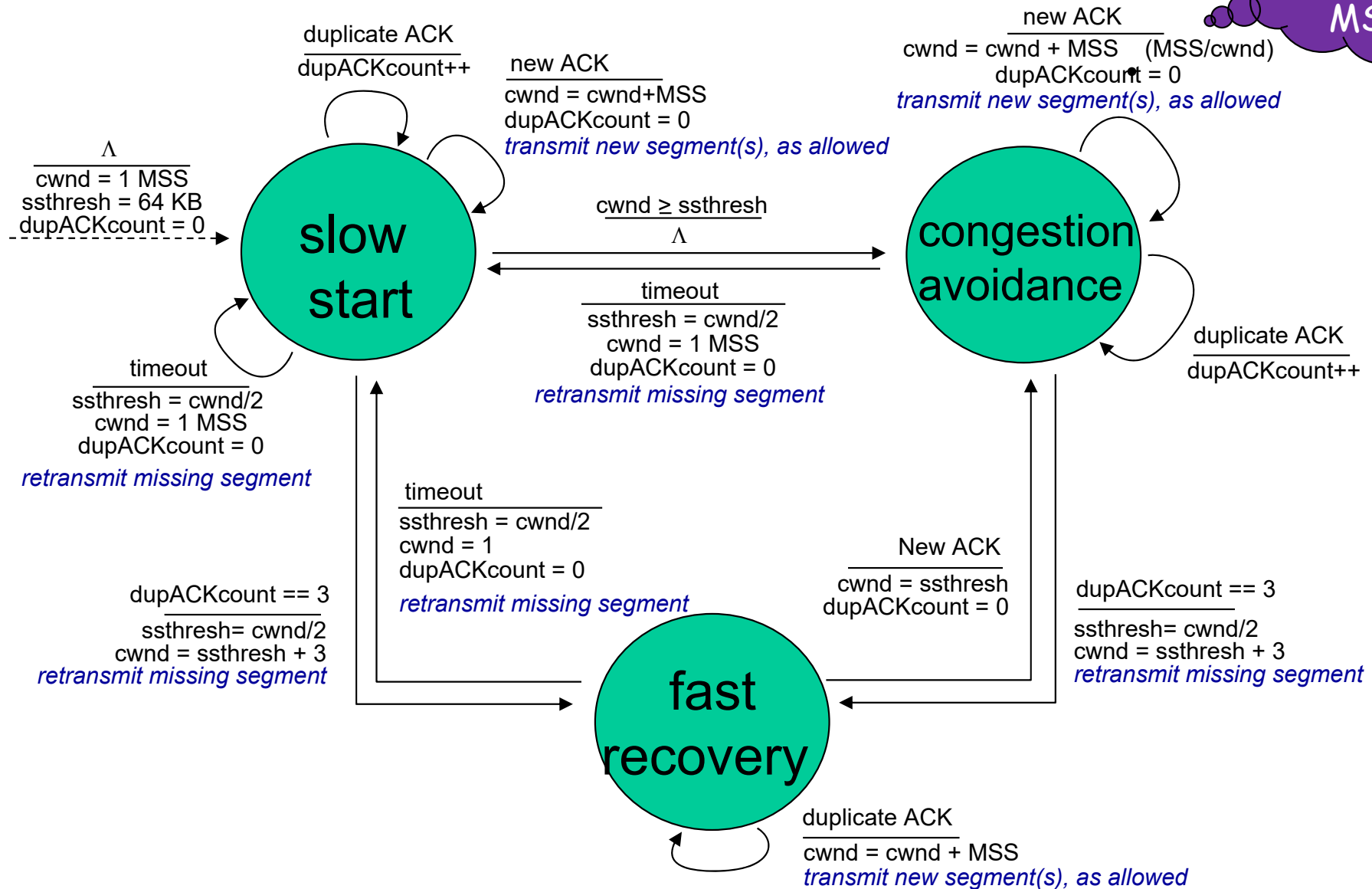
TCP Timeline





Summary: TCP Congestion Control

意思是每个
RTT只增长1个
MSS



作业

□ P40

P40. 考虑图 3-58。假设 TCP Reno 是一个经历如上所示行为的协议，回答下列问题。在各种情况中，简要地论证你的回答。

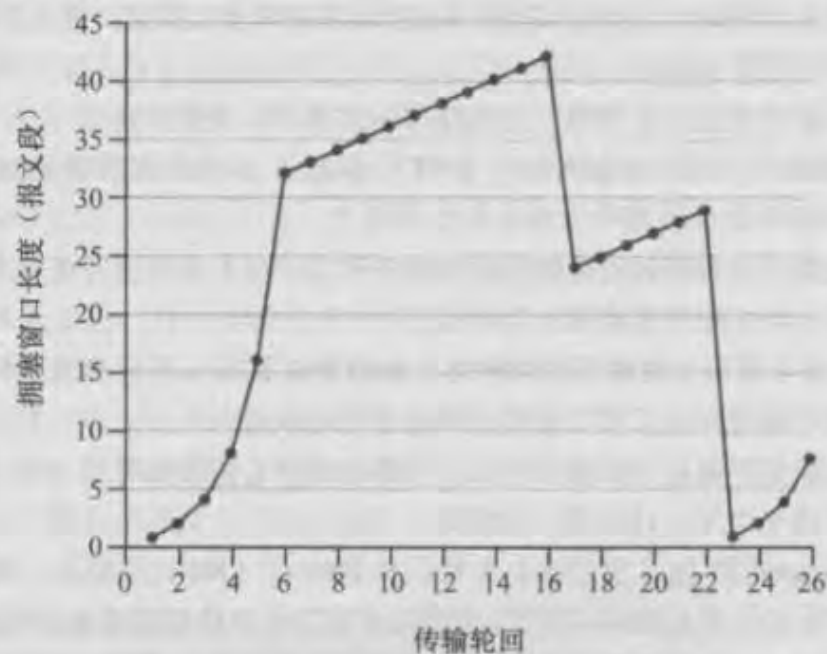


图 3-58 TCP 窗口长度作为时间的函数

- 指出 TCP 慢启动运行时的时间间隔。
- 指出 TCP 拥塞避免运行时的时间间隔。
- 在第 16 个传输轮回之后，报文段的丢失是根据 3 个冗余 ACK 还是根据超时检测出来的？
- 在第 22 个传输轮回之后，报文段的丢失是根据 3 个冗余 ACK 还是根据超时检测出来的？
- 在第 1 个传输轮回里， $ssthresh$ 的初始值设置为多少？
- 在第 18 个传输轮回里， $ssthresh$ 的值设置为多少？
- 在第 24 个传输轮回里， $ssthresh$ 的值设置为多少？
- 在哪个传输轮回内发送第 70 个报文段？
- 假定在第 26 个传输轮回后，通过收到 3 个冗余 ACK 检测出有分组丢失，拥塞的窗口长度和 $ssthresh$ 的值应当是多少？
- 假定使用 TCP Tahoe（而不是 TCP Reno），并假定在第 16 个传输轮回收到 3 个冗余 ACK。在第 19 个传输轮回， $ssthresh$ 和拥塞窗口长度是什么？
- 再次假设使用 TCP Tahoe，在第 22 个传输轮回有一个超时事件。从第 17 个传输轮回 to 第 22 个传输轮回（包括这两个传输轮回），一共发送了多少分组？

□ 提醒：这个题目涉及到了TCP拥塞控制的Tahoe和Reno（新版）。

(1)Tahoe遇到超时和3个冗余的ACK是一样的处理方式：
拥塞窗口=1， 阈值=上个轮回拥塞窗口的1/2

(2)Reno（新版）遇到超时， 阈值=上个轮回拥塞窗口的1/2, 拥塞窗口=1; 遇到3个冗余的ACK后, 阈值=上个轮回拥塞窗口的1/2, 拥塞窗口=(上个轮回拥塞窗口的1/2)+3

(3)k题是延续了j题的题意的, 也就是说, k题里,16个传输轮回里收到了3个冗余的ACK, 22个传输轮回里有一个超时事件

(4)遇到除2后有小数的情况， 向下取整

谢谢大家！